

MyBatis 基础

注：该笔记参考来源如下：

哔哩哔哩动力节点MyBatis教程：https://www.bilibili.com/video/BV1JP4y1Z73S/?spm_id_from=333.788.videopod.episodes&vd_source=c899d1ec12749d974be5871ce6410cd7

动力节点教程笔记：<https://www.yuque.com/dujubin/ltckqu/pozck9?singleDoc#PXV5K>

MyBatis官网、Gemini AI大模型以及本人的深度思考。

初始 MyBatis

初始框架

框架的概述

框架是一种可复用的、半成品的软件结构，它定义了应用的架构骨架，开发者通过"填充"特定内容来完成具体功能。

框架 = 架构模式的具体实现 + 控制反转机制 + 可扩展的插件体系

在文献中看到的framework被翻译为框架

框架一般都以jar包的形式存在。(jar包中有class文件以及各种配置文件等。)

框架其实就是对通用代码的封装，提前写好了一堆接口和类，我们可以在做项目的时候直接引入这些接口和类（引入框架），基于这些现有的接口和类

进行开发，可以大大提高开发效率。

它不是简单的工具集合，而是一种结构化思考的产物——将某类问题的最佳实践固化下来，让开发者专注于业务创新而非基础设施。

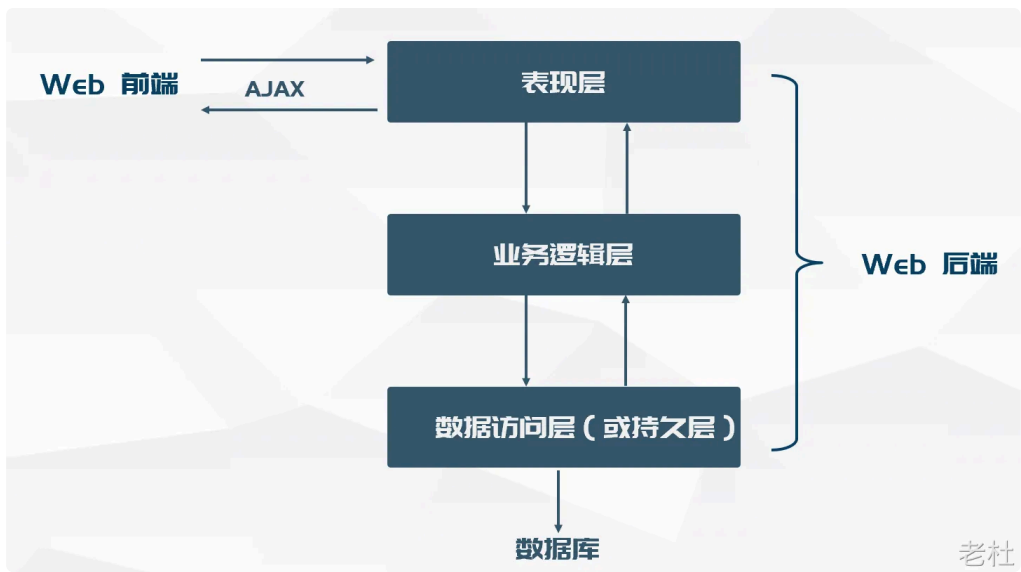
框架的优势

1. 标准化：统一规范，避免厂商锁定。
2. 分层架构：职责清晰，便于团队协作。
3. 容器化管理：框架接管对象生命周期。
4. 企业级能力：内置安全、事务、并发等。
5. 生态成熟：丰富的解决方案和社区支持。

当前主流框架

- SSM三大框架：Spring + SpringMVC + MyBatis
- SpringBoot
- SpringMVC
- SpringCloud

三层架构



- 表现层（UI）：直接跟前端打交互（一是接收前端ajax请求，二是返回json数据给前端）
- 业务逻辑层（BLL）：一是处理表现层转发过来的前端请求（也就是具体业务），二是将从持久层获取的数据返回到表现层。
- 数据访问层（DAL）：直接操作数据库完成CRUD，并将获得的数据返回到上一层（也就是业务逻辑层）。
- Java持久层框架：
 - MyBatis
 - Hibernate（实现了JPA规范）
 - jOOQ
 - Guzz
 - Spring Data（实现了JPA规范）
 - ActiveJDBC

MyBatis 简介

JDBC 的不足

示例代码1：

```
● ● ●
// .....
// sql 语句写死在java程序中
String sql = "insert into t_user(id,idCard,username,password,birth,gender,email,city,street,zipcode,phone,grade)
values(?,?,?,?,?,?,?,?,?,?,?,?,?)";
PreparedStatement ps = conn.prepareStatement(sql);
// 繁琐的赋值：思考一下，这种有规律的代码能不能通过反射机制来做自动化。
ps.setString(1, "1");
ps.setString(2, "123456789");
ps.setString(3, "zhangsan");
ps.setString(4, "123456");
ps.setString(5, "1980-10-11");
ps.setString(6, "男");
ps.setString(7, "zhangsan@126.com");
ps.setString(8, "北京");
ps.setString(9, "大兴区凉水河二街");
ps.setString(10, "1000000");
ps.setString(11, "16398574152");
```

```
ps.setString(12, "A");
// 执行SQL
int count = ps.executeUpdate();
// .....
```

示例代码2:

```

● ● ●
// .....
// sql 语句写在java程序中
String sql = "select id,idCard,username,password,birth,gender,email,city,street,zipcode,phone,grade from t_user";
PreparedStatement ps = conn.prepareStatement(sql);
ResultSet rs = ps.executeQuery();
List<User> userList = new ArrayList<>();
// 思考以下循环中的所有代码是否可以使用反射进行自动化封装。
while(rs.next()){
    // 获取数据
    String id = rs.getString("id");
    String idCard = rs.getString("idCard");
    String username = rs.getString("username");
    String password = rs.getString("password");
    String birth = rs.getString("birth");
    String gender = rs.getString("gender");
    String email = rs.getString("email");
    String city = rs.getString("city");
    String street = rs.getString("street");
    String zipcode = rs.getString("zipcode");
    String phone = rs.getString("phone");
    String grade = rs.getString("grade");
    // 创建对象
    User user = new User();
    // 给对象属性赋值
    user.setId(id);
    user.setIdCard(idCard);
    user.setUsername(username);
    user.setPassword(password);
    user.setBirth(birth);
    user.setGender(gender);
    user.setEmail(email);
    user.setCity(city);
    user.setStreet(street);
    user.setZipcode(zipcode);
    user.setPhone(phone);
    user.setGrade(grade);
    // 添加到集合
    userList.add(user);
}
// .....
```

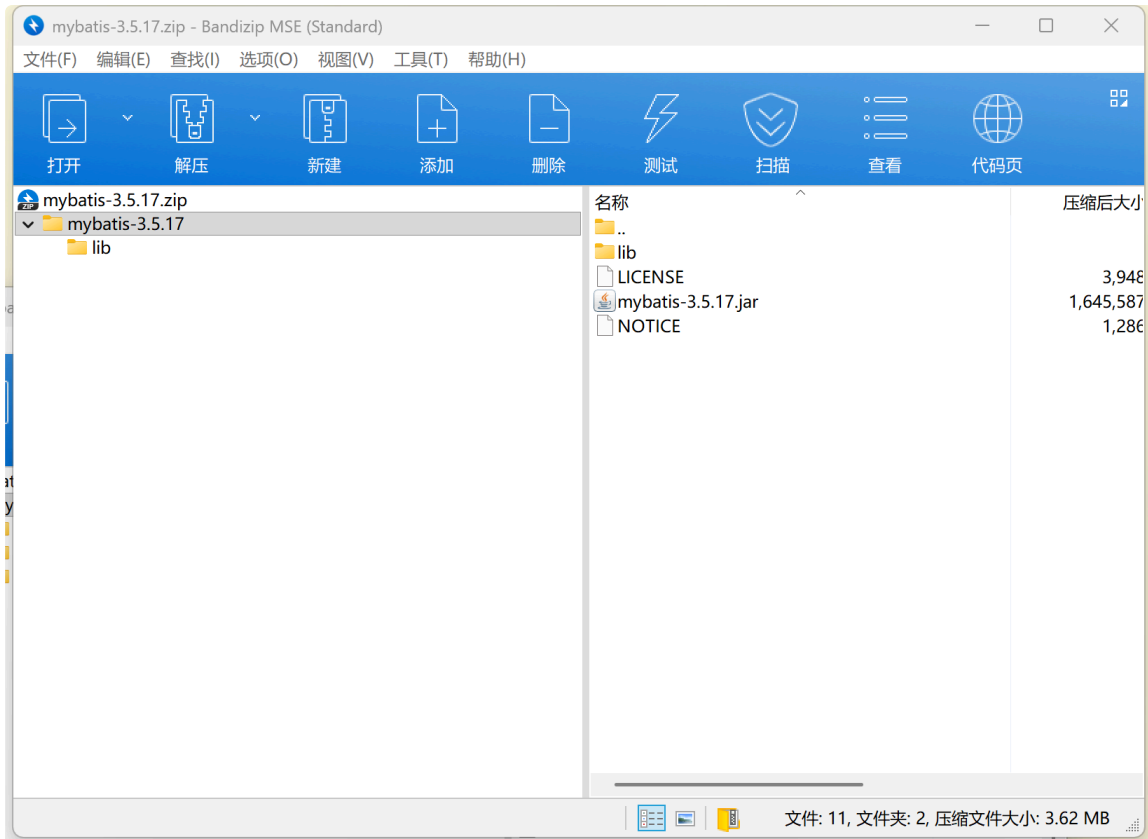
- JDBC 的不足
 - 大量重复的模板代码
 - 硬编码，缺乏灵活性。
 - 手动参数绑定与结果映射，反人类的体力活。

MyBatis 概述

MyBatis 是一个一流的持久层框架。它的核心目标只有一个：让 Java 程序和关系型数据库之间的交互变得极其简单和优雅。

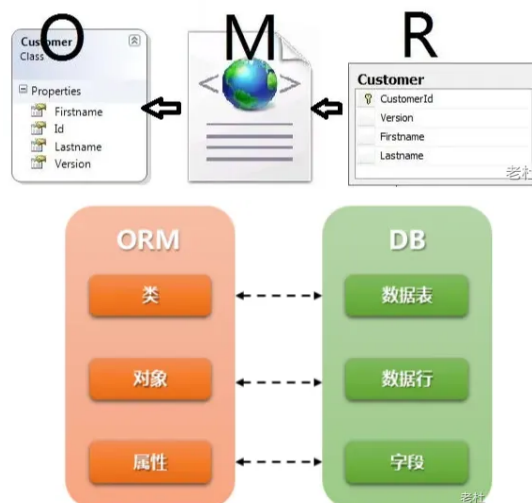
- MyBatis本质上就是对JDBC的封装，通过MyBatis完成CRUD。
- MyBatis在三层架构中负责持久层的，属于持久层框架。
- MyBatis的发展历程：

- MyBatis本是apache的一个开源项目iBatis，2010年这个项目由apache software foundation迁移到了google code，并且改名为MyBatis。2013年11月迁移到Github。
 - iBATIS一词来源于“internet”和“abatis”的组合，是一个基于Java的持久层框架。iBATIS提供的持久层框架包括SQL Maps和Data Access Objects（DAOs）。
- 打开mybatis代码可以看到它的包结构中包含：lib



- ORM：对象关系映射
 - O（Object）：Java虚拟机中的Java对象
 - R（Relational）：关系型数据库
 - M（Mapping）：将Java虚拟机中的Java对象映射到数据库表中一行记录，或是将数据库表中一行记录映射成Java虚拟机中的一个Java对象。

- ORM图示



- MyBatis属于半自动化ORM框架。
 - Hibernate属于全自动化的ORM框架。
- MyBatis框架特点：

- 支持定制化 SQL、存储过程、基本映射以及高级映射
- 避免了几乎所有的 JDBC 代码中手动设置参数以及获取结果集
- 支持XML开发，也支持注解式开发。【为了保证sql语句的灵活，所以mybatis大部分是采用XML方式开发。】
- 将接口和 Java 的 POJOs(Plain Ordinary Java Object，简单普通的Java对象)映射成数据库中的记录
- 体积小好学：两个jar包，两个XML配置文件。
- 完全做到sql解耦合。
- 提供了基本映射标签。
- 提供了高级映射标签。
- 提供了XML标签，支持动态SQL的编写。

MyBatis 入门程序

版本

软件版本

- IntelliJ IDEA：2023.2 (Ultimate Edition)
- Navicat：17.3.8
- MySQL数据库：8.4.8
- MySQL密码：20060329

组件版本：

- MyBatis：3.5.19
- MySQL驱动：8.4.0
- JDK：Java21
- Maven 3.9.12
- JUnit：4.13.2
- Logback：1.5.6

MyBatis 下载

- 从github上下载，地址：<https://github.com/mybatis/mybatis-3>

 README.md

MyBatis SQL Mapper Framework for Java

 \$3.72M Cost



MyBatis

The MyBatis SQL mapper framework makes it easier to use a relational database with object-oriented applications. MyBatis couples objects with stored procedures or SQL statements using an XML descriptor or annotations. Simplicity is the biggest advantage of the MyBatis data mapper over object relational mapping tools.

Essentials

- [See the docs](#)
- [Download Latest](#)
- [Download Snapshot](#)

老杜

mybatis-3.5.17

Bugs:

- `VendorDatabaseIdProvider#getDatabaseId()` should return product name when properties is empty [#3297](#)
- Update `NClobTypeHandler` to use methods for national character set [#3298](#)

Enhancements:

- Allow `DefaultSqlSessionFactory` to provide a custom `SqlSession` [#3128](#)

There also are many pull requests for document updates and code improvements!

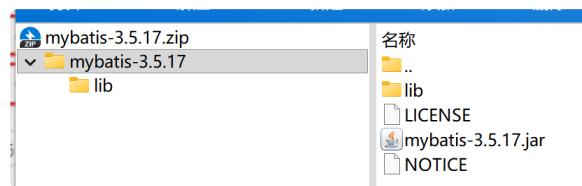
Please see the full changelog: [mybatis-3.5.16...mybatis-3.5.17](#)

▼ Assets 3

 mybatis-3.5.17.zip	MyBatis框架	3.62 MB	Nov 27, 2024
 Source code (zip)	mybatis 框架源码--Windows压缩格式		Nov 26, 2024
 Source code (tar.gz)	mybatis 框架源码--Linux压缩格式		Nov 26, 2024

 6  4 8 people reacted

- 将框架以及框架的源码都下载下来，下载框架后解压，打开mybatis目录



- 通过以上解压可以看到，框架一般都是以jar包的形式存在。我们的mybatis课程使用maven，所以这个jar我们不需要。
- 官方手册需要。

可以无需在Github力下载MyBatis，直接使用Maven引入。

MyBatis入门程序开发步骤

- 写代码前准备：
 - 准备数据库表：汽车表t_car，字段包括：
 - id: 主键（自增）【bigint】
 - car_num: 汽车编号【varchar】
 - brand: 品牌【varchar】
 - guide_price: 厂家指导价【decimal类型，专门为财务数据准备的类型】
 - produce_time: 生产时间【char，年月日即可，10个长度，'2022-10-11'】
 - car_type: 汽车类型（燃油车、电车、氢能源）【varchar】
 - 使用navicat for mysql工具建表
- 在navicat 中
 - 连接MySQL

常规高级数据库SSLSSHHTTP备注

Navicat

数据库

连接名称:

本地MySQL

主机:

localhost

端口:

3306

用户名:

root

密码:

●●●●●●

👁

☒ 保存密码

测试连接

URI...

上一步

确定

取消

Navicat Premium

文件 编辑 查看 收藏夹 工具 窗口 帮助

连接 新建查询 表 视图 函数 用户 其它 查询 备份 自动运行 模型 BI

我的连接

本地MySQL

- information_schema
- mysql
- performance_schema
- sys

对象

- 打开表 设计表 新建表 删除表 导入向导 导出向导
- 名称 行 数据长度 引擎 修

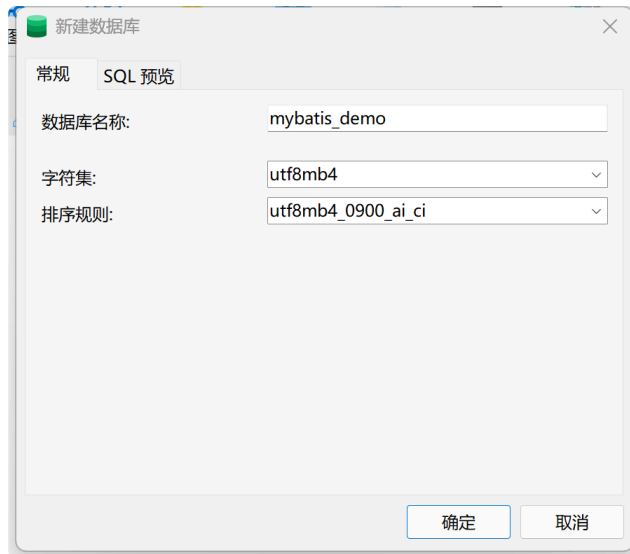
本地MySQL

- MySQL
- 共享
- 活动配置文件
- 主要配置文件
- 服务器版本 8.4.8
- 会话 1
- 主机 localhost
- 端口 3306
- 用户名 root
- 设置位置 C:\Users\jack\Documents\N
- 客户端字符集 自动
- 备注

4 数据库 本地MySQL

- 新建数据库

- 在左侧栏本地MySQL上点击右键，选择新建数据库。
- 数据库名：mybatis_demo
- 字符集：选择 utf8mb4
- 排序规则：选择 utf8mb4_0900_ai_ci



- 创建汽车表 t_car
 - 选中 mybatis_demo 数据库，点击顶部工具栏的新建查询。
 - 输入建表 SQL 语句：

```
CREATE TABLE `t_car` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `car_num` varchar(255) DEFAULT NULL COMMENT '汽车编号',
  `brand` varchar(255) DEFAULT NULL COMMENT '汽车品牌',
  `guide_price` decimal(10,2) DEFAULT NULL COMMENT '厂家指导价',
  `produce_time` char(10) DEFAULT NULL COMMENT '生产日期',
  `car_type` varchar(255) DEFAULT NULL COMMENT '汽车类型(燃油车/电车/氢能源)',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='汽车表';
```

名称	类型	长度	小数点	不是 null	虚拟	键	注释
id	bigint			☒	☐	1	主键
car_num	varchar	255		☐	☐		汽车编号
brand	varchar	255		☐	☐		汽车品牌
guide_price	decimal	10	2	☐	☐		厂家指导价
produce_time	char	10		☐	☐		生产日期
car_type	varchar	255		☐	☐		汽车类型(燃油车/电车/氢能源)

- 点击查询窗口左上角的运行
- 找到表节点，右键 -> 刷新



- 向t_car表中插入两条数据，如下：

```
INSERT INTO `t_car` (`id`, `car_num`, `brand`, `guide_price`, `produce_time`, `car_type`)
VALUES
(1, '100', '宝马520Li', 41.00, '2022-09-01', '燃油车'),
(2, '101', '奔驰E300L', 54.00, '2022-08-01', '电车');
```

对象 t_car @mybatis_demo (本地MySQL) -...

表配置文件 开始事务 单元格编辑器 筛选 & 排序 列 数据分析 导入 导出 数据生成 创建 BI 工作区

id # bigint	car_num # varchar(255)	brand # varchar(255)	guide_price # decimal(10,2)	produce_time # char(10)	car_type # varchar(255)
1	100	宝马520Li	41.00	2022-09-01	燃油车
2	101	奔驰E300L	54.00	2022-08-01	电车

- 在 IDEA 中创建 Maven 项目

新建项目

新建项目
空项目

生成器

Maven Archetype
Jakarta EE
Spring Initializr
JavaFX
Quarkus
Micronaut
Ktor
Kotlin Multiplatform
Compose for Desktop
HTML 文件
React
Express
Angular CLI
IDE 插件
Android
Vue.js
Vite

名称: my_mybatis_01

位置: D:\Java_mybatis_study
项目将在以下位置创建: D:\Java_mybatis_study\my_mybatis_01
☐ 创建 Git 仓库

语言: ☒ Java ☐ Kotlin ☐ Groovy ☐ JavaScript +

构建系统: ☐ IntelliJ ☒ Maven ☐ Gradle

JDK: 21 Oracle OpenJDK version 21.0.10

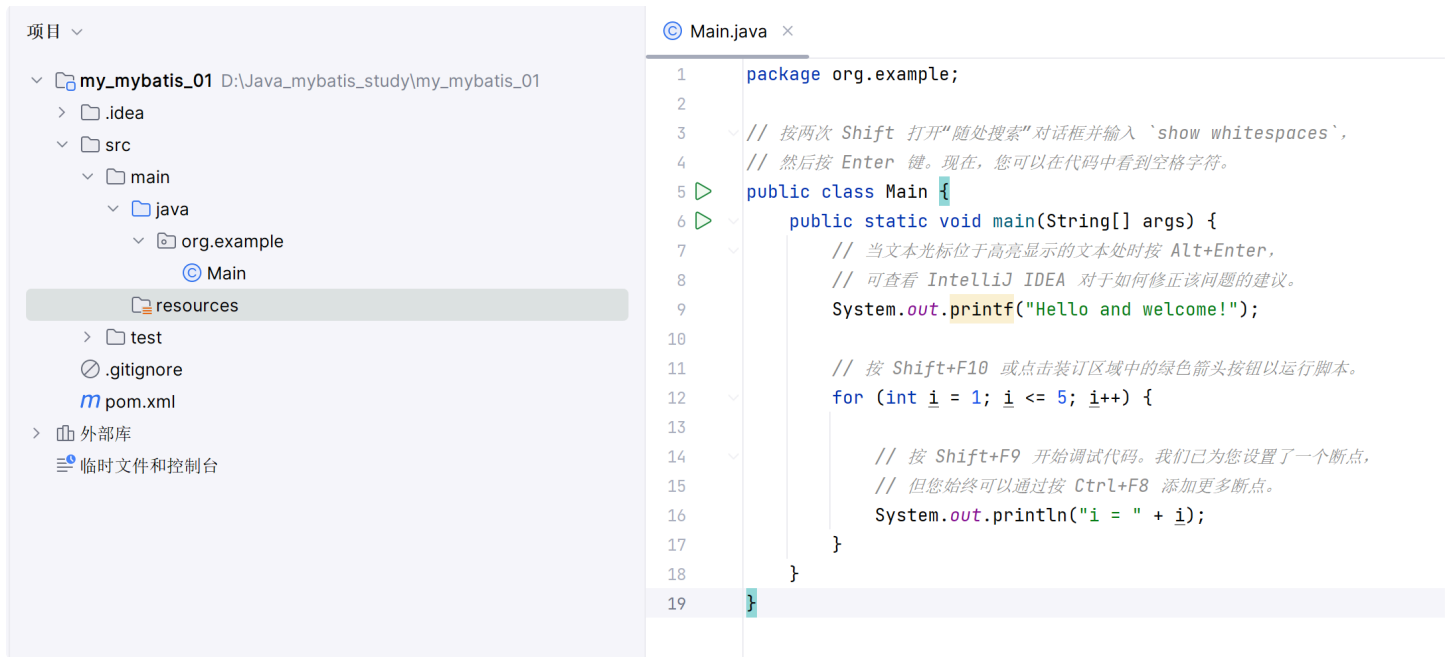
☒ 添加示例代码
☒ 使用入门提示生成代码

高级设置

组 ID: org.example

工件 ID: my_mybatis_01

创建(C) 取消



- Maven 的标准目录结构
 - src/main/java: 专门用来放以后编写的所有 Java 源代码 (.java 文件)。
 - src/main/resources: 专门用来放所有的配置文件 (如XML文件)。
 - pom.xml: 位于项目根目录, 是 Maven 项目的核心配置文件。
- 编写 pom.xml
 - 默认的pom.xml代码

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>my_mybatis_01</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

</project>
```

- 现在, 需将 **MyBatis** 、 **MySQL 驱动** 两大组件的Maven 依赖 GAV 坐标, 配置至项目的依赖管理文件 (pom.xml)
- 在pom.xml里的 **<dependencies>** 到 **</dependencies>** 进行编写
- MyBatis 持久层框架

```
<dependency>
    <groupId>org.mybatis</groupId>
    <artifactId>mybatis</artifactId>
    <version>3.5.19</version>
</dependency>
```

- MySQL JDBC 数据库驱动

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.4.0</version>
</dependency>
```

- 刷新 Maven，引入依赖

Maven 创建 Java 项目时，默认的打包方式就是 **jar**；**war** 是 JavaWeb 的打包方式。因此没必要在 pom.xml 编写 jar 打包方式。

- 在 resources 根目录下编写核心配置文件 mybatis-config.xml
 - 告诉 MyBatis 我的 MySQL 数据库在哪里，密码是什么？

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
  PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
  "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
  <environments default="development">
    <environment id="development">
      <transactionManager type="JDBC"/>
      <dataSource type="POOLED">
        <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
        <!-- 数据库名、用户名、密码根据实际情况修改 -->
        <property name="url" value="jdbc:mysql://localhost:3306/你的数据库名?
useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai"/>
        <property name="username" value="root"/>
        <property name="password" value="你的密码"/>
      </dataSource>
    </environment>
  </environments>
  <mappers>
    <!-- 配置 SQL 映射文件路径，例如：<mapper resource="CarMapper.xml"/> -->
    <mapper resource="你的SQL映射文件路径"/>
  </mappers>
</configuration>
```

mybatis 核心配置文件的文件名不一定是 mybatis-config.xml，可以是其它名字。

mybatis 核心配置文件存放的位置也可以随意。这里选择放在 resources 根下，相当于放到了类的根路径下。

- 在 resources 根目录下新建 CarMapper.xml 配置文件
 - MyBatis 核心配置文件（**mybatis-config.xml**）中，尚未对 Mapper 映射文件（**CarMapper.xml**）进行注册配置。
 - CarMapper.xml 代码如下

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
  PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
  "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<!-- namespace 先随意写一个 -->
<mapper namespace="car">
  <!-- 查询所有汽车信息 -->
  <select id="selectAllCars" resultType="map">
    SELECT * FROM t_car
  </select>
```

```

<!-- 插入一条汽车信息 -->
<insert id="insertCar">
    insert into t_car
        (id, car_num, brand, guide_price, produce_time, car_type)
    values
        (null, '102', '丰田未来', 40.30, '2014-10-05', '氢能源')
</insert>

</mapper>

```

- 因此需要把这个 Mapper 文件注册到核心配置文件里（pom.xml）。

```

</environments>
    <mappers>
        <mapper resource="CarMapper.xml"/>
    </mappers>
</configuration>

```

sql语句最后结尾可以不写“;”

CarMapper.xml文件的名字不是固定的。可以使用其它名字。

CarMapper.xml文件的位置也是随意的。这里选择放在resources根下，相当于放到了类的根路径下。

- 在Java文件夹里的包编写MyBatisIntroductionTest代

```

package org.example;

import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;

import java.io.InputStream;

public class MyBatisIntroductionTest {
    public static void main(String[] args) {
        // 1. 创建SqlSessionFactoryBuilder对象
        SqlSessionFactoryBuilder sqlSessionFactoryBuilder = new SqlSessionFactoryBuilder();
        // 2. 创建SqlSessionFactory对象（读取你 resources 根目录下的配置文件）
        InputStream is = Thread.currentThread().getContextClassLoader().getResourceAsStream("mybatis-config.xml");
        SqlSessionFactory sqlSessionFactory = sqlSessionFactoryBuilder.build(is);
        // 3. 创建SqlSession对象（向工厂要一个数据库连接会话）
        SqlSession sqlSession = sqlSessionFactory.openSession();
        // 4. 执行sql（告诉MyBatis去执行CarMapper.xml里面那个id为"insertCar"的SQL语句）
        int count = sqlSession.insert("insertCar");
        System.out.println("插入几条数据: " + count);
        // 5. 提交（mybatis默认采用的事务管理器是JDBC，默认是不自动提交的，必须手动提交才能真正写入数据库）
        sqlSession.commit();
        // 6. 关闭资源（释放数据库连接）
        sqlSession.close();
    }
}

```

默认采用的事务管理器是：JDBC。JDBC事务默认是不提交的，需要手动提交。

- 运行程序，查看运行结果，以及数据库表中的数据


```
D:\Java21\bin\java.exe "
```

插入几条数据：1

进程已结束，退出代码为 0

对象 t_car @mybatis_demo (本地MySQL) -...					
表配置文件 开始事务 单元格编辑器 筛选 & 排序 列 数据分析 导入 导出 数据生成					
id # bigint	car_num varchar(255)	brand varchar(255)	guide_price decimal(10,2)	produce_time char(10)	car_type varchar(255)
1	100	宝马520Li	41.00	2022-09-01	燃油车
2	101	奔驰E300L	54.00	2022-08-01	电车
3	102	丰田未来	40.30	2014-10-05	氢能源

- 关于MyBatis核心配置文件的名字和路径详解
 - mybatis核心配置文件的名字是随意的，存放路径也是随意的。
 - 虽然mybatis核心配置文件的名字不是固定的，但通常该文件的名字叫做：mybatis-config.xml
 - 虽然mybatis核心配置文件的路径不是固定的，但通常该文件会存放到**类路径**当中，这样让项目的移植更加健壮。
 - 在mybatis中提供了一个类：Resources【org.apache.ibatis.io.Resources】，该类可以从类路径当中获取资源，我们通常使用它来获取输入流InputStream，代码如下

// 这种方式只能从类路径当中获取资源，也就是说mybatis-config.xml文件需要在类路径下。

```
InputStream is = Resources.getResourceAsStream("mybatis-config.xml");
```

MyBatis 第一个比较完整的代码写法

```
package org.example;

import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;

import java.io.InputStream;

public class MyBatisIntroductionTest {
    public static void main(String[] args) {

        SqlSession sqlSession = null;

        try {
            // 创建SqlSessionFactory构建器
            SqlSessionFactoryBuilder sqlSessionFactoryBuilder = new SqlSessionFactoryBuilder();
            // 加载MyBatis核心配置文件
            InputStream is = Thread.currentThread().getContextClassLoader().getResourceAsStream("mybatis-config.xml");
            // 构建SqlSessionFactory
            SqlSessionFactory sqlSessionFactory = sqlSessionFactoryBuilder.build(is);
            // 开启数据库会话
            sqlSession = sqlSessionFactory.openSession();
            // 执行插入操作，返回影响行数
        }
    }
}
```

```

        int count = sqlSession.insert("car.insertCar");
        System.out.println("成功插入了几条汽车数据: " + count);
        // 提交事务
        sqlSession.commit();
    } catch (Exception e) {
        // 异常时回滚事务
        if (sqlSession != null) {
            sqlSession.rollback();
            System.out.println("发生异常, 事务已回滚!");
        }
        e.printStackTrace();
    } finally {
        // 关闭会话释放资源
        if (sqlSession != null) {
            sqlSession.close();
        }
    }
}
}
}

```

- 运行后:

id # bigint	car_num # varchar(255)	brand # varchar(255)	guide_price # decimal(10,2)	produce_time # char(10)	car_type # varchar(255)
1	100	宝马520Li	41.00	2022-09-01	燃油车
2	101	奔驰E300L	54.00	2022-08-01	电车
3	102	丰田未来	40.30	2014-10-05	氢能源
4	102	丰田未来	40.30	2014-10-05	氢能源

引入JUnit

- JUnit是专门做单元测试的组件。
 - 在实际开发中, 单元测试一般是由我们Java程序员来完成的。
 - 我们要对我们自己写的每一个业务方法负责任, 要保证每个业务方法在进行测试的时候都能通过。
 - 测试的过程中涉及到两个概念:
 - 期望值
 - 实际值
 - 期望值和实际值相同表示测试通过, 期望值和实际值不同则单元测试执行时会报错。
- 这里引入JUnit是为了代替main方法。
- 使用JUnit步骤:
 - 第一步: 引入依赖

```

<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
</dependency>

```

- 第二步: 编写单元测试类【测试用例】, 测试用例中每一个测试方法上使用@Test注解进行标注。
 - 测试用例的名字以及每个测试方法的定义都是有规范的:
 - 测试用例的名字: XxxTest

- 测试方法声明格式：public void test业务方法名(){}

```
// 测试用例
public class CarMapperTest{

    // 测试方法
    @Test
    public void testInsert(){

    }

    @Test
    public void testUpdate(){

    }

}
```

- 第三步：可以在类上执行，也可以在方法上执行
 - 在类上执行时，该类中所有的测试方法都会执行。
 - 在方法上执行时，只执行当前的测试方法。
- 编写一个测试用例，来测试insertCar业务
- 在目录树中，这个类要建在 test 目录下的Java文件夹里：
 - 在 java 文件夹上右键 -> New -> Package，输入：org.example
 - 在新建的 org.example 包上右键 -> New -> Java Class，输入类名：CarMapperTest。

```
package org.example;

import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;
import org.junit.Test;

import java.util.List;

/**
 * 汽车数据访问层（Mapper）的单元测试
 */
public class CarMapperTest {

    /**
     * 测试用例 1：测试插入汽车数据
     */
    @Test
    public void testInsertCar() {
        SqlSession sqlSession = null;
        try {
            // 1. 创建SqlSessionFactoryBuilder对象
            SqlSessionFactoryBuilder sqlSessionFactoryBuilder = new SqlSessionFactoryBuilder();
            // 2. 创建SqlSessionFactory对象（使用了更加简便的 Resources 工具类）
            SqlSessionFactory sqlSessionFactory =
sqlSessionFactoryBuilder.build(Resources.getResourceAsStream("mybatis-config.xml"));
            // 3. 创建SqlSession对象
            sqlSession = sqlSessionFactory.openSession();

            // 4. 执行SQL（建议加上 namespace: car.insertCar）
            int count = sqlSession.insert("car.insertCar");
            System.out.println("更新了几条记录: " + count);

            // 5. 提交事务（增删改必须提交）
            sqlSession.commit();
        } catch (Exception e) {
```

```

        // 回滚事务，保证数据一致性
        if (sqlSession != null) {
            sqlSession.rollback();
            System.out.println("发生异常，事务已回滚！");
        }
        e.printStackTrace();
    } finally {
        // 6. 释放资源
        if (sqlSession != null) {
            sqlSession.close();
        }
    }
}

/**
 * 测试用例 2：测试查询所有汽车数据
 * 这是 JUnit 的魅力所在，它和上面那个方法互不干扰，可以单独运行
 */
@Test
public void testSelectAllCars() {
    SqlSession sqlSession = null;
    try {
        SqlSessionFactoryBuilder sqlSessionFactoryBuilder = new SqlSessionFactoryBuilder();
        SqlSessionFactory sqlSessionFactory =
sqlSessionFactoryBuilder.build(Resources.getResourceAsStream("mybatis-config.xml"));
        sqlSession = sqlSessionFactory.openSession();

        // 执行查询 SQL（调用我们在 CarMapper.xml 里写的 selectAllCars）
        // 注意：因为 resultType="map"，所以查出来的是 List<Map>
        List<Object> cars = sqlSession.selectList("car.selectAllCars");

        System.out.println("==== 查询到的汽车列表 ====");
        for (Object car : cars) {
            System.out.println(car);
        }

    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        if (sqlSession != null) {
            sqlSession.close();
        }
    }
}
}

```

如果点击 testInsertCar() 旁边的箭头，它就只执行插入代码。

如果点击 testSelectAllCars() 旁边的箭头，它就只执行查询代码。

如果点击类名 CarMapperTest 旁边的箭头，它会把这两个方法按顺序全部测试一遍。

- 执行单元测试，查看数据库表的变化：

运行 CarMapperTest.testInsertCar x

CarMapperTest (org.exar 544毫秒) 测试已通过: 共 1 个测试 - 544毫秒

testInsertCar 544毫秒

D:\Java21\bin\java.exe ...
20:16:23.025 [main] DEBUG org.apache.ibatis.logging.LogFactory -- Logging initialized using 'class org.apache.ibatis.logging.slf
20:16:23.035 [main] DEBUG org.apache.ibatis.datasource.pooled.PooledDataSource -- PooledDataSource forcefully closed/removed all
20:16:23.035 [main] DEBUG org.apache.ibatis.datasource.pooled.PooledDataSource -- PooledDataSource forcefully closed/removed all
20:16:23.035 [main] DEBUG org.apache.ibatis.datasource.pooled.PooledDataSource -- PooledDataSource forcefully closed/removed all
20:16:23.035 [main] DEBUG org.apache.ibatis.datasource.pooled.PooledDataSource -- PooledDataSource forcefully closed/removed all
20:16:23.077 [main] DEBUG org.apache.ibatis.transaction.jdbc.JdbcTransaction -- Opening JDBC Connection
20:16:23.361 [main] DEBUG org.apache.ibatis.datasource.pooled.PooledDataSource -- Created connection 1043358826.
20:16:23.362 [main] DEBUG org.apache.ibatis.transaction.jdbc.JdbcTransaction -- Setting autocommit to false on JDBC Connection [
20:16:23.370 [main] DEBUG car.insertCar -- ==> Preparing: insert into t_car (id, car_num, brand, guide_price, produce_time, car
20:16:23.393 [main] DEBUG car.insertCar -- ==> Parameters:
20:16:23.395 [main] DEBUG car.insertCar -- <== Updates: 1
更新了几条记录: 1
20:16:23.397 [main] DEBUG org.apache.ibatis.transaction.jdbc.JdbcTransaction -- Committing JDBC Connection [com.mysql.cj.jdbc.Co
20:16:23.399 [main] DEBUG org.apache.ibatis.transaction.jdbc.JdbcTransaction -- Resetting autocommit to true on JDBC Connection
20:16:23.400 [main] DEBUG org.apache.ibatis.transaction.jdbc.JdbcTransaction -- Closing JDBC Connection [com.mysql.cj.jdbc.Conne
20:16:23.400 [main] DEBUG org.apache.ibatis.datasource.pooled.PooledDataSource -- Returned connection 1043358826 to pool.

对象 t_car @mybatis_demo (本地MySQL) -...

表配置文件 开始事务 单元格编辑器 筛选 & 排序 列 数据分析 导入 导出 数据生成

id # bigint	car_num varchar(255)	brand varchar(255)	guide_price decimal(10,2)	produce_time char(10)	car_type varchar(255)
1	100	宝马520Li	41.00	2022-09-01	燃油车
2	101	奔驰E300L	54.00	2022-08-01	电车
3	102	丰田未来	40.30	2014-10-05	氢能源
4	102	丰田未来	40.30	2014-10-05	氢能源
5	102	丰田未来	40.30	2014-10-05	氢能源

引入日志框架logback

对于 MyBatis 而言，日志框架是透视其内部运行机制的核心工具。MyBatis 会自动将 Java 代码转换为 SQL 并发送至数据库（如 MySQL 8.4.8），若缺少日志，该过程将成为“黑盒”——一旦出现数据查询异常或报错，难以定位具体原因（如 SQL 语句生成错误、参数值传递偏差或类型不匹配等）。

引入 Logback 1.5.6 后，可在 IntelliJ IDEA 控制台清晰查看 SQL 的完整组装与执行流程。

需明确 SLF4J 与 Logback 的关系：Java 后端开发通常以 SLF4J（Simple Logging Facade for Java）作为统一日志抽象接口，Logback 则是该接口的具体实现。MyBatis 默认支持 SLF4J 日志输出，因此只需引入 Logback 依赖，MyBatis 即可自动将底层运行信息交由 Logback 打印。

标准日志也可以用，但是配置不够灵活，可以集成其他的日志组件，例如：log4j，logback等。

- logback是目前日志框架中性能较好的，较流行的，所以我们选它。
- 引入logback的步骤：
 - 第一步：引入logback相关依赖

● ● ●

```
<dependency>  
  <groupId>ch.qos.logback</groupId>  
  <artifactId>logback-classic</artifactId>  
  <version>1.5.6</version>  
  <scope>test</scope>  
</dependency>
```

- 第二步：引入logback相关配置文件（文件名叫做logback.xml或logback-test.xml，放到类路径当中）

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Logback 配置文件, 关闭 debug 模式 -->
<configuration debug="false">

    <!-- 定义日志文件存放目录变量 -->
    <property name="LOG_HOME" value="./Logs" />

    <!-- 控制台输出 appender -->
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
        <encoder class="ch.qos.logback.classic.encoder.PatternLayoutEncoder">
            <!-- 输出格式: 时间 线程 日志级别 日志名称 - 消息 换行 -->
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50} - %msg%n</pattern>
        </encoder>
    </appender>

    <!-- 滚动文件输出 appender -->
    <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
        <!-- 滚动策略: 基于文件大小和时间 -->
        <rollingPolicy class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
            <!-- 日志文件名格式, %i 是索引号 -->
            <fileNamePattern>${LOG_HOME}/mybatis_demo.%d{yyyy-MM-dd}.%i.log</fileNamePattern>
            <!-- 单个日志文件最大大小 -->
            <maxFileSize>100MB</maxFileSize>
            <!-- 保留最近 30 天的日志 -->
            <maxHistory>30</maxHistory>
        </rollingPolicy>
        <encoder class="ch.qos.logback.classic.encoder.PatternLayoutEncoder">
            <!-- 文件输出格式与控制台相同 -->
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50} - %msg%n</pattern>
        </encoder>
    </appender>

    <!-- 设置 MyBatis 框架日志级别为 TRACE, 输出详细 SQL 信息 -->
    <logger name="org.apache.ibatis" level="TRACE"/>
    <!-- 打印 JDBC 连接信息 -->
    <logger name="java.sql.Connection" level="DEBUG"/>
    <!-- 打印 JDBC 语句信息 -->
    <logger name="java.sql.Statement" level="DEBUG"/>
    <!-- 打印预编译语句参数信息 -->
    <logger name="java.sql.PreparedStatement" level="DEBUG"/>

    <!-- 自定义包名 car 的日志级别为 DEBUG -->
    <logger name="car" level="DEBUG"/>

    <!-- 根日志级别为 DEBUG, 并关联控制台和文件 appender -->
    <root level="DEBUG">
        <appender-ref ref="STDOUT"/>
        <appender-ref ref="FILE"/>
    </root>

</configuration>
```

- 第三步：在 MyBatis 核心配置文件中显式指定日志

虽然现代版本的 MyBatis 非常聪明，通常能自动发现项目中的 SLF4J/Logback，但为了稳妥起见，我们通常会在 MyBatis 的核心配置文件（如

`mybatis-config.xml`）中明确告诉它我们要用什么日志。

在 `<configuration>` 标签内，添加 `<settings>`：

```

<configuration>
  <settings>
    <setting name="logImpl" value="SLF4J"/>
  </settings>
</configuration>

```

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
  PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
  "http://mybatis.org/dtd/mybatis-3-config.dtd">
<!-- MyBatis 主配置文件 -->
<configuration>

  <!-- 全局设置：配置日志实现为 SLF4J -->
  <settings>
    <setting name="logImpl" value="SLF4J"/>
  </settings>

  <!-- 配置环境，默认使用 development 环境 -->
  <environments default="development">
    <!-- 开发环境配置 -->
    <environment id="development">
      <!-- 事务管理器：JDBC 事务 -->
      <transactionManager type="JDBC"/>
      <!-- 数据源：连接池方式 -->
      <dataSource type="POOLED">
        <!-- MySQL 驱动 -->
        <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
        <!-- 数据库连接 URL，包含编码和时区设置 -->
        <property name="url" value="jdbc:mysql://localhost:3306/mybatis_demo?
useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai"/>
        <!-- 数据库用户名 -->
        <property name="username" value="root"/>
        <!-- 数据库密码 -->
        <property name="password" value="20060329"/>
      </dataSource>
    </environment>
  </environments>

  <!-- 映射器配置：加载 CarMapper.xml 文件 -->
  <mappers>
    <mapper resource="CarMapper.xml"/>
  </mappers>

</configuration>

```

- 再次执行单元测试方法testInsertCar，查看控制台是否有sql语句输出

✓ 测试已通过: 1共1个测试 - 623毫秒

```
2026-03-05 20:16:54.438 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4jImpl' adapter.
2026-03-05 20:16:54.439 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4jImpl' adapter.
2026-03-05 20:16:54.455 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.
2026-03-05 20:16:54.456 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.
2026-03-05 20:16:54.456 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.
2026-03-05 20:16:54.456 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.
2026-03-05 20:16:54.494 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2026-03-05 20:16:54.777 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 2123914473.
2026-03-05 20:16:54.778 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@7e985ce9]
2026-03-05 20:16:54.781 [main] DEBUG car.insertCar - ==> Preparing: insert into t_car (id, car_num, brand, guide_price, produce_time, car_type) values (null, '10
2026-03-05 20:16:54.805 [main] DEBUG car.insertCar - ==> Parameters:
2026-03-05 20:16:54.807 [main] DEBUG car.insertCar - <== Updates: 1
更新了1条记录: 1
2026-03-05 20:16:54.809 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@7e985ce9]
2026-03-05 20:16:54.811 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@7e985ce9]
2026-03-05 20:16:54.812 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@7e985ce9]
2026-03-05 20:16:54.812 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 2123914473 to pool.
```

MyBatis工具类SqlSessionUtil的封装

- 为什么需要封装 `SqlSessionUtil` ?
 - 回顾刚才写的 `CarMapperTest` , 每次执行数据库操作时, 我们都要写下面这堆代码:

```
// 1. 创建SqlSessionFactoryBuilder对象
SqlSessionFactoryBuilder sqlSessionFactoryBuilder = new SqlSessionFactoryBuilder();
// 2. 创建SqlSessionFactory对象 (使用了更加简便的 Resources 工具类)
SqlSessionFactory sqlSessionFactory = sqlSessionFactoryBuilder.build(Resources.getResourceAsStream("mybatis-config.xml"));
// 3. 创建SqlSession对象
sqlSession = sqlSessionFactory.openSession();
```

- 两个致命问题:
 - 代码冗余: 每次增删改查都要写这三行, 极其繁琐。
 - 性能灾难: `SqlSessionFactory` 是一个“重量级”对象。它在创建时会解析庞大的 XML 配置文件。
- 封装的核心设计思想
 - 静态代码块: 利用 Java 类的加载机制, 把读取配置文件和创建 `SqlSessionFactory` 的操作放到 `static {}` 代码块中。类加载时只执行一次, 完美实现全局复用。
 - 私有化构造方法: 防止别人在外部 `new SqlSessionUtil()`, 强制要求通过静态方法获取资源。
 - `ThreadLocal` 绑定: 在未来的 Java 后端开发中, 一个业务逻辑可能包含多个数据库更新操作。为了保证这些操作都在同一个“事务”中, 它们必须使用同一个数据库连接。
- 每一次获取`SqlSession`对象代码太繁琐, 封装一个工具类
 - 在 `org.example` 包下新建一个 `utils` 子包, 将这个工具类放进去。

```
package org.example.utils;

import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;

import java.io.IOException;

/**
 * MyBatis 核心工具类
 */
public class SqlSessionUtil {
```



```

// 1. 声明一个全局静态的 SqlSessionFactory 对象（保证单例）
private static SqlSessionFactory sqlSessionFactory;

// 2. 声明一个 ThreadLocal 容器，用于绑定当前线程的 SqlSession
private static ThreadLocal<SqlSession> local = new ThreadLocal<>();

// 3. 私有化构造方法，防止外部随便 new 这个工具类
private SqlSessionUtil() {}

// 4. 静态代码块：类加载时只执行一次，保证工厂全局只创建一次
static {
    try {
        sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-
config.xml"));
    } catch (IOException e) {
        e.printStackTrace();
        throw new RuntimeException("解析 mybatis-config.xml 失败: ", e);
    }
}

/**
 * 获取当前线程绑定的 SqlSession 对象
 */
public static SqlSession openSession() {
    // 先尝试从当前线程的 ThreadLocal 中取
    SqlSession sqlSession = local.get();
    // 如果没取到（说明当前线程是第一次请求）
    if (sqlSession == null) {
        // 从工厂中开启一个新的 SqlSession
        sqlSession = sqlSessionFactory.openSession();
        // 放进 ThreadLocal 绑定当前线程，供后续操作复用
        local.set(sqlSession);
    }
    return sqlSession;
}

/**
 * 关闭 SqlSession 释放资源，并将其从 ThreadLocal 中移除
 */
public static void close(SqlSession sqlSession) {
    if (sqlSession != null) {
        sqlSession.close();
        // 必须移除，否则在 Tomcat 等使用线程池的 Web 容器中会导致内存泄漏！
        local.remove();
    }
}
}

```

- 改造 CarMapperTest

```

package org.example;

import org.apache.ibatis.session.SqlSession;
import org.example.utils.SqlSessionUtil; // 导入刚刚写的工具类
import org.junit.Test;

import java.util.List;

/**
 * 汽车数据访问层（Mapper）的单元测试
 */
public class CarMapperTest {

```

```

/**
 * 测试用例 1: 测试插入汽车数据
 */
@Test
public void testInsertCar() {
    SqlSession sqlSession = null;
    try {
        // 【改造点 1】不再需要手动 build 工厂，直接一行代码获取当前线程的 SqlSession!
        sqlSession = SqlSessionUtil.openSession();

        // 执行SQL
        int count = sqlSession.insert("car.insertCar");
        System.out.println("更新了几条记录: " + count);

        // 提交事务（增删改必须提交）
        sqlSession.commit();
    } catch (Exception e) {
        // 回滚事务，保证数据一致性
        if (sqlSession != null) {
            sqlSession.rollback();
            System.out.println("发生异常，事务已回滚！");
        }
        e.printStackTrace();
    } finally {
        // 【改造点 2】使用工具类的 close 方法，安全释放资源并清理 ThreadLocal
        SqlSessionUtil.close(sqlSession);
    }
}

/**
 * 测试用例 2: 测试查询所有汽车数据
 */
@Test
public void testSelectAllCars() {
    SqlSession sqlSession = null;
    try {
        // 【改造点 1】同样，一行代码搞定！
        sqlSession = SqlSessionUtil.openSession();

        // 执行查询 SQL
        List<Object> cars = sqlSession.selectList("car.selectALLCars");

        System.out.println("==== 查询到的汽车列表 ====");
        for (Object car : cars) {
            System.out.println(car);
        }

    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        // 【改造点 2】安全释放
        SqlSessionUtil.close(sqlSession);
    }
}
}

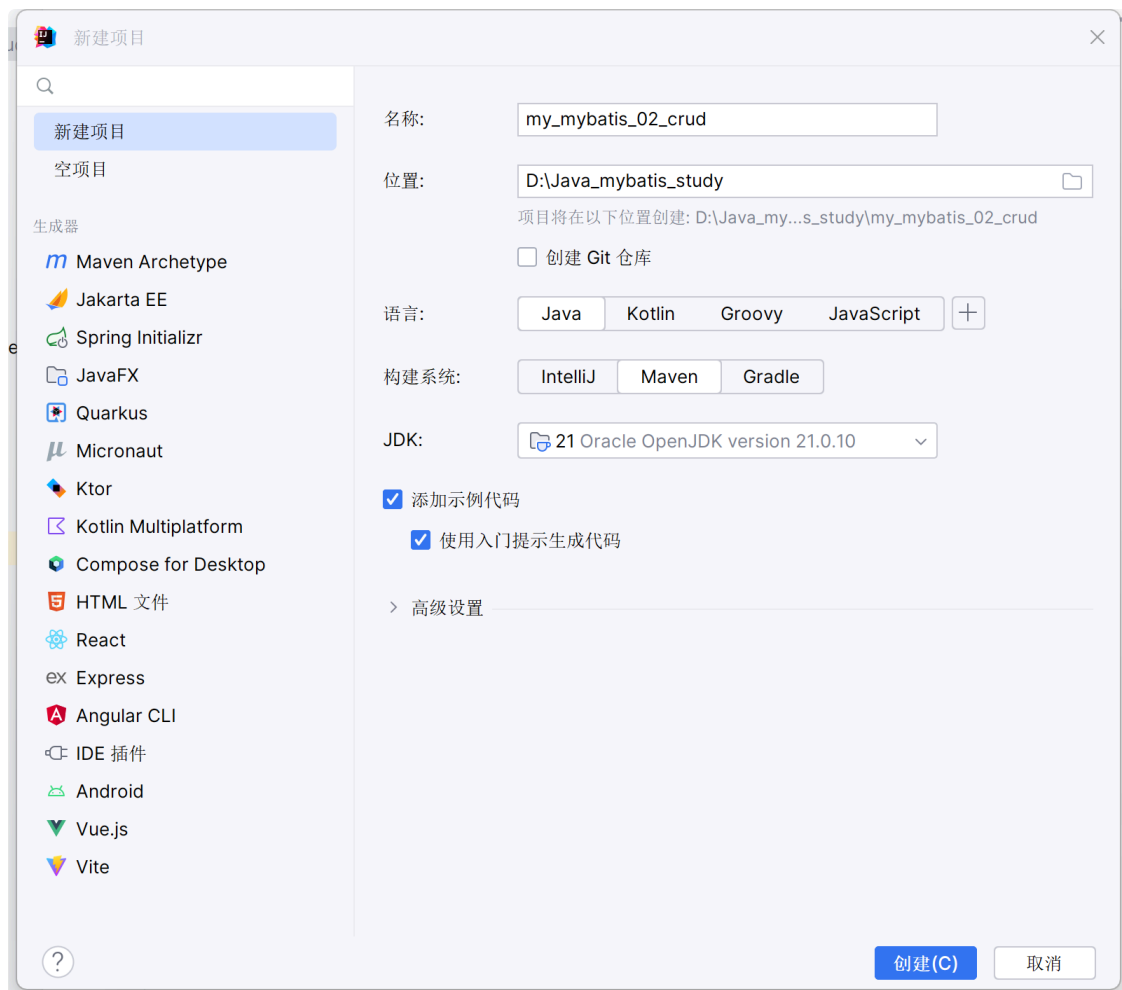
```

使用MyBatis完成CRUD

准备工作

- 在 IDEA 中创建新模块
1. 在左侧选择 New Module (或者 Maven)。

2. Name: 填入 my_mybatis_02_crud。
3. Language: 选择 Java。
4. Build system: 选择 Maven。
5. JDK: 选择 Java 21。



- 配置 pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.powernode</groupId>
  <artifactId>my_mybatis_02_crud</artifactId>
  <version>1.0-SNAPSHOT</version>

  <packaging>jar</packaging>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.mybatis</groupId>
      <artifactId>mybatis</artifactId>
```

```

        <version>3.5.19</version>
    </dependency>

    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>8.4.0</version>
    </dependency>

    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>ch.qos.logback</groupId>
        <artifactId>logback-classic</artifactId>
        <version>1.5.6</version>
    </dependency>
</dependencies>

</project>

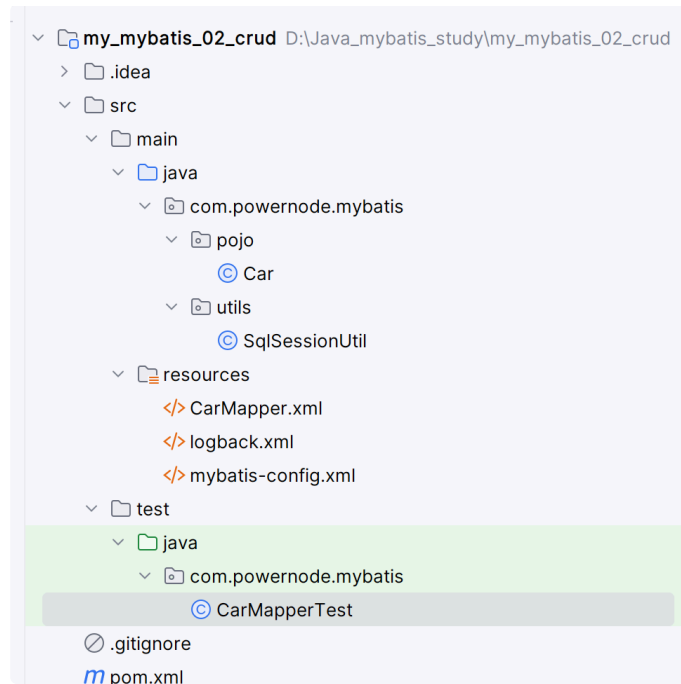
```

- 搭建标准目录结构与配置文件

```

my_mybatis_02_crud
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── powernode
│   │   │   │   │   ├── mybatis
│   │   │   │   │   │   ├── pojo (Car.java 实体类)
│   │   │   │   │   │   │   ├── Car.java
│   │   │   │   │   │   │   └── utils (放工具类)
│   │   │   │   │   │       └── SqlSessionUtil.java
│   │   │   └── resources (类的根路径)
│   │   │       ├── mybatis-config.xml (把上一节的复制过来)
│   │   │       ├── logback.xml (把上一节的复制过来)
│   │   │       └── CarMapper.xml (留空)
│   └── test
│       ├── java
│       │   ├── com
│       │   │   ├── powernode
│       │   │   │   ├── mybatis
│       │   │   │   │   └── CarMapperTest.java (测试类)
└── pom.xml

```



- SqlSessionUtil.java代码

```
package com.powernode.mybatis.utils; //更新包名

import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;

import java.io.IOException;

public class SqlSessionUtil {
    private static SqlSessionFactory sqlSessionFactory;
    private static ThreadLocal<SqlSession> local = new ThreadLocal<>();
    private SqlSessionUtil() {}

    static {
        try {
            sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-config.xml"));
        } catch (IOException e) {
            e.printStackTrace();
            throw new RuntimeException("解析 mybatis-config.xml 失败: ", e);
        }
    }

    public static SqlSession openSession() {
        SqlSession sqlSession = local.get();
        if (sqlSession == null) {
            sqlSession = sqlSessionFactory.openSession();
            local.set(sqlSession);
        }
        return sqlSession;
    }

    public static void close(SqlSession sqlSession) {
        if (sqlSession != null) {
            sqlSession.close();
            local.remove();
        }
    }
}
```

```
}
```

- Car.java代码

```
package com.powernode.mybatis.pojo;

/**
 * 汽车实体类，对应数据库中的 t_car 表
 */
public class Car {

    private Long id;           // 对应 id
    private String carNum;     // 对应 car_num
    private String brand;      // 对应 brand
    private Double guidePrice; // 对应 guide_price
    private String produceTime; // 对应 produce_time
    private String carType;    // 对应 car_type

    // 无参构造
    public Car() {
    }

    // 全参构造
    public Car(Long id, String carNum, String brand, Double guidePrice, String produceTime, String carType) {
        this.id = id;
        this.carNum = carNum;
        this.brand = brand;
        this.guidePrice = guidePrice;
        this.produceTime = produceTime;
        this.carType = carType;
    }

    // Getter 和 Setter (MyBatis 核心依赖)
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getCarNum() { return carNum; }
    public void setCarNum(String carNum) { this.carNum = carNum; }

    public String getBrand() { return brand; }
    public void setBrand(String brand) { this.brand = brand; }

    public Double getGuidePrice() { return guidePrice; }
    public void setGuidePrice(Double guidePrice) { this.guidePrice = guidePrice; }

    public String getProduceTime() { return produceTime; }
    public void setProduceTime(String produceTime) { this.produceTime = produceTime; }

    public String getCarType() { return carType; }
    public void setCarType(String carType) { this.carType = carType; }

    @Override
    public String toString() {
        return "Car{" +
            "id=" + id +
            ", carNum='" + carNum + '\'' +
            ", brand='" + brand + '\'' +
            ", guidePrice=" + guidePrice +
            ", produceTime='" + produceTime + '\'' +
            ", carType='" + carType + '\'' +
            '}';
    }
}
```

insert (Create)

- 分析以下SQL映射文件中SQL语句存在的问题
 - CarMapper.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<!--namespace 先随便写-->
<mapper namespace="car">
    <insert id="insertCar">
        insert into t_car(car_num,brand,guide_price,produce_time,car_type) values('103', '奔驰E300L', 50.3, '2022-01-01', '燃油车')
    </insert>
</mapper>
```

存在的问题是：SQL语句中的值不应该写死，值应该是用户提供的。之前的JDBC代码是这样写的：

```
// JDBC 中使用 ? 作为占位符。那么MyBatis 中会使用什么作为占位符呢？
String sql = "insert into t_car(car_num,brand,guide_price,produce_time,car_type) values(?,?,?, ?,?)";
// .....
// 给 ? 传值。那么MyBatis 中应该怎么传值呢？
ps.setString(1,"103");
ps.setString(2,"奔驰E300L");
ps.setDouble(3,50.3);
ps.setString(4,"2022-01-01");
ps.setString(5,"燃油车");
```

由于编写了Car.java 实体在MyBatis中可以这样做：

- 编写 CarMapper.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="car">

    <insert id="insertCar" parameterType="com.powernode.mybatis.pojo.Car">
        insert into t_car(car_num, brand, guide_price, produce_time, car_type)
        values("#{carNum}", #{brand}, #{guidePrice}, #{produceTime}, #{carType})
    </insert>

</mapper>
```

- #{} 是怎么工作的？
 - 当 MyBatis 执行到 #{carNum} 时，它底层会使用 Java 的反射机制，去调用你传进来的 Car 对象的 getCarNum() 方法。
 - 它拿到了返回值，就会自动填入到 SQL 语句对应的位置中。
 - 这就是为什么我们在写实体类时，必须生成标准的 Getter 方法
- 编写 CarMapperTest.java 单元测试

```
package com.powernode.mybatis;
```

```

import com.powernode.mybatis.pojo.Car;
import com.powernode.mybatis.utils.SqlSessionUtil;
import org.apache.ibatis.session.SqlSession;
import org.junit.Test;

public class CarMapperTest {

    @Test
    public void testInsertCar() {
        SqlSession sqlSession = null;
        try {
            // 1. 通过工具类获取会话
            sqlSession = SqlSessionUtil.openSession();

            // 2. 准备一个 Car 对象（我们要插入的数据）
            // 注意: id 是自增的，我们传 null 即可
            Car car = new Car(null, "粤B12345", "保时捷911", 120.50, "2025-10-01", "燃油车");

            // 3. 执行插入操作
            // 第一个参数是 XML 里定义的 namespace.id
            // 第二个参数就是我们刚刚 new 出来的 car 对象
            int count = sqlSession.insert("car.insertCar", car);

            System.out.println("成功插入了 " + count + " 条汽车记录!");

            // 4. 提交事务（所有的增删改操作都必须手动 commit）
            sqlSession.commit();

        } catch (Exception e) {
            if (sqlSession != null) {
                sqlSession.rollback(); // 发生异常时回滚
            }
            e.printStackTrace();
        } finally {
            // 5. 释放资源
            SqlSessionUtil.close(sqlSession);
        }
    }
}

```

- 实体类Java程序，运行结果后

id # bigint	car_num varchar(255)	brand varchar(255)	guide_price # decimal(10,2)	produce_time char(10)	car_type varchar(255)
1	100	宝马520Li	41.00	2022-09-01	燃油车
2	101	奔驰E300L	54.00	2022-08-01	电车
3	102	丰田未来	40.30	2014-10-05	氢能源
4	102	丰田未来	40.30	2014-10-05	氢能源
5	102	丰田未来	40.30	2014-10-05	氢能源
6	102	丰田未来	40.30	2014-10-05	氢能源
7	102	丰田未来	40.30	2014-10-05	氢能源
8	102	丰田未来	40.30	2014-10-05	氢能源
9	102	丰田未来	40.30	2014-10-05	氢能源
10	102	丰田未来	40.30	2014-10-05	氢能源
11	粤B12345	保时捷911	120.50	2025-10-01	燃油车

```

测试 已通过: 1共 1 个测试 - 895毫秒

java21\bin\java.exe ...
>03-05 22:03:05.548 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4jImpl'
>03-05 22:03:05.549 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4jImpl'
>03-05 22:03:05.566 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.
>03-05 22:03:05.567 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.
>03-05 22:03:05.567 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.
>03-05 22:03:05.567 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.
>03-05 22:03:05.612 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
>03-05 22:03:06.060 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 635288507.
>03-05 22:03:06.061 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jdbc.
>03-05 22:03:06.066 [main] DEBUG car.insertCar - ==> Preparing: insert into t_car(car_num, brand, guide_price, produce_time, car_type) values(?, ?,
>03-05 22:03:06.102 [main] DEBUG car.insertCar - ==> Parameters: 粤B12345(String), 保时捷911(String), 120.5(Double), 2025-10-01(String), 燃油车(String)
>03-05 22:03:06.109 [main] DEBUG car.insertCar - <== Updates: 1
插入了 1 条汽车记录!
>03-05 22:03:06.112 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@25c
>03-05 22:03:06.114 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.cj.jc
>03-05 22:03:06.114 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@25ddb
>03-05 22:03:06.114 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 635288507 to pool.

已结束, 退出代码为 0

```

- #{} 里写的是POJO的属性名，如果写成其他的会有问题吗？

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="car">

    <insert id="insertCar" parameterType="com.powernode.mybatis.pojo.Car">
        insert into t_car(car_num, brand, guide_price, produce_time, car_type)
        values("#{a}", #{brand}, #{guidePrice}, #{produceTime}, #{carType})
    </insert>

</mapper>

```

- 运行程序，出现了以下异常：

```
>> Tests failed: 1 of 1 test - 820ms
2022-08-05 11:00:33.041 [main] DEBUG org.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2022-08-05 11:00:33.451 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 1429483328.
2022-08-05 11:00:33.454 [main] DEBUG car.insertCarByPOJO - ==> Preparing: insert into t_car(car_num,brand,guide_price,produce_time,car_type) values(?,?,
org.apache.ibatis.exceptions.PersistenceException:
### Error updating database. Cause: org.apache.ibatis.reflection.ReflectionException: There is no getter for property named 'a' in 'class com.powernode
### The error may exist in CarMapper.xml
### The error may involve defaultParameterMap
### The error occurred while setting parameters
### SQL: insert into t_car(car_num,brand,guide_price,produce_time,car_type) values(?,?,?,?,?)
### Cause: org.apache.ibatis.reflection.ReflectionException: There is no getter for property named 'a' in 'class com.powernode.mybatis.pojo.Car
```

错误信息中描述：在Car类中没有找到a属性的getter方法。

修改POJO类Car的代码，只将getCarNum()方法名修改为getA()，其他代码不变，如下：

```
apperTest.java x Car.java x CarMapper.xml x
/*public String getCarNum() {
    return carNum;
}*/

public String getA() {
    return carNum;
}
```

运行结果后，成功插入数据。

- 优点：
 - 类型安全
 - 完美的代码提示
 - 接轨企业级标准

注意：这里是写了Car.java实体类，如果不写会怎么样？

- MyBatis 非常灵活，如果你没有实体类，它完全允许你使用 Map<String, Object> 来传递参数和接收结果。
- 不写实体类（使用 Map）
 - Java程序这样写：

```
package com.powernode.mybatis;

import com.powernode.mybatis.utils.SqlSessionUtil;
import org.apache.ibatis.session.SqlSession;
import org.junit.Test;

import java.util.HashMap;
import java.util.Map;

/**
 * 测试MyBatis的CRUD
 */
public class CarMapperTest {
    @Test
    public void testInsertCar(){
        // 准备数据
        Map<String, Object> map = new HashMap<>();
        map.put("k1", "103");
        map.put("k2", "奔驰E300L");
        map.put("k3", 50.3);
```

```

        map.put("k4", "2020-10-01");
        map.put("k5", "燃油车");
        // 获取SqlSession对象
        SqlSession sqlSession = SqlSessionUtil.openSession();
        // 执行SQL语句（使用map集合给sql语句传递数据）
        int count = sqlSession.insert("insertCar", map);
        System.out.println("插入了几条记录: " + count);
    }
}

```

- SQL语句这样写：
 - CarMapper.xml

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<!--namespace 先随便写-->
<mapper namespace="car">
    <insert id="insertCar">
        insert into t_car(car_num,brand,guide_price,produce_time,car_type) values(#{k1},#{k2},#{k3},#{k4},#{k5})
    </insert>
</mapper>

```

- #{ } 的里面必须填写map集合的key，不能随便写。
- 如果#{ }里写的是map集合中不存在的key会有什么问题？

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="car">
    <insert id="insertCar">
        insert into t_car(car_num,brand,guide_price,produce_time,car_type) values(#{kk},#{k2},#{k3},#{k4},#{k5})
    </insert>
</mapper>

```

通过测试，看到程序并没有报错。正常执行。不过 #{kk} 的写法导致无法获取到map集合中的数据，最终导致数据库表car_num插入了NULL。

在以上sql语句中，可以看到#{k1} #{k2} #{k3} #{k4} #{k5}的可读性太差，为了增强可读性，我们可以将Java程序做如下修改：

```

Map<String, Object> map = new HashMap<>();
// 让key的可读性增强
map.put("carNum", "103");
map.put("brand", "奔驰E300L");
map.put("guidePrice", 50.3);
map.put("produceTime", "2020-10-01");
map.put("carType", "燃油车");

```

SQL语句做如下修改，这样可以增强程序的可读性：



```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="car">
    <insert id="insertCar">
        insert into t_car(car_num,brand,guide_price,produce_time,car_type) values(#{carNum},#{brand},#{guidePrice},#{
produceTime},#{carType})
    </insert>
</mapper>
```

- 运行后，不会随意插入null值
- 缺点：
 - 没有代码提示
 - 极易写出 Bug
 - 可读性极差

如果采用map集合传参，#{ } 里写的是map集合的key，如果key不存在不会报错，数据库表中会插入NULL。

如果采用POJO传参，#{ } 里写的是get方法的方法名去掉get之后将剩下的单词首字母变小写（例如：getAge对应的是#{age}，getUserName对应的是#{userName}），如果这样的get方法不存在会报错。

注意：其实传参数的时候有一个属性parameterType，这个属性用来指定传参的数据类型，不过这个属性是可以省略的



```
<insert id="insertCar" parameterType="java.util.Map">
    insert into t_car(car_num,brand,guide_price,produce_time,car_type) values(#{carNum},#{brand},#{guidePrice},#{
produceTime},#{carType})
</insert>

<insert id="insertCarByPOJO" parameterType="com.powernode.mybatis.pojo.Car">
    insert into t_car(car_num,brand,guide_price,produce_time,car_type) values(#{carNum},#{brand},#{guidePrice},#{
produceTime},#{carType})
</insert>
```

delete (Delete)

在 CRUD 操作中，Delete 通常是最简单的，因为我们大多数情况下都是根据主键（ID）去删除一条特定的记录。

也就是说，我们不需要像 Insert 那样传一个庞大的 Car 对象进去，只需要传一个简单的数字（ID）就可以了。

- 在 CarMapper.xml 中编写 `<delete>` 标签
 - 在 `<insert>` 标签下面，添加这段删除的 SQL 配置：



```
<delete id="deleteCarById" parameterType="Long">
    delete from t_car where id = #{id}
</delete>
```



```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="car">
```

```

<insert id="insertCar" parameterType="com.powernode.mybatis.pojo.Car">
    insert into t_car(car_num, brand, guide_price, produce_time, car_type)
    values("#{carNum}", #{brand}, #{guidePrice}, #{produceTime}, #{carType})
</insert>

<delete id="deleteCarById" parameterType="Long">
    delete from t_car where id = #{id}
</delete>

</mapper>

```

- 在 CarMapperTest.java 中编写测试方法

```

/**
 * 测试用例：根据 ID 删除汽车数据
 */
@Test
public void testDeleteCarById() {
    SqlSession sqlSession = null;
    try {
        // 1. 获取会话
        sqlSession = SqlSessionUtil.openSession();

        // 2. 执行删除操作
        // 第一个参数是 namespace.id
        // 第二个参数是你想要删除的那个 ID（请先打开 Navicat 看一下你表里真实存在的 ID 是多少，比如我这里填 1L）
        // 注意：因为实体类里定义的是 Long 类型，所以数字后面最好加上大写字母 L
        int count = sqlSession.delete("car.deleteCarById", 1L);

        System.out.println("成功删除了 " + count + " 条汽车记录！");

        // 3. 提交事务（千万记住，增删改都必须 commit，否则数据库不会真正发生变化）
        sqlSession.commit();

    } catch (Exception e) {
        if (sqlSession != null) {
            sqlSession.rollback();
        }
        e.printStackTrace();
    } finally {
        // 4. 释放资源
        SqlSessionUtil.close(sqlSession);
    }
}

```

- 运行与验证

```
✓ 1秒 150毫秒  ✓ 测试 已通过: 1共 1 个测试 - 1秒 150毫秒
✓ testDelete D:\Java21\bin\java.exe ...
2026-03-06 10:42:41.361 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4jIm
2026-03-06 10:42:41.363 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4jIm
2026-03-06 10:42:41.387 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections
2026-03-06 10:42:41.387 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections
2026-03-06 10:42:41.389 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections
2026-03-06 10:42:41.389 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections
2026-03-06 10:42:41.456 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2026-03-06 10:42:42.004 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 1321115948.
2026-03-06 10:42:42.006 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj
2026-03-06 10:42:42.011 [main] DEBUG car.deleteCarById - ==> Preparing: delete from t_car where id = ?
2026-03-06 10:42:42.051 [main] DEBUG car.deleteCarById - ==> Parameters: 11(Long) 删除ID为11的数据
2026-03-06 10:42:42.059 [main] DEBUG car.deleteCarById - <== Updates: 1 删除一条
成功删除了 1 条汽车记录!
2026-03-06 10:42:42.061 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl
2026-03-06 10:42:42.064 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.c
2026-03-06 10:42:42.065 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@4e
2026-03-06 10:42:42.065 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 1321115948 to pool.

进程已结束，退出代码为 0
```

对象		t_car @mybatis_demo (本地MySQL) -...				
表配置文件		开始事务	单元格编辑器	筛选 & 排序	列	数据分析 导入 导出 数据
id	car_num	brand	guide_price	produce_time	car_type	
# bigint	varchar(255)	varchar(255)	# decimal(10,2)	char(10)	varchar(255)	
1	100	宝马520Li	41.00	2022-09-01	燃油车	
2	101	奔驰E300L	54.00	2022-08-01	电车	
3	102	丰田未来	40.30	2014-10-05	氢能源	
4	102	丰田未来	40.30	2014-10-05	氢能源	
5	102	丰田未来	40.30	2014-10-05	氢能源	
6	102	丰田未来	40.30	2014-10-05	氢能源	
7	102	丰田未来	40.30	2014-10-05	氢能源	
8	102	丰田未来	40.30	2014-10-05	氢能源	
9	102	丰田未来	40.30	2014-10-05	氢能源	
10	102	丰田未来	40.30	2014-10-05	氢能源	

- id为11的数据已被删除

注意：当占位符只有一个的时候，\${} 里面的内容可以随便写。

update (Update)

在 MyBatis 中，Update（修改）操作的逻辑和 Insert 非常相似。

因为我们需要修改多个字段，同时还要指定是哪一条记录（通过 ID），所以我们通常会直接传递一个装满新数据的 Car 实体类对象进去。

需求：修改id=9的Car信息，car_num为102，brand为比亚迪汉，guide_price为30.23，produce_time为2018-09-10，car_type为电车

修改前：

id # bigint	car_num # varchar(255)	brand # varchar(255)	guide_price # decimal(10,2)	produce_time # char(10)	car_type # varchar(255)
1	100	宝马520Li	41.00	2022-09-01	燃油车
2	101	奔驰E300L	54.00	2022-08-01	电车
3	102	丰田未来	40.30	2014-10-05	氢能源
4	102	丰田未来	40.30	2014-10-05	氢能源
5	102	丰田未来	40.30	2014-10-05	氢能源
6	102	丰田未来	40.30	2014-10-05	氢能源
7	102	丰田未来	40.30	2014-10-05	氢能源
8	102	丰田未来	40.30	2014-10-05	氢能源
9	102	丰田未来	40.30	2014-10-05	氢能源
10	102	丰田未来	40.30	2014-10-05	氢能源

- 在 CarMapper.xml 中编写 `<update>` 标签

```

<update id="updateCar" parameterType="com.powernode.mybatis.pojo.Car">
    update t_car
    set
        car_num = #{carNum},
        brand = #{brand},
        guide_price = #{guidePrice},
        produce_time = #{produceTime},
        car_type = #{carType}
    where id = #{id}
</update>

```

- 在 CarMapperTest.java 中编写测试方法
 - 新增一个用于测试修改的方法。

```

/**
 * 测试用例：根据 ID 更新汽车数据
 */
@Test
public void testUpdateCar() {
    SqlSession sqlSession = null;
    try {
        // 1. 获取会话
        sqlSession = SqlSessionUtil.openSession();

        // 2. 准备要更新的数据对象
        // 关键点：必须要有 id (9L)，因为 SQL 里的 where id = #{id} 靠它来定位
        // 其他参数填入你要求的新数据
        Car car = new Car(9L, "102", "比亚迪汉", 30.23, "2018-09-10", "电车");

        // 3. 执行修改操作
        // 第一个参数是 namespace.id
        // 第二个参数是我们封装好新数据的 car 对象
        int count = sqlSession.update("car.updateCar", car);

        System.out.println("成功修改了 " + count + " 条汽车记录!");

        // 4. 提交事务（增删改都必须 commit）
        sqlSession.commit();

    } catch (Exception e) {
        if (sqlSession != null) {

```



```

        sqlSession.rollback();
    }
    e.printStackTrace();
} finally {
    // 5. 释放资源
    SqlSessionUtil.close(sqlSession);
}
}

```

- 运行 `testUpdateCar` 方法

```

✓ 测试已通过: 1共 1 个测试 - 1秒 21毫秒
2026-03-06 10:55:29.726 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4jIm
2026-03-06 10:55:29.728 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4jIm
2026-03-06 10:55:29.754 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections
2026-03-06 10:55:29.754 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections
2026-03-06 10:55:29.755 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections
2026-03-06 10:55:29.816 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2026-03-06 10:55:30.268 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 1321115948.
2026-03-06 10:55:30.269 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj
2026-03-06 10:55:30.274 [main] DEBUG car.updateCar - ==> Preparing: update t_car set car_num = ?, brand = ?, guide_price = ?, produce_time = ?, car
2026-03-06 10:55:30.319 [main] DEBUG car.updateCar - ==> Parameters: 102(String), 比亚迪汉(String), 30.23(Double), 2018-09-10(String), 电车(String), 9
2026-03-06 10:55:30.327 [main] DEBUG car.updateCar - <== Updates: 1
成功修改了 1 条汽车记录!
2026-03-06 10:55:30.330 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl
2026-03-06 10:55:30.332 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.c
2026-03-06 10:55:30.333 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@4e
2026-03-06 10:55:30.334 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 1321115948 to pool.

进程已结束，退出代码为 0

```

id # bigint	car_num varchar(255)	brand varchar(255)	guide_price # decimal(10,2)	produce_time char(10)	car_type varchar(255)
1	100	宝马520Li	41.00	2022-09-01	燃油车
2	101	奔驰E300L	54.00	2022-08-01	电车
3	102	丰田未来	40.30	2014-10-05	氢能源
4	102	丰田未来	40.30	2014-10-05	氢能源
5	102	丰田未来	40.30	2014-10-05	氢能源
6	102	丰田未来	40.30	2014-10-05	氢能源
7	102	丰田未来	40.30	2014-10-05	氢能源
8	102	丰田未来	40.30	2014-10-05	氢能源
9	102	比亚迪汉	30.23	2018-09-10	电车
10	102	丰田未来	40.30	2014-10-05	氢能源

当然了，如果使用map传数据也是可以的。

select (Retrieve)

select语句和其它语句不同的是：查询会有一个结果集。来看mybatis是怎么处理结果集的！！

查询一条数据

- 之前我们做的增删改（CRUD），MyBatis 的返回值永远是一个 int（影响了多少行）。
- 但查询操作，我们需要 MyBatis 把数据库里查出来的结果，自动打包变成我们写的 Java 实体类对象（如 Car 对象），然后交回给我们。
- 为了让 MyBatis 知道怎么打包，需要引入一个新属性：`resultType`。

- 需求：查询id为9的Car信息
- 在 CarMapper.xml 中编写 `<select>` 标签

```
<select id="selectCarById" resultType="com.powernode.mybatis.pojo.Car">
    select
        id,
        car_num as carNum,
        brand,
        guide_price as guidePrice,
        produce_time as produceTime,
        car_type as carType
    from t_car
    where id = #{id}
</select>
```

- 在 CarMapperTest.java 中编写测试方法
 - 打开测试类，编写查询一条数据的测试用例。
- 注意：这里的增删改的两个最大不同点：调用的是 `selectOne` 方法，而且不需要提交事务

```
/**
 * 测试用例：根据 ID 查询一条汽车数据
 */
@Test
public void testSelectCarById() {
    SqlSession sqlSession = null;
    try {
        // 1. 获取会话
        sqlSession = SqlSessionUtil.openSession();

        // 2. 执行查询操作
        // 使用 selectOne 方法：专门用来查询一条数据（如果查出多条会报错！）
        // 第一个参数是 namespace.id
        // 第二个参数是你查询的 ID，咱们查刚才那辆 id=9 的比亚迪汉
        // 【重点】：它的返回值直接就是我们在 resultType 里配置的 Car 对象！
        Object obj = sqlSession.selectOne("car.selectCarById", 9L);

        // 将 Object 向下转型为 Car（因为我们确信 resultType 就是 Car）
        Car car = (Car) obj;

        // 3. 打印结果，看看 MyBatis 有没有帮我们把数据封装好
        System.out.println("查到的汽车信息：");
        System.out.println(car);

        // 注意：查询操作只是读取数据，不会改变数据库的内容，所以【不需要】执行 sqlSession.commit();

    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        // 4. 释放资源
        SqlSessionUtil.close(sqlSession);
    }
}
```

- 运行

```
D:\Java21\bin\java.exe ...
2026-03-06 11:06:40.299 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4jIm
2026-03-06 11:06:40.300 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4jIm
2026-03-06 11:06:40.322 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections
2026-03-06 11:06:40.322 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections
2026-03-06 11:06:40.323 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections
2026-03-06 11:06:40.323 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections
2026-03-06 11:06:40.385 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2026-03-06 11:06:40.828 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 917568725.
2026-03-06 11:06:40.851 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj
2026-03-06 11:06:40.856 [main] DEBUG car.selectCarById - ==> Preparing: select id, car_num as carNum, brand, guide_price as guidePrice, produce_tim
2026-03-06 11:06:40.891 [main] DEBUG car.selectCarById - ==> Parameters: 9(Long)
2026-03-06 11:06:40.931 [main] DEBUG car.selectCarById - <== Total: 1
查到的汽车信息:
Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10', carType='电车'}
2026-03-06 11:06:40.939 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.c
2026-03-06 11:06:40.940 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@36
2026-03-06 11:06:40.940 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 917568725 to pool.
```

如果不加resultType，会怎么样？

sql语句写法的区别，as写法与 * 写法？

- 运行结果后

```
### Error querying database. Cause: org.apache.ibatis.executor.ExecutorException:
A query was run and no Result Maps were found for the Mapped Statement 'car.selectCarById'.
【翻译】：对于一个查询语句来说，没有找到查询的结果映射。
It's Likely that neither a Result Type nor a Result Map was specified.
【翻译】：很可能既没有指定结果类型，也没有指定结果映射。
```

以上的异常大致的意思是：对于一个查询语句来说，你需要指定它的“结果类型”或者“结果映射”。

所以说，你想让mybatis查询之后返回一个Java对象的话，至少你要告诉mybatis返回一个什么类型的Java对象，可以在 `<select>` 标签中添加resultType属性，用来指定查询要转换的类型：

```
<select id="selectCarById" resultType="com.powernode.mybatis.pojo.Car">
    select * from t_car where id = #{id}
</select>
```

如果sql语句使用的是 `select * from t_car where id = #{id}`

运行后会导致个别值出现NULL值

```
Tests passed: 1 of 1 test - 890ms
2022-08-05 15:02:22.965 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed a
2022-08-05 15:02:22.965 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed a
2022-08-05 15:02:23.030 [main] DEBUG org.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2022-08-05 15:02:23.438 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 203149502.
2022-08-05 15:02:23.441 [main] DEBUG car.selectCarById - ==> Preparing: select * from t_car where id = ?
2022-08-05 15:02:23.476 [main] DEBUG car.selectCarById - ==> Parameters: 1(Integer)
2022-08-05 15:02:23.508 [main] TRACE car.selectCarById - <== Columns: id, car_num, brand, guide_price, produce_time, car_type
2022-08-05 15:02:23.508 [main] TRACE car.selectCarById - <== Row: 1, 100, 宝马520Li, 41.00, 2022-09-01, 燃油车
2022-08-05 15:02:23.510 [main] DEBUG car.selectCarById - <== Total: 1
Car{id=1, carNum='null', brand='宝马520Li', guidePrice=null, produceTime='null', carType='null'}
```

老杜

运行后之前的异常不再出现了，这说明添加了resultType属性之后，解决了之前的异常，可以看出resultType是不能省略的。

仔细观察控制台的日志信息，不难看出，结果查询出了一条。并且每个字段都查询到值了：Row: 1, 100, 宝马520Li, 41.00, 2022-09-01, 燃油车

但是奇怪的是返回的Car对象，只有id和brand两个属性有值，其它属性的值都是null，这是为什么呢？我们来观察一下查询结果列名和Car类的属性名是否能一一对应：

查询结果集的列名：id, car_num, brand, guide_price, produce_time, car_type
Car类的属性名：id, carNum, brand, guidePrice, produceTime, carType

通过观察发现：只有id和brand是一致的，其他字段名和属性名对应不上，这就是导致null的原因。

如果当查询结果的字段名和java类的属性名对应不上，可以采用as关键字起别名，当然还有其它解决方案，我们后面再看。

查询多条数据

虽然我们期望返回的是一个 `List<Car>` 集合，但在 MyBatis 的 XML 配置文件中，`resultType` 依然只写 `Car` 类的名字

MyBatis 非常聪明，它看到你 Java 代码里调用的是 `selectList` 方法，就会自动把数据库里的每一行数据封装成一个 `Car` 对象，然后统一塞进一个 `List` 集合里交给你。

- 需求：查询所有的Car信息。
- 在 CarMapper.xml 中编写 `<select>` 标签

```
<select id="selectAllCars" resultType="com.powernode.mybatis.pojo.Car">
    select
        id,
        car_num as carNum,
        brand,
        guide_price as guidePrice,
        produce_time as produceTime,
        car_type as carType
    from t_car
</select>
```

- 在 CarMapperTest.java 中编写测试方法

```
import java.util.List; //在最前面导入 java.util.List

/**
 * 测试用例：查询所有汽车数据
 */
@Test
public void testSelectAllCars() {
    SqlSession sqlSession = null;
    try {
        // 1. 获取会话
        sqlSession = SqlSessionUtil.openSession();

        // 2. 执行查询操作
        // 【核心区别】：这次我们调用的是 selectList 方法！
        // MyBatis 查到多条记录后，会自动生成一个 List 集合，里面装满了 Car 对象。
        List<Car> cars = sqlSession.selectList("car.selectAllCars");

        // 3. 遍历并打印结果
        System.out.println("==== 数据库中所有的汽车信息 =====");
        for (Car car : cars) {
            System.out.println(car);
        }

        // 查询操作不需要 commit

    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        // 4. 释放资源
        SqlSessionUtil.close(sqlSession);
    }
}
```

```
}
```

• 运行 testSelectAllCars 方法

```
2026-03-06 11:35:46.303 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - opening JDBC Connection
2026-03-06 11:35:46.774 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 917568725.
2026-03-06 11:35:46.775 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj
2026-03-06 11:35:46.782 [main] DEBUG car.selectAllCars - ==> Preparing: select id, car_num as carNum, brand, guide_price as guidePrice, produce_tim
2026-03-06 11:35:46.820 [main] DEBUG car.selectAllCars - ==> Parameters:
2026-03-06 11:35:46.864 [main] DEBUG car.selectAllCars - <== Total: 10
==== 数据库中所有的汽车信息 ====
Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}
Car{id=2, carNum='101', brand='奔驰E300L', guidePrice=54.0, produceTime='2022-08-01', carType='电车'}
Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=6, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=7, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10', carType='电车'}
Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
2026-03-06 11:35:46.873 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.c
2026-03-06 11:35:46.875 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@36
2026-03-06 11:35:46.875 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 917568725 to pool.

进程已结束, 退出代码为 0
```

关于SQL Mapper的namespace

- 在大型企业级项目中，表和开发人员都非常多。假设有这样一个场景：
 - 老王写了一个 CarMapper.xml，里面有一条查询所有汽车的 SQL，id 叫 selectAllCars。
 - 老李后来加入项目，为了做另外一个汽车业务模块，他建了一个 CarMapper2.xml，好巧不巧，他里面也写了一条查询 SQL，id 也叫 selectAllCars。

如果 MyBatis 没有 namespace，你在 Java 代码里写 sqlSession.selectList("selectAllCars")，MyBatis 当场就会“精神分裂”：“老天，两个文件里都有 selectAllCars，你到底要我执行哪一个？！”

为了解决这种命名冲突，MyBatis 引入了 namespace。用来给 SQL 语句做隔离。

- 创建 CarMapper2.xml
 - 编写新的 Mapper 文件
 - 我们在里面的 `<select>` 标签的 id 故意和之前的 CarMapper.xml 写得一模一样，都是 selectAllCars。但是，我们将 namespace 设置为 car2。

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="car2">

    <select id="selectAllCars" resultType="com.powernode.mybatis.pojo.Car">
        select
            id,
            car_num as carNum,
            brand,
            guide_price as guidePrice,
            produce_time as produceTime,
            car_type as carType
        from t_car
        order by guide_price desc </select>
```

</mapper>

- 在核心配置文件中注册它
 - 打开 mybatis-config.xml 文件
 - 在 `<mappers>` 标签里把它加进去

```
<mappers>
  <mapper resource="CarMapper.xml"/>
  <mapper resource="CarMapper2.xml"/>
</mappers>
```

- 编写测试用例验证

```
/**
 * 测试用例：深入理解 namespace 的隔离作用
 */
@Test
public void testNamespace() {
    SqlSession sqlSession = null;
    try {
        sqlSession = SqlSessionUtil.openSession();

        System.out.println("==== 正在执行 car.selectAllCars ====");
        // 明确指出：我要调用 namespace 为 car 里面的 selectAllCars
        List<Car> cars1 = sqlSession.selectList("car.selectAllCars");
        for (Car car : cars1) {
            System.out.println(car);
        }

        System.out.println("\n==== 正在执行 car2.selectAllCars ====");
        // 明确指出：我要调用 namespace 为 car2 里面的 selectAllCars
        List<Car> cars2 = sqlSession.selectList("car2.selectAllCars");
        for (Car car : cars2) {
            System.out.println(car);
        }

    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        SqlSessionUtil.close(sqlSession);
    }
}
```

- 运行结果

```
D:\Java21\bin\java.exe -ea -Didea.test.cyclic.buffer.size=1048576 "-javaagent:D:\JAVAidea\IntelliJ IDEA
2023.2\lib\idea_rt.jar=8359:D:\JAVAidea\IntelliJ IDEA 2023.2\bin" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -
Dsun.stderr.encoding=UTF-8 -classpath "D:\JAVAidea\IntelliJ IDEA 2023.2\lib\idea_rt.jar;D:\JAVAidea\IntelliJ IDEA
2023.2\plugins\junit\lib\junit5-rt.jar;D:\JAVAidea\IntelliJ IDEA 2023.2\plugins\junit\lib\junit-
rt.jar;D:\Java_mybatis_study\my_mybatis_02_crud\target\classes;C:\Users\jack\.m2\repository\org\mybatis\mybatis\3.5.1
9\mybatis-3.5.19.jar;C:\Users\jack\.m2\repository\com\mysql\mysql-connector-j\8.4.0\mysql-connector-j-
8.4.0.jar;C:\Users\jack\.m2\repository\com\google\protobuf\protobuf-java\3.25.1\protobuf-java-
3.25.1.jar;C:\Users\jack\.m2\repository\junit\junit\4.13.2\junit-
4.13.2.jar;C:\Users\jack\.m2\repository\org\hamcrest\hamcrest-core\1.3\hamcrest-core-
1.3.jar;C:\Users\jack\.m2\repository\ch\qos\logback\logback-classic\1.5.6\logback-classic-
1.5.6.jar;C:\Users\jack\.m2\repository\ch\qos\logback\logback-core\1.5.6\logback-core-
1.5.6.jar;C:\Users\jack\.m2\repository\org\slf4j\slf4j-api\2.0.13\slf4j-api-2.0.13.jar"
com.intellij.rt.junit.JUnit4Starter -ideVersion5 -junit4 com.powernode.mybatis.CarMapperTest,testNamespace
```

```

2026-03-06 11:51:33.228 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class
org.apache.ibatis.logging.slf4j.Slf4jImpl' adapter.
2026-03-06 11:51:33.230 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class
org.apache.ibatis.logging.slf4j.Slf4jImpl' adapter.
2026-03-06 11:51:33.251 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.
2026-03-06 11:51:33.252 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.
2026-03-06 11:51:33.252 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.
2026-03-06 11:51:33.253 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.
==== 正在执行 car.selectAllCars ====
2026-03-06 11:51:33.324 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2026-03-06 11:51:33.775 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection
1380044647.
2026-03-06 11:51:33.777 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on
JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@5241cf67]
2026-03-06 11:51:33.784 [main] DEBUG car.selectAllCars - ==> Preparing: select id, car_num as carNum, brand,
guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car
2026-03-06 11:51:33.820 [main] DEBUG car.selectAllCars - ==> Parameters:
2026-03-06 11:51:33.887 [main] DEBUG car.selectAllCars - <==          Total: 10
Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}
Car{id=2, carNum='101', brand='奔驰E300L', guidePrice=54.0, produceTime='2022-08-01', carType='电车'}
Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=6, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=7, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10', carType='电车'}
Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}

==== 正在执行 car2.selectAllCars ====
2026-03-06 11:51:33.895 [main] DEBUG car2.selectAllCars - ==> Preparing: select id, car_num as carNum, brand,
guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car order by guide_price desc
2026-03-06 11:51:33.896 [main] DEBUG car2.selectAllCars - ==> Parameters:
2026-03-06 11:51:33.903 [main] DEBUG car2.selectAllCars - <==          Total: 10
Car{id=2, carNum='101', brand='奔驰E300L', guidePrice=54.0, produceTime='2022-08-01', carType='电车'}
Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}
Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=6, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=7, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10', carType='电车'}
2026-03-06 11:51:33.904 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true
on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@5241cf67]
2026-03-06 11:51:33.905 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection
[com.mysql.cj.jdbc.ConnectionImpl@5241cf67]
2026-03-06 11:51:33.906 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection
1380044647 to pool.

```

小结

在 MyBatis 的底层源码中，其实并不存在孤立的 id 这个东西。

我们在 Java 代码中执行 CRUD 操作时，传入的那个字符串，在 MyBatis 官方文档中被称为 statement（语句的唯一标识），我们习惯叫它 sqlId。

公式：`sqlId = namespace + . + id`

如果只写 id（比如只写 "selectAllCars"），在老版本的 MyBatis 中如果全局只有一个，它还能勉强跑通；**但如果有两个，直接报错。**

在SQL Mapper配置文件中 `<mapper>` 标签的namespace属性可以翻译为命名空间，这个命名空间主要是为了防止sqlId冲突的。

它是隔离 SQL 的身份证前缀，防重名。

它是连接 XML 映射文件和 Java 接口的唯一桥梁。后期学习

MyBatis核心配置文件详解

mybatis-config.xml文件中标签的编写顺序

在 MyBatis 的核心配置文件 mybatis-config.xml 中，标签的编写顺序是被严格写死、绝对不能颠倒的。

- 为什么必须按顺序写？
 - 在配置文件的第二行，我们总是会写这样一段代码：

```
<!DOCTYPE configuration
PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-config.dtd">
```

这个 `.dtd` 文件就是 MyBatis 官方制定的“宪法”。它里面有一行底层源码严格规定了 `<configuration>` 下面子标签出现的先后顺序。如果顺序写错了，IDEA 马上就会给你画红线报错，程序启动瞬间就会崩溃。

- 完整的 11 个标签顺序如下：

`<properties>`：属性，用来引入外部的 `jdbc.properties`。

`<settings>`：全局设置，用来配置 MyBatis 的全局开关，比如开启打印日志、开启驼峰命名自动映射等。

`<typeAliases>`：类型别名，用来给 Java 实体类起个短名字，以后写 XML 就不用写又长又臭的全限定类名了。

`<typeHandlers>`：类型处理器，Java 类型和数据库类型转换用的（日常开发极少直接手写）。

`<objectFactory>`：对象工厂，MyBatis 创建结果对象时用的（极少使用）。

`<objectWrapperFactory>`：对象包装工厂，极少使用。

`<reflectorFactory>`：反射工厂，极少使用。

`<plugins>`：插件，非常重要！以后我们在企业里做分页查询，引入的 `PageHelper` 分页插件就配置在这里。

`<environments>`：环境配置，配置多套数据库环境、事务管理器和数据源。

`<databaseIdProvider>`：数据库厂商标识，用于兼容多套不同数据库（比如同时兼容 Oracle 和 MySQL）。

`<mappers>`：映射器，用来注册 `XxxMapper.xml` 文件。

在 MyBatis 中，`mybatis-config.xml` 就是整个框架的核心总控室。框架怎么运行、连哪个数据库、开启哪些高级特性，全凭这个文件说了算。

- mybatis-config.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-config.dtd">
<!-- MyBatis 主配置文件 -->
<configuration>
```

```

<!-- 全局设置：配置日志实现为 SLF4J -->
<settings>
    <setting name="LogImpl" value="SLF4J"/>
</settings>

<!-- 配置环境，默认使用 development 环境 -->
<environments default="development">
    <!-- 开发环境配置 -->
    <environment id="development">
        <!-- 事务管理器：JDBC 事务 -->
        <transactionManager type="JDBC"/>
        <!-- 数据源：连接池方式 -->
        <dataSource type="POOLED">
            <!-- MySQL 驱动 -->
            <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
            <!-- 数据库连接 URL，包含编码和时区设置 -->
            <property name="url" value="jdbc:mysql://localhost:3306/mybatis_demo?
useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai"/>
            <!-- 数据库用户名 -->
            <property name="username" value="root"/>
            <!-- 数据库密码 -->
            <property name="password" value="20060329"/>
        </dataSource>
    </environment>
</environments>

<!-- 映射器配置：加载 CarMapper.xml 文件 -->
<mappers>
    <mapper resource="CarMapper.xml"/>
    <mapper resource="CarMapper2.xml"/>
</mappers>

</configuration>

```

- configuration：根标签，表示配置信息。
- environments：环境（多个），以“s”结尾表示复数，也就是说mybatis的环境可以配置多个数据源。
 - default属性：表示默认使用的是哪个环境，default后面填写的是environment的id。default的值只需要和environment的id值一致即可。
 - 可选值有三个
 - development：你自己在电脑上用的本地数据库。
 - test：测试工程师用的内网数据库。
 - production：真正面向用户的线上数据库。
- environment：具体的环境配置（主要包括：事务管理器的配置 + 数据源的配置）
 - id：给当前环境一个唯一标识，该标识用在environments的default后面，用来指定默认环境的选择。
- transactionManager：配置事务管理器
 - type属性：指定事务管理器具体使用什么方式，可选值包括两个
 - **JDBC**：使用JDBC原生的事务管理机制。底层原理：事务开启conn.setAutoCommit(false); ...处理业务...事务提交 conn.commit();
 - **MANAGED**：交给其它容器来管理事务，比如WebLogic、JBoss等。如果没有管理事务的容器，则没有事务。没有事务的含义：只要执行一条DML语句，则提交一次。
- dataSource：指定数据源
 - type属性：用来指定具体使用的数据库连接池的策略，可选值包括三个
 - **UNPOOLED**：采用传统的获取连接的方式，虽然也实现javax.sql.DataSource接口，但是并没有使用池的思想。
 - property可以是：
 - driver 这是 JDBC 驱动的 Java 类全限定名。

- url 这是数据库的 JDBC URL 地址。
 - username 登录数据库的用户名。
 - password 登录数据库的密码。
 - defaultTransactionIsolationLevel 默认的连接事务隔离级别。
 - defaultNetworkTimeout 等待数据库操作完成的默认网络超时时间（单位：毫秒）
- **POOLED**：采用传统的javax.sql.DataSource规范中的连接池，mybatis中有针对规范的实现。
 - property可以是（除了包含**UNPOOLED**中之外）：
 - poolMaximumActiveConnections 在任意时间可存在的活动（正在使用）连接数量，默认值：10
 - poolMaximumIdleConnections 任意时间可能存在的空闲连接数。
 - 其它....
- **JNDI**：采用服务器提供的JNDI技术实现，来获取DataSource对象，不同的服务器所能拿到DataSource是不一样。如果不是web或者maven的war工程，JNDI是不能使用的。
 - property可以是（最多只包含以下两个属性）：
 - initial_context 这个属性用来在 InitialContext 中寻找上下文（即，initialContext.lookup(initial_context)）这是个可选属性，如果忽略，那么将会直接从 InitialContext 中寻找 data_source 属性。
 - data_source 这是引用数据源实例位置的上下文路径。提供了 initial_context 配置时会会在其返回的上下文中进行查找，没有提供时则直接在 InitialContext 中查找。
- mappers：在mappers标签中可以配置多个sql映射文件的路径。
- mapper：配置某个sql映射文件的路径
 - resource属性：使用相对于类路径的资源引用方式
 - url属性：使用完全限定资源定位符（URL）方式

准备工作

- 创建 **my_mybatis_03_configuration** 项目
- 配置 pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>my_mybatis_03_configuration</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.mybatis</groupId>
      <artifactId>mybatis</artifactId>
      <version>3.5.19</version>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.4.0</version>
    </dependency>
  </dependencies>
```

```

    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>ch.qos.logback</groupId>
      <artifactId>logback-classic</artifactId>
      <version>1.5.6</version>
    </dependency>
  </dependencies>
</project>

```

- 补齐基础 Java 类和日志文件
 - Car.java 实体类

```

package org.example.pojo;

/**
 * 汽车实体类，对应数据库中的 t_car 表
 */
public class Car {

    private Long id;           // 对应 id
    private String carNum;     // 对应 car_num
    private String brand;      // 对应 brand
    private Double guidePrice; // 对应 guide_price
    private String produceTime; // 对应 produce_time
    private String carType;    // 对应 car_type

    // 无参构造
    public Car() {
    }

    // 全参构造
    public Car(Long id, String carNum, String brand, Double guidePrice, String produceTime, String carType) {
        this.id = id;
        this.carNum = carNum;
        this.brand = brand;
        this.guidePrice = guidePrice;
        this.produceTime = produceTime;
        this.carType = carType;
    }

    // Getter 和 Setter (MyBatis 核心依赖)
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getCarNum() { return carNum; }
    public void setCarNum(String carNum) { this.carNum = carNum; }

    public String getBrand() { return brand; }
    public void setBrand(String brand) { this.brand = brand; }

    public Double getGuidePrice() { return guidePrice; }
    public void setGuidePrice(Double guidePrice) { this.guidePrice = guidePrice; }

    public String getProduceTime() { return produceTime; }
    public void setProduceTime(String produceTime) { this.produceTime = produceTime; }

    public String getCarType() { return carType; }

```

```

public void setCarType(String carType) { this.carType = carType; }

@Override
public String toString() {
    return "Car{" +
        "id=" + id +
        ", carNum='" + carNum + '\'' +
        ", brand='" + brand + '\'' +
        ", guidePrice=" + guidePrice +
        ", produceTime='" + produceTime + '\'' +
        ", carType='" + carType + '\'' +
        '}';
}
}

```

- SqlSessionUtil.java 工具类

```

package org.example.utils;

import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;

import java.io.IOException;

public class SqlSessionUtil {
    private static SqlSessionFactory sqlSessionFactory;
    private static ThreadLocal<SqlSession> local = new ThreadLocal<>();
    private SqlSessionUtil() {}

    static {
        try {
            sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-
config.xml"));
        } catch (IOException e) {
            e.printStackTrace();
            throw new RuntimeException("解析 mybatis-config.xml 失败: ", e);
        }
    }

    public static SqlSession openSession() {
        SqlSession sqlSession = local.get();
        if (sqlSession == null) {
            sqlSession = sqlSessionFactory.openSession();
            local.set(sqlSession);
        }
        return sqlSession;
    }

    public static void close(SqlSession sqlSession) {
        if (sqlSession != null) {
            sqlSession.close();
            local.remove();
        }
    }
}

```

- logback.xml 日志配置

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration debug="false">

```

```
<property name="LOG_HOME" value="./Logs" />

<appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
  <encoder class="ch.qos.logback.classic.encoder.PatternLayoutEncoder">
    <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50} - %msg%n</pattern>
  </encoder>
</appender>

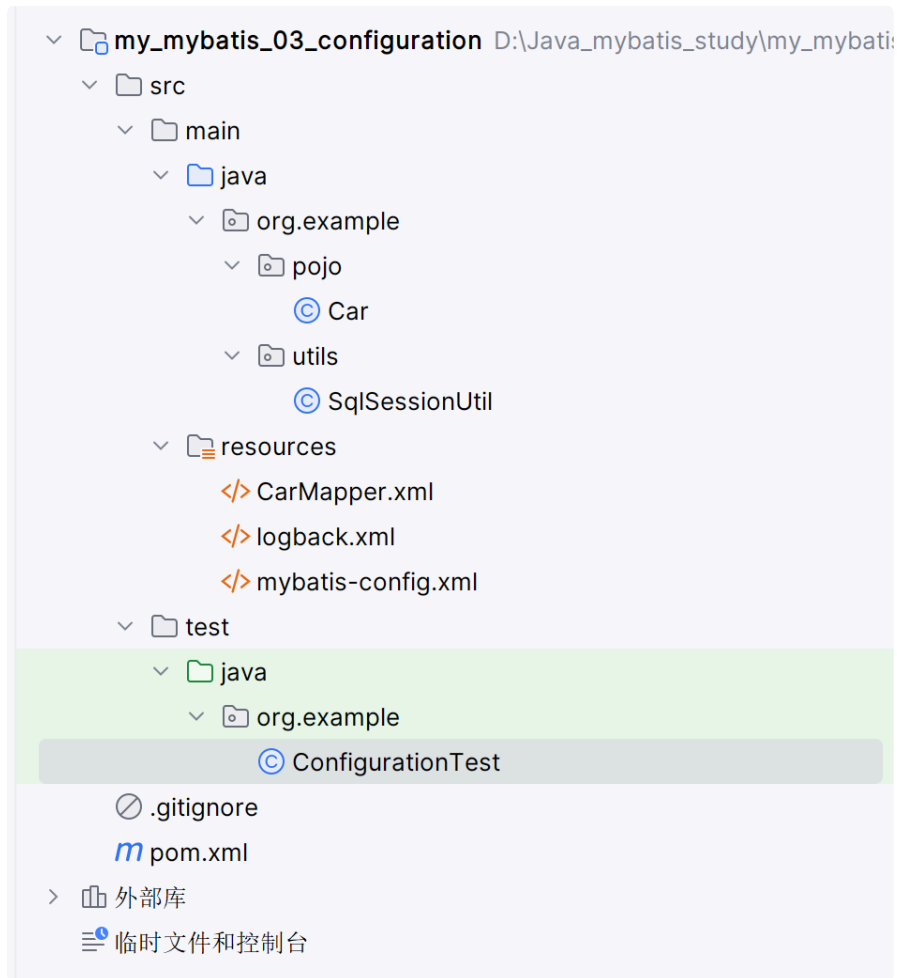
<appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <rollingPolicy class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
    <FileNamePattern>${LOG_HOME}/mybatis_demo.%d{yyyy-MM-dd}.%i.log</FileNamePattern>
    <maxFileSize>100MB</maxFileSize>
    <maxHistory>30</maxHistory>
  </rollingPolicy>
  <encoder class="ch.qos.logback.classic.encoder.PatternLayoutEncoder">
    <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50} - %msg%n</pattern>
  </encoder>
</appender>

<logger name="org.apache.ibatis" level="TRACE"/>
<logger name="java.sql.Connection" level="DEBUG"/>
<logger name="java.sql.Statement" level="DEBUG"/>
<logger name="java.sql.PreparedStatement" level="DEBUG"/>

<logger name="car" level="DEBUG"/>

<root level="DEBUG">
  <appender-ref ref="STDOUT"/>
  <appender-ref ref="FILE"/>
</root>

</configuration>
```



environments

在 MyBatis 中，`<environments>` 标签的作用可以用一句话来概括：一套代码，无缝切换多套数据库环境。

常见的三个的数据库环境

1. development：开发环境，电脑本地 MySQL。
2. test：测试环境，写完代码后，交给测试工程师找 Bug 时用的内部服务器数据库。
3. production：生产环境，项目正式上线，真实用户访问的线上数据库。

MyBatis 允许你把这三个环境全部配置在一个文件里，接下来我们来逐层“解剖”这个标签：

- 配置 mybatis-config.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
    PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

    <environments default="dev">

        <environment id="dev">
            <transactionManager type="JDBC"/>
            <dataSource type="POOLED">
                <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
                <property name="url" value="jdbc:mysql://localhost:3306/mybatis_demo?
useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai"/>
                <property name="username" value="root"/>
            </dataSource>
        </environment>
    </environments>
</configuration>
```

```

        <property name="password" value="20060329"/>
    </dataSource>
</environment>

<environment id="test">
    <transactionManager type="JDBC"/>
    <dataSource type="POOLED">
        <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
        <property name="url" value="jdbc:mysql://localhost:3306/mybatis_test?
useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai"/>
        <property name="username" value="root"/>
        <property name="password" value="20060329"/>
    </dataSource>
</environment>

<environment id="prod">
    <transactionManager type="JDBC"/>
    <dataSource type="POOLED">
        <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
        <property name="url" value="jdbc:mysql://localhost:3306/mybatis_prod?
useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai"/>
        <property name="username" value="root"/>
        <property name="password" value="20060329"/>
    </dataSource>
</environment>

</environments>

<mappers>
    <mapper resource="CarMapper.xml"/>
</mappers>

</configuration>

```

- 配置 CarMapper.xml

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="car">
    <insert id="insertCar" parameterType="org.example.pojo.Car">
        insert into t_car(car_num, brand, guide_price, produce_time, car_type)
        values(#{carNum}, #{brand}, #{guidePrice}, #{produceTime}, #{carType})
    </insert>
</mapper>

```

- 编写 ConfigurationTest
 - SqlSessionFactoryBuilder 的 build() 方法，其实支持传入第二个参数。如果你传了第二个参数，MyBatis 就会无视 XML 里的 default 属性，强行激活你指定的那个环境。

```

package org.example;

import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;
import org.example.pojo.Car;
import org.junit.Test;

```

```

import java.io.InputStream;

public class ConfigurationTest {

    @Test
    public void testEnvironment() throws Exception {
        SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();

        // 测试 1: 使用默认环境 (dev)
        InputStream is1 = Resources.getResourceAsStream("mybatis-config.xml");
        // 不传第二个参数, 读取 XML 里的 default="dev"
        SqlSessionFactory factoryDev = builder.build(is1);
        SqlSession sessionDev = factoryDev.openSession();

        Car car1 = new Car(null, "DEV-001", "程序员本地自测车", 10.0, "2026-01-01", "燃油车");
        sessionDev.insert("car.insertCar", car1);
        sessionDev.commit();
        sessionDev.close();
        System.out.println("成功: 已经向【开发库 mybatis_demo】插入了一条数据!");

        // 测试 2: 强制切换到测试环境 (test)
        InputStream is2 = Resources.getResourceAsStream("mybatis-config.xml");
        // 传入 "test", 强制激活测试环境
        SqlSessionFactory factoryTest = builder.build(is2, "test");
        SqlSession sessionTest = factoryTest.openSession();

        Car car2 = new Car(null, "TEST-002", "测试组找Bug专用车", 20.0, "2026-02-02", "电车");
        sessionTest.insert("car.insertCar", car2);
        sessionTest.commit();
        sessionTest.close();
        System.out.println("成功: 已经向【测试库 mybatis_test】插入了一条数据!");

        // 测试 3: 强制切换到生产环境 (prod)
        InputStream is3 = Resources.getResourceAsStream("mybatis-config.xml");
        // 传入 "prod", 强制激活生产环境
        SqlSessionFactory factoryProd = builder.build(is3, "prod");
        SqlSession sessionProd = factoryProd.openSession();

        Car car3 = new Car(null, "PROD-003", "真实用户购买的量产车", 30.0, "2026-03-03", "氢能源");
        sessionProd.insert("car.insertCar", car3);
        sessionProd.commit();
        sessionProd.close();
        System.out.println("成功: 已经向【生产库 mybatis_prod】插入了一条数据!");
    }
}

```

- 运行后

id # bigint	car_num VARCHAR(255)	brand VARCHAR(255)	guide_price # decimal(10,2)	produce_time CHAR(10)	car_type VARCHAR(255)
1	100	宝马520Li	41.00	2022-09-01	燃油车
2	101	奔驰E300L	54.00	2022-08-01	电车
3	102	丰田未来	40.30	2014-10-05	氢能源
4	102	丰田未来	40.30	2014-10-05	氢能源
5	102	丰田未来	40.30	2014-10-05	氢能源
6	102	丰田未来	40.30	2014-10-05	氢能源
7	102	丰田未来	40.30	2014-10-05	氢能源
8	102	丰田未来	40.30	2014-10-05	氢能源
9	102	比亚迪汉	30.23	2018-09-10	电车
10	102	丰田未来	40.30	2014-10-05	氢能源
12	104	开发库测试车	10.00	2026-01-01	燃油车
13	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车
14	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车
15	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车

• 开发环境 (dev)成功运行

```

2026-03-06 21:13:16.715 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4j
2026-03-06 21:13:16.722 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connectio
2026-03-06 21:13:16.723 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connectio
2026-03-06 21:13:16.723 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connectio
2026-03-06 21:13:16.723 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connectio
2026-03-06 21:13:16.760 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2026-03-06 21:13:17.045 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 1415439780.
2026-03-06 21:13:17.046 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql
2026-03-06 21:13:17.049 [main] DEBUG car.insertCar - ==> Preparing: insert into t_car(car_num, brand, guide_price, produce_time, car_type) values
2026-03-06 21:13:17.074 [main] DEBUG car.insertCar - ==> Parameters: DEV-001(String), 程序员本地自测车(String), 10.0(Double), 2026-01-01(String), 燃油
2026-03-06 21:13:17.077 [main] DEBUG car.insertCar - <== Updates: 1
2026-03-06 21:13:17.078 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection [com.mysql.cj.jdbc.ConnectionIn
2026-03-06 21:13:17.080 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysq
2026-03-06 21:13:17.080 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl
2026-03-06 21:13:17.081 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 1415439780 to pool.
成功: 已经向【开发库 mybatis_demo】插入了一条数据!
2026-03-06 21:13:17.086 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connectio
2026-03-06 21:13:17.086 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connectio
2026-03-06 21:13:17.086 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connectio
2026-03-06 21:13:17.086 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connectio
2026-03-06 21:13:17.092 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection

```

• 测试环境 (test)与生产环境 (prod)


```
org.apache.ibatis.exceptions.PersistenceException:
### Error updating database. Cause: java.sql.SQLException: Unknown database 'mybatis_test'
### The error may exist in CarMapper.xml
### The error may involve car.insertCar
### The error occurred while executing an update
### Cause: java.sql.SQLException: Unknown database 'mybatis_test'
```

```
at org.apache.ibatis.exceptions.ExceptionFactory.wrapException(ExceptionFactory.java:30)
at org.apache.ibatis.session.defaults.DefaultSqlSession.update(DefaultSqlSession.java:199)
at org.apache.ibatis.session.defaults.DefaultSqlSession.insert(DefaultSqlSession.java:184)
> at org.example.ConfigurationTest.testEnvironment(ConfigurationTest.java:37) <25 个内部行>
Caused by: java.sql.SQLException: Create breakpoint : Unknown database 'mybatis_test'
at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:112)
at com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.java:113)
at com.mysql.cj.jdbc.ConnectionImpl.createNewIO(ConnectionImpl.java:861)
at com.mysql.cj.jdbc.ConnectionImpl.<init>(ConnectionImpl.java:449)
at com.mysql.cj.jdbc.ConnectionImpl.getInstance(ConnectionImpl.java:234)
at com.mysql.cj.jdbc.NonRegisteringDriver.connect(NonRegisteringDriver.java:180)
at java.sql/java.sql.DriverManager.getConnection(DriverManager.java:683)
at java.sql/java.sql.DriverManager.getConnection(DriverManager.java:191)
at org.apache.ibatis.datasource.unpooled.UnpooledDataSource.doGetConnection(UnpooledDataSource.java:229)
at org.apache.ibatis.datasource.unpooled.UnpooledDataSource.doGetConnection(UnpooledDataSource.java:224)
```

语法错误：未知的数据库 **mybatis_test**

MyBatis 非常听话地读取了你在 mybatis-config.xml 里配置的 **<environment id="test">**。它看到 URL 里写着要去连接

localhost:3306/mybatis_test。它跑去敲 MySQL 的门，结果 MySQL 告诉它：“我这里只有 mybatis_demo，根本没有叫 mybatis_test 的数据库”

- 怎么解决这个错误？
 - 方法一：创建 mybatis_test、mybatis_prod 数据库，在库里创建和 mybatis_demo 数据库一样的 t_car 表
 - 方法二：dev、test、prod 都使用 mybatis_demo 数据库
- 运行结果

```
2026-03-06 21:20:38.812 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class
org.apache.ibatis.logging.slf4j.Slf4jImpl' adapter.
2026-03-06 21:20:38.820 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.
2026-03-06 21:20:38.820 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.
2026-03-06 21:20:38.821 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.
2026-03-06 21:20:38.821 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.
2026-03-06 21:20:38.859 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2026-03-06 21:20:39.146 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection
1415439780.
2026-03-06 21:20:39.147 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on
JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@545de5a4]
2026-03-06 21:20:39.151 [main] DEBUG car.insertCar - ==> Preparing: insert into t_car(car_num, brand, guide_price,
produce_time, car_type) values(?, ?, ?, ?, ?)
2026-03-06 21:20:39.175 [main] DEBUG car.insertCar - ==> Parameters: DEV-001(String), 程序员本地自测车(String),
10.0(Double), 2026-01-01(String), 燃油车(String)
2026-03-06 21:20:39.178 [main] DEBUG car.insertCar - <== Updates: 1
2026-03-06 21:20:39.179 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection
[com.mysql.cj.jdbc.ConnectionImpl@545de5a4]
2026-03-06 21:20:39.182 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true
on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@545de5a4]
2026-03-06 21:20:39.182 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection
[com.mysql.cj.jdbc.ConnectionImpl@545de5a4]
```

2026-03-06 21:20:39.182 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 1415439780 to pool.

成功: 已经向【开发库 mybatis_demo】插入了一条数据!

2026-03-06 21:20:39.187 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.

2026-03-06 21:20:39.187 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.

2026-03-06 21:20:39.187 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.

2026-03-06 21:20:39.187 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.

2026-03-06 21:20:39.192 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection

2026-03-06 21:20:39.207 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 546796969.

2026-03-06 21:20:39.207 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@209775a9]

2026-03-06 21:20:39.208 [main] DEBUG car.insertCar - ==> Preparing: insert into t_car(car_num, brand, guide_price, produce_time, car_type) values(?, ?, ?, ?, ?)

2026-03-06 21:20:39.209 [main] DEBUG car.insertCar - ==> Parameters: TEST-002(String), 测试组找Bug专用车(String), 20.0(Double), 2026-02-02(String), 电车(String)

2026-03-06 21:20:39.210 [main] DEBUG car.insertCar - <== Updates: 1

2026-03-06 21:20:39.210 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@209775a9]

2026-03-06 21:20:39.212 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@209775a9]

2026-03-06 21:20:39.213 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@209775a9]

2026-03-06 21:20:39.213 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 546796969 to pool.

成功: 已经向【测试库 mybatis_test】插入了一条数据!

2026-03-06 21:20:39.222 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.

2026-03-06 21:20:39.222 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.

2026-03-06 21:20:39.222 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.

2026-03-06 21:20:39.222 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.

2026-03-06 21:20:39.227 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection

2026-03-06 21:20:39.238 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 145066402.

2026-03-06 21:20:39.238 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@8a589a2]

2026-03-06 21:20:39.238 [main] DEBUG car.insertCar - ==> Preparing: insert into t_car(car_num, brand, guide_price, produce_time, car_type) values(?, ?, ?, ?, ?)

2026-03-06 21:20:39.239 [main] DEBUG car.insertCar - ==> Parameters: PROD-003(String), 真实用户购买的量产车(String), 30.0(Double), 2026-03-03(String), 氢能源(String)

2026-03-06 21:20:39.240 [main] DEBUG car.insertCar - <== Updates: 1

2026-03-06 21:20:39.240 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@8a589a2]

2026-03-06 21:20:39.242 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@8a589a2]

2026-03-06 21:20:39.242 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@8a589a2]

2026-03-06 21:20:39.242 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 145066402 to pool.

成功: 已经向【生产库 mybatis_prod】插入了一条数据!

# bigint	car_num varchar(255)	brand varchar(255)	guide_price decimal(10,2)	produce_time char(10)	car_type varchar(255)
2	101	奔驰E300L	54.00	2022-08-01	电车
3	102	丰田未来	40.30	2014-10-05	氢能源
4	102	丰田未来	40.30	2014-10-05	氢能源
5	102	丰田未来	40.30	2014-10-05	氢能源
6	102	丰田未来	40.30	2014-10-05	氢能源
7	102	丰田未来	40.30	2014-10-05	氢能源
8	102	丰田未来	40.30	2014-10-05	氢能源
9	102	比亚迪汉	30.23	2018-09-10	电车
10	102	丰田未来	40.30	2014-10-05	氢能源
12	104	开发库测试车	10.00	2026-01-01	燃油车
13	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车
14	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车
15	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车
16	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车
17	TEST-002	测试组找Bug专用车	20.00	2026-02-02	电车
18	PROD-003	真实用户购买的量	30.00	2026-03-03	氢能源

transactionManager

<transactionManager> (事务管理器)

在 MyBatis 中，配置事务管理器只需要一个 type 属性，它内置了两个选项，代表了 MyBatis 对待事务的两种事务管理态度：

- **type="JDBC"**
 - MyBatis 底层会直接抓取 java.sql.Connection 对象，亲自调用它的 commit() 和 rollback() 方法。
 - 现象：必须在 Java 代码里老老实实地写上 sqlSession.commit()，数据才能真正在数据库里落库。这也是我们目前一直使用的方式。
- **type="MANAGED"** :
 - MyBatis 表示：“我只负责生成和执行 SQL，至于事务提交和回滚？我不干了，交给外面的大哥（比如 Spring 容器或 Java EE 服务器）去管吧！”
 - 现象：如果在没有 Spring 等容器的纯净环境里，你把它设为 MANAGED，即使你代码里写了 sqlSession.commit()，MyBatis 也会直接无视它。最终结果就是：SQL 执行了，但数据永远不会存进数据库！
- 创建 mybatis-config.xml
 - 把事务管理器换成 MANAGED

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
  PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
  "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

  <environments default="dev">
    <environment id="dev">
      <transactionManager type="MANAGED"/>
      <dataSource type="POOLED">
        <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
        <property name="url" value="jdbc:mysql://localhost:3306/mybatis_demo?useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai"/>
        <property name="username" value="root"/>
      </dataSource>
    </environment>
  </environments>

```

```

        <property name="password" value="20060329"/>
    </dataSource>
</environment>
</environments>

<mappers>
    <mapper resource="CarMapper.xml"/>
</mappers>

</configuration>

```

- 编写 ConfigurationTest.testTransactionManager

```

/**
 * 测试用例：验证 MANAGED 事务管理器的“甩手掌柜”行为
 */
@Test
public void testTransactionManager() throws Exception {
    SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();

    // 【关键】：这里读取的是我们刚刚新建的 mybatis-config2.xml
    InputStream is = Resources.getResourceAsStream("mybatis-config2.xml");
    SqlSessionFactory factory = builder.build(is);
    SqlSession session = factory.openSession();

    // 准备一辆特征极其明显的测试车
    Car car = new Car(null, "MANAGED-999", "MANAGED测试车", 99.9, "2026-10-10", "燃油车");

    // 执行插入操作
    int count = session.insert("car.insertCar", car);
    System.out.println("代码执行情况：成功执行了 insert SQL, MyBatis 告诉你影响了 " + count + " 行数据。");

    // 【重头戏】：我们老老实实在地手写了 commit 提交事务
    session.commit();
    System.out.println("代码执行情况：已经调用了 session.commit()。");

    // 释放资源
    session.close();

    System.out.println("=====");
    System.out.println("请现在打开 Navicat，刷新 mybatis_demo 库的 t_car 表！");
    System.out.println("看看那辆叫 'MANAGED-999' 的车真的存进去了吗？");
}

```

- 运行后
 - 看控制台的日志，会发现没有任何报错。
 - 但是，当你打开 Navicat 去刷新表时，会发现没有 MANAGED-999 这辆车

这证明了，在 type="MANAGED" 模式下，MyBatis 真的把事务控制权交出去了。因为我们现在是纯本地 Java 环境，外面没有 Spring 容器帮我们接

管事务，所以这个事务在结束时被直接回滚了，写的 session.commit() 被当成了废气。

这也是为什么在日后学习 Spring 整合 MyBatis 之前，我们在单独练习 MyBatis 时，必须死死绑定 type="JDBC" 的原因

- 如果在MySQL数据库看到了这一条数据 MANAGED-999，原因如下：

在使用 type="JDBC" 事务状态时，在执行SQL语句之前，有一行日志如下



```
2026-03-06 21:20:39.147 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on
JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@545de5a4]
```

翻译：MyBatis 拿到了数据库连接后，第一件事就是主动对 MySQL 说：“把你的自动提交功能（autocommit）给我关掉（设为 false），接下来的数据

不能随便存，必须听到我的 commit 命令才能存！”

结果：因为自动提交被关了，所以如果你在 Java 代码里不手动写 session.commit()，刚才执行的插入操作就会被撤销（回滚），数据不会进数据库。



```
2026-03-06 21:37:32.672 [main] DEBUG o.a.ibatis.transaction.managed.ManagedTransaction - Opening JDBC Connection
2026-03-06 21:37:32.956 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection
198098993.
```

翻译：MyBatis 拿到了数据库连接后，什么都没做（Opening 之后直接就 Created connection 去准备执行 SQL 了）。

结果：因为 MyBatis 没有去关掉自动提交，而 MySQL 数据库就是默认开启自动提交（autocommit = true）的。所以，当你的 insert 语句发给 MySQL 时，MySQL 直接将数据存了进去。

- 怎么解决？
 - 改回 type="JDBC"
 - 用 Java 代码手动关闭自动提交



@Test

```
public void testTransactionManager() throws Exception {
    // ... 前面获取 session 的代码保持不变
    SqlSession session = factory.openSession();

    // 【硬核操作】：伸手进去拿到最底层的 JDBC Connection，强行关闭自动提交！
    session.getConnection().setAutoCommit(false);

    Car car = new Car(null, "MANAGED-Hack", "手动拦截车", 88.8, "2026-11-11", "燃油车");
    session.insert("car.insertCar", car);

    // 因为自动提交被你关了，如果你下面不写 commit，这辆车绝对存不进去！
    // session.commit();

    session.close();
}
```

- 交给 Spring 框架
 - 在真实的企业级开发中，我们会引入强大的 Spring 框架。Spring 里面有一个组件叫 DataSourceTransactionManager（数据源事务管理器）。
 - 当你把 MyBatis 整合进 Spring 后，XML 里配成 MANAGED，此时 Spring 框架就会化身为那个“管家”，它会在最外层全盘接管数据库连接，自动帮你执行 setAutoCommit(false)，自动帮你 commit，报错了自动帮你 rollback。这时候，MyBatis 只需要安心写 SQL 就行了。

当事务管理器是：JDBC

- 采用JDBC的原生事务机制：
- 开启事务：conn.setAutoCommit(false);
- 处理业务.....
- 提交事务：conn.commit();

当事务管理器是：MANAGED

- 交给容器去管理事务，但目前使用的是本地程序，没有容器的支持，**当mybatis找不到容器的支持时：没有事务**。也就是说只要执行一条DML语句，则提交一次。

dataSource

在企业级高并发开发中，数据库连接是非常昂贵的资源。

每次跟 MySQL 握手建立物理连接都要消耗大量时间和内存。

- MyBatis 提供了两种最基础的数据源：
 - POOLED：使用连接池（提前建好一堆连接放在池子里，用完不关，放回池子复用）。企业绝对主力。
- UNPOOLED：不使用连接池（每次要执行 SQL，都老老实实去向 MySQL 申请建立一次全新的网络连接，用完立刻物理关闭）。极度浪费，仅作学习对比用。
- 创建 mybatis-config3.xml
 - type 改为 UNPOOLED

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
    PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

    <environments default="dev">
        <environment id="dev">
            <transactionManager type="JDBC"/>

            <dataSource type="UNPOOLED">
                <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
                <property name="url" value="jdbc:mysql://localhost:3306/mybatis_demo?
useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai"/>
                <property name="username" value="root"/>
                <property name="password" value="20060329"/>
            </dataSource>
        </environment>
    </environments>

    <mapppers>
        <mapper resource="CarMapper.xml"/>
    </mapppers>

</configuration>
```

- 编写 ConfigurationTest.testDataSource

```
/**
 * 测试用例：验证 UNPOOLED 数据源的底层行为
 */
@Test
public void testDataSource() throws Exception {
    SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();
    InputStream is = Resources.getResourceAsStream("mybatis-config3.xml");
    SqlSessionFactory factory = builder.build(is);

    System.out.println("===== 第一次执行 SQL =====");
    SqlSession session1 = factory.openSession();
    Car car1 = new Car(null, "UNPOOLED-01", "非连接池测试车1", 10.0, "2026-04-01", "燃油车");
    session1.insert("car.insertCar", car1);
    session1.commit();
    session1.close(); // 关闭会话
```



```

System.out.println("\n===== 第二次执行 SQL =====");
SqlSession session2 = factory.openSession();
Car car2 = new Car(null, "UNPOOLED-02", "非连接池测试车2", 20.0, "2026-04-02", "燃油车");
session2.insert("car.insertCar", car2);
session2.commit();
session2.close(); // 再次关闭会话
}

```

- 这是一个“连续两次向数据库插入数据”的测试。
- 在有连接池的情况下，第二次操作会直接复用第一次的连接；但在 UNPOOLED 下，它会重新建立连接。
- 运行代码

```

21\bin\java.exe ...
-07 20:42:22.743 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class org.apache.ibatis.logging.slf4j.Slf4jImpl' a
===== 第一次执行 SQL =====
-07 20:42:22.753 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
-07 20:42:23.162 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jdbc
-07 20:42:23.172 [main] DEBUG car.insertCar - ==> Preparing: insert into t_car(car_num, brand, guide_price, produce_time, car_type) values(?, ?,
-07 20:42:23.199 [main] DEBUG car.insertCar - ==> Parameters: UNPOOLED-01(String), 非连接池测试车1(String), 10.0(Double), 2026-04-01(String), 燃油车(S
-07 20:42:23.205 [main] DEBUG car.insertCar - <== Updates: 1
-07 20:42:23.206 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@3185
-07 20:42:23.210 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.cj.jdb
-07 20:42:23.210 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@3185fa6

===== 第二次执行 SQL =====
-07 20:42:23.212 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
-07 20:42:23.230 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jdbc
-07 20:42:23.231 [main] DEBUG car.insertCar - ==> Preparing: insert into t_car(car_num, brand, guide_price, produce_time, car_type) values(?, ?,
-07 20:42:23.232 [main] DEBUG car.insertCar - ==> Parameters: UNPOOLED-02(String), 非连接池测试车2(String), 20.0(Double), 2026-04-02(String), 燃油车(S
-07 20:42:23.233 [main] DEBUG car.insertCar - <== Updates: 1
-07 20:42:23.234 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@7bf9
-07 20:42:23.236 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.cj.jdb
-07 20:42:23.236 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@7bf9b09

```

修改配置文件mybatis-config3.xml中的配置：

- type 改为 POOLED：

```

<dataSource type="POOLED">

```

Java测试程序不需要修改，直接执行，看控制台输出：

```

33-07 20:45:12.105 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.
33-07 20:45:12.105 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.
33-07 20:45:12.105 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully closed/removed all connections.
===== 第一次执行 SQL =====
33-07 20:45:12.146 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
33-07 20:45:12.433 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 1415439780.
33-07 20:45:12.434 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jd
33-07 20:45:12.438 [main] DEBUG car.insertCar - ==> Preparing: insert into t_car(car_num, brand, guide_price, produce_time, car_type) values(?, ?
33-07 20:45:12.466 [main] DEBUG car.insertCar - ==> Parameters: UNPOOLED-01(String), 非连接池测试车1(String), 10.0(Double), 2026-04-01(String), 燃油车
33-07 20:45:12.470 [main] DEBUG car.insertCar - <== Updates: 1
33-07 20:45:12.472 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@54
33-07 20:45:12.478 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.cj.j
33-07 20:45:12.479 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@545de
33-07 20:45:12.480 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 1415439780 to pool.

===== 第二次执行 SQL =====
33-07 20:45:12.480 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
33-07 20:45:12.480 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Checked out connection 1415439780 from pool.
33-07 20:45:12.480 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jd
33-07 20:45:12.481 [main] DEBUG car.insertCar - ==> Preparing: insert into t_car(car_num, brand, guide_price, produce_time, car_type) values(?, ?
33-07 20:45:12.482 [main] DEBUG car.insertCar - ==> Parameters: UNPOOLED-02(String), 非连接池测试车2(String), 20.0(Double), 2026-04-02(String), 燃油车
33-07 20:45:12.483 [main] DEBUG car.insertCar - <== Updates: 1
33-07 20:45:12.484 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Committing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@54
33-07 20:45:12.486 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.cj.j
33-07 20:45:12.486 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@545de
33-07 20:45:12.486 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection 1415439780 to pool.

```

一致的

通过测试得出：UNPOOLED不会使用连接池，每一次都会新建JDBC连接对象。POOLED会使用数据库连接池。【这个连接池是mybatis自己实现的。】

- 修改配置文件mybatis-config3.xml中的配置：
 - type 改为 JNDI：

```
<dataSource type="JNDI">
```

JNDI的方式：表示对接JNDI服务器中的连接池。这种方式给了我们使用第三方连接池的接口。如果想使用dbcp、c3p0、druid（德鲁伊）等，需要使用这种方式。

- 这种再重点说一下type="POOLED"的时候，它的属性有哪些？
 - POOLED 数据源的属性分为两大类：基础连接属性和高级池化管理属性。
 - 基础连接属性；无论 POOLED 还是 UNPOOLED 都必须有的四大属性：
 - driver：数据库驱动。

- url：数据库连接地址。

- username：账号。

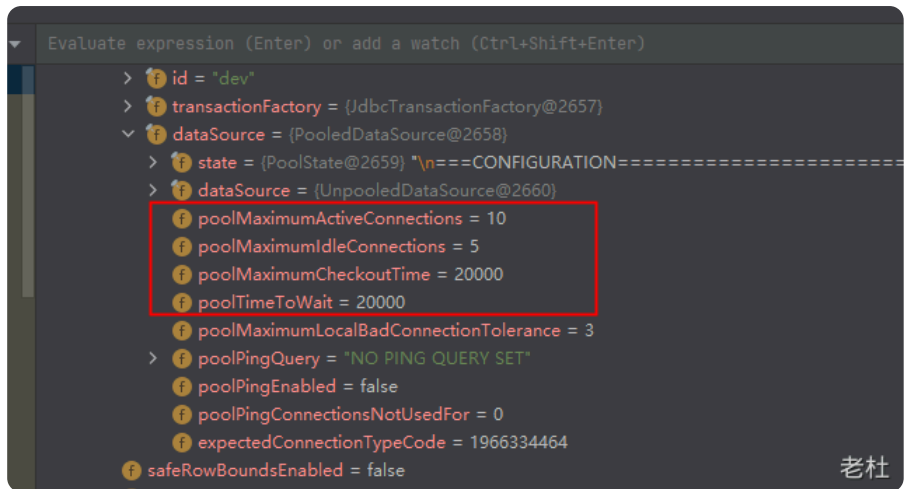
- password：密码。
- 高级池化管理属性
 - poolMaximumActiveConnections：最大的活动的连接数量。默认值10
 - poolMaximumIdleConnections：最大的空闲连接数量。默认值5
 - poolMaximumCheckoutTime：最长强行回收时间 / 最大使用时间。默认值20秒。
 - poolTimeToWait：最长等待时间。默认值：20秒。
 - poolPingEnabled 与 poolPingQuery：共同实现 MySQL 连接池的连接心跳保活能力
 - poolPingEnabled：心跳保活功能的启停总开关
 - poolPingQuery：心跳检测执行的专用 SQL 语句

当然，还有其他属性。对于连接池来说，以上几个属性比较重要。

- 以餐厅运营管理做为比喻：
 - 最大的活动的连接数量就是餐厅里最多允许同时有几桌客人在吃饭。默认值10，如果有10个请求正在使用这10个连接，第11个请求只能等待空闲连接。
 - 最大的空闲连接数量就是餐厅里最多保留几张空桌子随时待命。默认值5，如果已经有了5个空闲连接，当第6个连接要空闲下来的时候，连接池会选择关闭该连接对象。来减少数据库的开销。
 - 最长强行回收时间 / 最大使用时间就是一桌客人吃饭时间太长，超过多少毫秒，餐厅经理就会强行把他赶走，把桌子清空给别人用。为了防止连接泄露。默认值为20000毫秒。
 - 最长等待时间就是客人在门外排队，最多愿意等多久。默认值：20000 毫秒，如果池子里的连接全被占满了，新来的请求排队等了 20 秒还没拿到连接，MyBatis 就会放弃等待，并直接抛出异常，并且在日志里打印一条警告信息。
 - 连接心跳保活能力就是服务员隔一段时间去空桌子前喊两声，看看桌子坏没坏。
 - poolPingEnabled：是否开启心跳侦测，默认是 false。
 - poolPingQuery：侦测时发送的 SQL 语句。通常配置为极其简单的 select 1。如果 MySQL 成功返回了 1，说明这个连接还是活的；如果报错，MyBatis 就知道这个连接废了，会把它扔掉换个新的。

需要根据系统的并发情况，来合理调整连接池最大连接数以及最多空闲数量。充分发挥数据库连接池的性能。【可以根据实际情况进行测试，然后调整一个合理的数量。】

下图是默认配置：



在以上配置的基础之上，可以编写如下文件：

- 编写 mybatis-config4.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
    PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

    <environments default="dev">
        <environment id="dev">
            <transactionManager type="JDBC"/>

            <dataSource type="POOLED">
                <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
                <property name="url" value="jdbc:mysql://localhost:3306/mybatis_demo?useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai"/>
                <property name="username" value="root"/>
                <property name="password" value="20060329"/>

                <property name="poolMaximumActiveConnections" value="3"/>
                <property name="poolMaximumIdleConnections" value="2"/>
                <property name="poolMaximumCheckoutTime" value="10000"/>
                <property name="poolTimeToWait" value="2000"/>
                <property name="poolPingEnabled" value="true"/>
                <property name="poolPingQuery" value="select 1"/>
            </dataSource>
        </environment>
    </environments>

    <mappers>
        <mapper resource="CarMapper2.xml"/>
    </mappers>

</configuration>
```

- 编写 CarMapper2.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
```

```
PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
```

```
<mapper namespace="car2">

    <select id="selectCarByCarNum" resultType="org.example.pojo.Car">
        select
            id,
            car_num as carNum,
            brand,
            guide_price as guidePrice,
            produce_time as produceTime,
            car_type as carType
        from t_car
        where car_num = #{carNum}
    </select>

</mapper>
```

- 编写 ConfigurationTest.testPool

```
/**
 * 测试用例：极限压测，亲眼见证 POOLED 连接池的高级管理机制
 */
@Test
public void testPool() throws Exception {
    SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();
    InputStream is = Resources.getResourceAsStream("mybatis-config4.xml");
    SqlSessionFactory factory = builder.build(is);

    // 准备一个 List，用来“扣留”拿到的会话，防止它们被垃圾回收或自动归还
    List<SqlSession> activeSessions = new ArrayList<>();

    System.out.println("===== 第一阶段：测试最大活跃数 =====");
    try {
        // 我们的池子最大活跃数(Active)设为了 3。我们故意循环 4 次去拿！
        for (int i = 1; i <= 4; i++) {
            System.out.println("尝试获取第 " + i + " 个连接...");
            SqlSession session = factory.openSession();

            // 【核心强硬手段】：强行调用 getConnection()，逼 MyBatis 马上从池子里拿出一个真实的物理连接！
            session.getConnection();

            // 把连接扣留下来，坚决不调用 session.close()
            activeSessions.add(session);
            System.out.println("第 " + i + " 个连接获取成功！当前池子已被占用 " + i + " 个。\\n");
        }
    } catch (Exception e) {
        System.out.println("\\n触发连接池防御机制！拿不到第 4 个连接了！");
        System.out.println("报错原因：" + e.getMessage());
    }

    System.out.println("\\n===== 第二阶段：测试回收机制 =====");
    // 现在池子是被榨干的，我们把之前扣留的 3 个连接全部归还
    for (int i = 0; i < activeSessions.size(); i++) {
        System.out.println("准备归还第 " + (i + 1) + " 个连接...");
        // 调用 close()，对于 POOLED 来说，这就是把连接洗干净放回池子
        activeSessions.get(i).close();
    }
    System.out.println("所有连接已成功归还！");
}
```


2026-03-07 21:22:39.395 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Connection 836969741 is GOOD!
2026-03-07 21:22:39.397 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@31e3250d]
第 1 个连接获取成功! 当前池子已被占用 1 个。

尝试获取第 2 个连接...

2026-03-07 21:22:39.401 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2026-03-07 21:22:39.424 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 1910896157.
2026-03-07 21:22:39.425 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Testing connection 1910896157 ...
2026-03-07 21:22:39.426 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Connection 1910896157 is GOOD!
2026-03-07 21:22:39.426 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@71e5f61d]
第 2 个连接获取成功! 当前池子已被占用 2 个。

尝试获取第 3 个连接...

2026-03-07 21:22:39.426 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2026-03-07 21:22:39.440 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection 1373577787.
2026-03-07 21:22:39.440 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@51df223b]
第 3 个连接获取成功! 当前池子已被占用 3 个。

尝试获取第 4 个连接...

2026-03-07 21:22:39.441 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2026-03-07 21:22:39.441 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Waiting as long as 2000 milliseconds for connection.
2026-03-07 21:22:41.453 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Wait failed...
2026-03-07 21:22:41.455 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Waiting as long as 2000 milliseconds for connection.
2026-03-07 21:22:43.463 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Wait failed...
2026-03-07 21:22:43.464 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Waiting as long as 2000 milliseconds for connection.
2026-03-07 21:22:45.475 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Wait failed...
2026-03-07 21:22:45.476 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Waiting as long as 2000 milliseconds for connection.
2026-03-07 21:22:47.481 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Wait failed...
2026-03-07 21:22:47.482 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Waiting as long as 2000 milliseconds for connection.
2026-03-07 21:22:49.485 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Wait failed...
2026-03-07 21:22:49.489 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Claimed overdue connection 836969741.
2026-03-07 21:22:49.490 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Testing connection 836969741 ...
2026-03-07 21:22:49.493 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Connection 836969741 is GOOD!
第 4 个连接获取成功! 当前池子已被占用 4 个。

===== 第二阶段: 测试回收机制 =====

准备归还第 1 个连接...

2026-03-07 21:22:49.496 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Error resetting autocommit to true before closing the connection. Cause: java.sql.SQLException: Error accessing PooledConnection. Connection is invalid.
2026-03-07 21:22:49.496 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@31e3250d]
2026-03-07 21:22:49.496 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - A bad connection (836969741) attempted to return to the pool, discarding connection.
准备归还第 2 个连接...
2026-03-07 21:22:49.496 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@71e5f61d]

```
2026-03-07 21:22:49.497 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection
[com.mysql.cj.jdbc.ConnectionImpl@71e5f61d]
2026-03-07 21:22:49.498 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Testing connection
1910896157 ...
2026-03-07 21:22:49.499 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Connection 1910896157 is
GOOD!
2026-03-07 21:22:49.499 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection
1910896157 to pool.
准备归还第 3 个连接...
2026-03-07 21:22:49.499 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true
on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@51df223b]
2026-03-07 21:22:49.500 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection
[com.mysql.cj.jdbc.ConnectionImpl@51df223b]
2026-03-07 21:22:49.501 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Testing connection
1373577787 ...
2026-03-07 21:22:49.502 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Connection 1373577787 is
GOOD!
2026-03-07 21:22:49.502 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection
1373577787 to pool.
准备归还第 4 个连接...
2026-03-07 21:22:49.503 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true
on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@31e3250d]
2026-03-07 21:22:49.503 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection
[com.mysql.cj.jdbc.ConnectionImpl@31e3250d]
2026-03-07 21:22:49.504 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Testing connection 836969741
...
2026-03-07 21:22:49.505 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Connection 836969741 is
GOOD!
2026-03-07 21:22:49.508 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Closed connection 836969741.
所有连接已成功归还!

进程已结束, 退出代码为 0
```

properties

在之前的配置中，我们一直把数据库的 URL、账号、密码等敏感信息“硬编码”在 XML 文件里。在企业里，这是绝对的“大忌”：

- 不安全：XML 经常会被提交到代码库，密码容易泄露。
- 不好维护：将来如果要改密码，要在一大堆 XML 标签里找，很容易改错。

<properties> 标签的出现就是为了解耦！它可以引入外部独立的 .properties 属性文件。

- 编写 jdbc.properties
 - 提取外部配置文件



```
# 数据库连接配置
# 习惯上我们会加上一个统一的前缀，比如 jdbc.，防止和系统其他属性重名
jdbc.driver=com.mysql.cj.jdbc.Driver
jdbc.url=jdbc:mysql://localhost:3306/mybatis_demo?useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai
jdbc.username=root
jdbc.password=20060329
```

- 编写 mybatis-config.xml
 - 使用标签和占位符
 - MyBatis 对核心配置文件的标签顺序有极其严格的要求，**<properties>** 标签必须放在所有其他标签的最前面。



```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
```

```

PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

    <properties resource="jdbc.properties" />

    <environments default="dev">
        <environment id="dev">
            <transactionManager type="JDBC"/>
            <dataSource type="POOLED">
                <property name="driver" value="${jdbc.driver}"/>
                <property name="url" value="${jdbc.url}"/>
                <property name="username" value="${jdbc.username}"/>
                <property name="password" value="${jdbc.password}"/>
            </dataSource>
        </environment>
    </environments>

    <mappers>
        <mapper resource="CarMapper2.xml"/>
    </mappers>

</configuration>

```

- 编写 ConfigurationTest.testProperties

```

/**
 * 测试用例：验证 properties 标签是否成功读取了外部配置文件
 */
@Test
public void testProperties() throws Exception {
    // 1. 获取建造者
    SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();

    // 2. 读取刚刚写好的 mybatis-config5.xml
    InputStream is = Resources.getResourceAsStream("mybatis-config5.xml");
    SqlSessionFactory factory = builder.build(is);

    // 3. 打开会话
    SqlSession session = factory.openSession();

    System.out.println("成功加载 mybatis-config5.xml，即将测试数据库连接...");

    // 4. 复用上一次写的按车牌号查询的 SQL 语句来验证连接是否通畅
    // 注意：请将 "102" 替换为你数据库里真实存在的车牌号，否则查出来是个空集合
    java.util.List<Car> cars = session.selectList("car2.selectCarByCarNum", "102");

    System.out.println("==== 查询成功，证明外部 jdbc.properties 加载完美! ====");
    for (Car car : cars) {
        System.out.println(car);
    }

    // 5. 释放资源
    session.close();
}

```

运行后

```

2026-03-07 21:45:08.248 [main] DEBUG org.apache.ibatis.logging.LogFactory - Logging initialized using 'class
org.apache.ibatis.logging.slf4j.Slf4jImpl' adapter.
2026-03-07 21:45:08.257 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.

```

```

2026-03-07 21:45:08.257 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.
2026-03-07 21:45:08.257 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.
2026-03-07 21:45:08.258 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - PooledDataSource forcefully
closed/removed all connections.
成功加载 mybatis-config5.xml, 即将测试数据库连接...
2026-03-07 21:45:08.304 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Opening JDBC Connection
2026-03-07 21:45:08.642 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Created connection
1832283291.
2026-03-07 21:45:08.644 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Setting autocommit to false on
JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@6d366c9b]
2026-03-07 21:45:08.648 [main] DEBUG car2.selectCarByCarNum - ==> Preparing: select id, car_num as carNum, brand,
guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car where car_num = ?
2026-03-07 21:45:08.678 [main] DEBUG car2.selectCarByCarNum - ==> Parameters: 102(String)
2026-03-07 21:45:08.710 [main] DEBUG car2.selectCarByCarNum - <==          Total: 8
==== 查询成功, 证明外部 jdbc.properties 加载完美! ====
Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=6, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=7, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10', carType='电车'}
Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
2026-03-07 21:45:08.717 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Resetting autocommit to true
on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@6d366c9b]
2026-03-07 21:45:08.718 [main] DEBUG o.apache.ibatis.transaction.jdbc.JdbcTransaction - Closing JDBC Connection
[com.mysql.cj.jdbc.ConnectionImpl@6d366c9b]
2026-03-07 21:45:08.718 [main] DEBUG o.apache.ibatis.datasource.pooled.PooledDataSource - Returned connection
1832283291 to pool.

```

- properties两个属性：
 - resource：这个属性从类的根路径下开始加载。【常用的】
 - url：从指定的url加载，假设文件放在d:/jdbc.properties，这个url可以写成：file:///d:/jdbc.properties。注意是三个斜杠哦。
- 注意：如果不知道mybatis-config.xml文件中标签的编写顺序的话，可以有两种方式知道它的顺序：
 - 第一种方式：查看dtd约束文件。
 - 第二种方式：通过idea的报错提示信息。【一般采用这种方式】

mapper

在 MyBatis 中，无论 SQL 映射文件（如 **CarMapper.xml**）的逻辑编写得多么完善，若未在全局配置文件中完成映射资源的注册声明，MyBatis 将无法识别、解析并加载这些映射文件。

<mapper> 标签提供了好几种方式来指明映射文件在哪。现阶段我们主要掌握针对 XML 文件的两种核心路径指定法：**resource** 和 **url**。

- 编写 mybatis-config.xml
 - 第一种：resource，从类的根路径下开始加载【比url常用】

```

<mappers>
  <mapper resource="CarMapper.xml"/>
</mappers>

```

- 如果是这样写的话，必须保证类的根下有CarMapper.xml文件。
- 如果类的根路径下有一个包叫做test，CarMapper.xml如果放在test包下的话，这个配置应该是这样写：



```
<mappers>
  <mapper resource="test/CarMapper.xml"/>
</mappers>
```

- 第二种：url，从指定的url位置加载
 - 假设CarMapper.xml文件放在d盘的根下，这个配置就需要这样写：



```
<mappers>
  <mapper url="file:///d:/CarMapper.xml"/>
</mappers>
```

mapper还有其他的指定方式，后面再看！！

手写MyBatis框架（掌握原理）

警示：该部分内容有难度，基础较弱的程序员可能有些部分是听不懂的，如果无法跟下来，可直接跳过，不影响后续知识点的学习。当然，如果你能够跟下来，必然会让你加深对MyBatis框架的理解。

我们给自己的框架起个名：LztBatis

dom4j解析XML文件

MyBatis 的核心思想之一就是硬编码的 JDBC 连接信息和 SQL 语句剥离到 XML 配置文件中。

为了让 LztBatis 能够正常工作，它必须在始化阶段做两件事：

- 读取核心配置：lztbatis-config.xml，拿到数据库的 driver、url、username 和 password，以便底层利用 JDBC 建立数据库连接。



```
<configuration>
  <environments default="development">
    <environment id="development">
      <dataSource type="POOLED">
        <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
        <property name="url" value="jdbc:mysql://localhost:3306/mybatis_demo"/>
        <property name="username" value="root"/>
        <property name="password" value="20060329"/>
      </dataSource>
    </environment>
  </environments>
</configuration>
```

- 读取 SQL 映射：CarMapper.xml，拿到 SQL 语句、传入参数的类型（parameterType）以及返回结果的类型（resultType）。



```
<mapper namespace="com.lzt.mapper.CarMapper">
  <select id="selectCarById" resultType="com.lzt.pojo.Car">
    select * from t_car where id = #{id}
  </select>
</mapper>
```

dom4j 是 Java 领域极其流行且高效的 XML 解析工具。MyBatis 源码底层其实也就是通过类似的方式来精准提取 XML 节点信息的。

模块名：my_mybatis_04_lztbatis（普通的Java Maven模块）



dom4j 解析的步骤规划

- 第一步：搭建 pom.xml 的具体依赖
 - 第一步：引入dom4j的依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.lzt</groupId>
    <artifactId>my_mybatis_04_lztbatis</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.dom4j</groupId>
            <artifactId>dom4j</artifactId>
            <version>2.1.4</version>
        </dependency>

        <dependency>
            <groupId>jaxen</groupId>
            <artifactId>jaxen</artifactId>
            <version>2.0.0</version>
        </dependency>

        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>8.4.0</version>
        </dependency>

        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>

        <dependency>
```

```

        <groupId>ch.qos.logback</groupId>
        <artifactId>logback-classic</artifactId>
        <version>1.5.6</version>
    </dependency>
</dependencies>

</project>

```

- dom4j: 2.1.4
- jaxen: 2.0.0
- mysql-connector-j: 8.4.0
- junit: 4.13.2
- logback-classic: 1.5.6

为什么要引入 jaxen ?

在 LztBatis 中，我们要解析 lztbatis-config.xml。如果没有 jaxen，使用纯 dom4j 查找数据库的 driver 节点，需要一层一层的按照标签顺序往下找，这非常繁琐。

有了 jaxen 提供 XPath 支持后，就相当于给 XML 加上了“文件路径查询”功能。我们只需要写一行表达式：

```

//property[@name='driver']

```

就能直接穿透层级，精准抓取到对应的节点

- 第二步：编写配置文件 lztbatis-config.xml
 - 这个文件的核心作用就是告诉 LztBatis：去哪里连数据库，账号密码是什么，以及去哪里找 SQL 映射文件。

```

<?xml version="1.0" encoding="UTF-8" ?>
<configuration>
    <environments default="development">
        <environment id="development">
            <transactionManager type="JDBC"/>

            <dataSource type="POOLED">
                <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
                <property name="url" value="jdbc:mysql://localhost:3306/mybatis_demo?
useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai"/>
                <property name="username" value="root"/>
                <property name="password" value="20060329"/>
            </dataSource>
        </environment>
    </environments>

    <mappers>
        <mapper resource="CarMapper.xml"/>
    </mappers>

</configuration>

```

- 第三步：解析 lztbatis-config.xml
 - 编写 ParseXMLByDom4j.testLztBatisConfig

```

package com.lzt.test;

import org.dom4j.Document;

```

```

import org.dom4j.Element;
import org.dom4j.Node;
import org.dom4j.io.SAXReader;
import org.junit.Test;

import java.io.InputStream;
import java.util.List;

public class ParseXMLByDom4j {

    @Test
    public void testLztBatisConfig() throws Exception {
        // 1. 获取核心配置文件的输入流 (模拟 SqlSessionFactoryBuilder 接收 InputStream)
        // 使用类加载器从类路径 (classpath) 下读取 lztbatis-config.xml
        InputStream is = ClassLoader.getSystemClassLoader().getResourceAsStream("lztbatis-config.xml");

        // 2. 创建 dom4j 的 SAXReader 对象, 用于读取 XML
        SAXReader reader = new SAXReader();

        // 3. 读取输入流, 生成 Document 文档对象 (这代表了整个 XML 文件的树形结构)
        Document document = reader.read(is);

        // 4. 解析数据库连接信息: 利用 XPath 表达式 //property 极速定位所有 property 标签
        System.out.println("==== LztBatis 正在加载数据库连接池配置 =====");
        List<Node> propertyNodes = document.selectNodes("//property");
        for (Node node : propertyNodes) {
            Element element = (Element) node;
            // 获取 name 和 value 属性的值
            String name = element.attributeValue("name");
            String value = element.attributeValue("value");
            System.out.println("解析到配置: " + name + " = " + value);

            // 在真正的框架中, 这里会将抓取到的 driver, url, username(root), password(20060329)
            // 存入一个自定义的 Configuration 对象, 或者直接用来创建 DataSource 数据库连接池。
        }

        System.out.println("\n==== LztBatis 正在加载 SQL 映射文件 =====");
        // 5. 解析 Mapper 映射文件位置: 利用 XPath 表达式 //mapper 定位
        List<Node> mapperNodes = document.selectNodes("//mapper");
        for (Node node : mapperNodes) {
            Element element = (Element) node;
            String resource = element.attributeValue("resource");
            System.out.println("发现 Mapper 映射文件: " + resource);

            // 在真正的框架中, 读取到 CarMapper.xml 后, 会立刻触发对 CarMapper.xml 的解析,
            // 把里面的 SQL 语句提取出来存入内存 (通常是一个 Map<String, MappedStatement>)。
        }
    }
}

```

✓ 测试 已通过: 1共 1 个测试 - 88毫秒

```

D:\Java21\bin\java.exe ...
==== LztBatis 正在加载数据库连接池配置 =====
解析到配置: driver = com.mysql.cj.jdbc.Driver
解析到配置: url = jdbc:mysql://localhost:3306/mybatis_demo?useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai
解析到配置: username = root
解析到配置: password = 20060329

==== LztBatis 正在加载 SQL 映射文件 =====
发现 Mapper 映射文件: CarMapper.xml

```

- 第四步: 编写配置文件CarMapper.xml
 - 在真正的 MyBatis 中, 这个文件用来将 Java 方法和 SQL 语句绑定在一起。

- 对于我们手写的 LztBatis 来说，它也是用来告诉底层框架：“当我调用某个方法时，你应该去执行哪条 SQL，并且把结果封装成什么类型的 Java 对象返回给我。”

```
<?xml version="1.0" encoding="UTF-8" ?>
<mapper namespace="com.Lzt.mapper.CarMapper">

    <select id="selectCarById" resultType="com.Lzt.pojo.Car">
        select id, car_num, brand, guide_price, produce_time, car_type
        from t_car
        where id = #{id}
    </select>

    <insert id="insertCar">
        insert into t_car(id, car_num, brand, guide_price, produce_time, car_type)
        values (#{id}, #{carNum}, #{brand}, #{guidePrice}, #{produceTime}, #{carType})
    </insert>

</mapper>
```

- 第五步：解析CarMapper.xml
 - 编写：ParseXMLByDom4j.testCarMapper
 - 在真正的 MyBatis 底层，提取出来的这些信息会被封装成一个个 MappedStatement 对象，并以 namespace.id 作为 Key，存入一个巨大的全局 Map 集合中备用。

```
@Test
public void testCarMapper() throws Exception {
    // 1. 获取 Mapper 映射文件的输入流
    InputStream is = ClassLoader.getSystemClassLoader().getResourceAsStream("CarMapper.xml");

    // 2. 创建 SAXReader 对象，读取 XML 文档
    SAXReader reader = new SAXReader();
    Document document = reader.read(is);

    // 3. 解析根标签 <mapper> 获取 namespace (命名空间)
    // 使用 selectSingleNode 直接获取唯一的 mapper 节点
    Element mapperElement = (Element) document.selectSingleNode("//mapper");
    String namespace = mapperElement.attributeValue("namespace");

    System.out.println("==== LztBatis 正在解析 Mapper 命名空间 =====");
    System.out.println("发现 Namespace: " + namespace);

    // 4. 解析所有的 <select> 查询标签
    System.out.println("\n===== LztBatis 正在解析 <select> 标签 =====");
    List<Node> selectNodes = document.selectNodes("//select");
    for (Node node : selectNodes) {
        Element element = (Element) node;

        // 提取核心属性
        String id = element.attributeValue("id");
        String resultType = element.attributeValue("resultType");

        // 提取 SQL 语句，getTextTrim() 可以自动去除 SQL 文本首尾的换行符和空格
        String sql = element.getTextTrim();

        // 框架底层动作：将 namespace 和 id 拼接，作为这条 SQL 的全局唯一坐标
        String statementId = namespace + "." + id;

        System.out.println("全局唯一标识 (Statement ID): " + statementId);
        System.out.println("返回结果类型 (Result Type): " + resultType);
        System.out.println("原始 SQL 语句: \n" + sql);
        System.out.println("-----");
    }
}
```

```

    // 在手写框架的后续步骤中，我们会把这三个变量存入 MappedStatement 对象中。
}

// 5. 解析所有的 <insert> 插入标签
System.out.println("\n===== LztBatis 正在解析 <insert> 标签 =====");
List<Node> insertNodes = document.selectNodes("//insert");
for (Node node : insertNodes) {
    Element element = (Element) node;

    String id = element.attributeValue("id");
    String sql = element.getTextTrim();

    String statementId = namespace + "." + id;

    System.out.println("全局唯一标识 (Statement ID): " + statementId);
    System.out.println("原始 SQL 语句: \n" + sql);
    System.out.println("-----");
}
}
}

```

```

D:\Java21\bin\java.exe ...
===== LztBatis 正在解析 Mapper 命名空间 =====
发现 Namespace: com.lzt.mapper.CarMapper

===== LztBatis 正在解析 <select> 标签 =====
全局唯一标识 (Statement ID): com.lzt.mapper.CarMapper.selectCarById
返回结果类型 (Result Type): com.lzt.pojo.Car
原始 SQL 语句:
select id, car_num, brand, guide_price, produce_time, car_type from t_car where id = #{id}
-----

===== LztBatis 正在解析 <insert> 标签 =====
全局唯一标识 (Statement ID): com.lzt.mapper.CarMapper.insertCar
原始 SQL 语句:
insert into t_car(id, car_num, brand, guide_price, produce_time, car_type) values (#{id}, #{carNum}, #{brand}, #{guidePrice}, #{produceTime}, #{carType})
-----

```

LztBatis

手写框架之前，如果没有思路，可以先参考一下mybatis的客户端程序，通过客户端程序来逆推需要的类，参考代码：

```

@Test
public void testInsert(){
    SqlSession sqlSession = null;
    try {
        // 1. 创建SqlSessionFactoryBuilder对象
        SqlSessionFactoryBuilder sqlSessionFactoryBuilder = new SqlSessionFactoryBuilder();
        // 2. 创建SqlSessionFactory对象
        SqlSessionFactory sqlSessionFactory = sqlSessionFactoryBuilder.build(Resources.getResourceAsStream("mybatis-config.xml"));
        // 3. 创建SqlSession对象
        sqlSession = sqlSessionFactory.openSession();
        // 4. 执行SQL
        Car car = new Car(null, "111", "宝马X7", "70.3", "2010-10-11", "燃油车");
        int count = sqlSession.insert("insertCar", car);
        System.out.println("更新了几条记录: " + count);
        // 5. 提交
        sqlSession.commit();
    } catch (Exception e) {
        // 回滚
        if (sqlSession != null) {
            sqlSession.rollback();
        }
    }
}

```

```

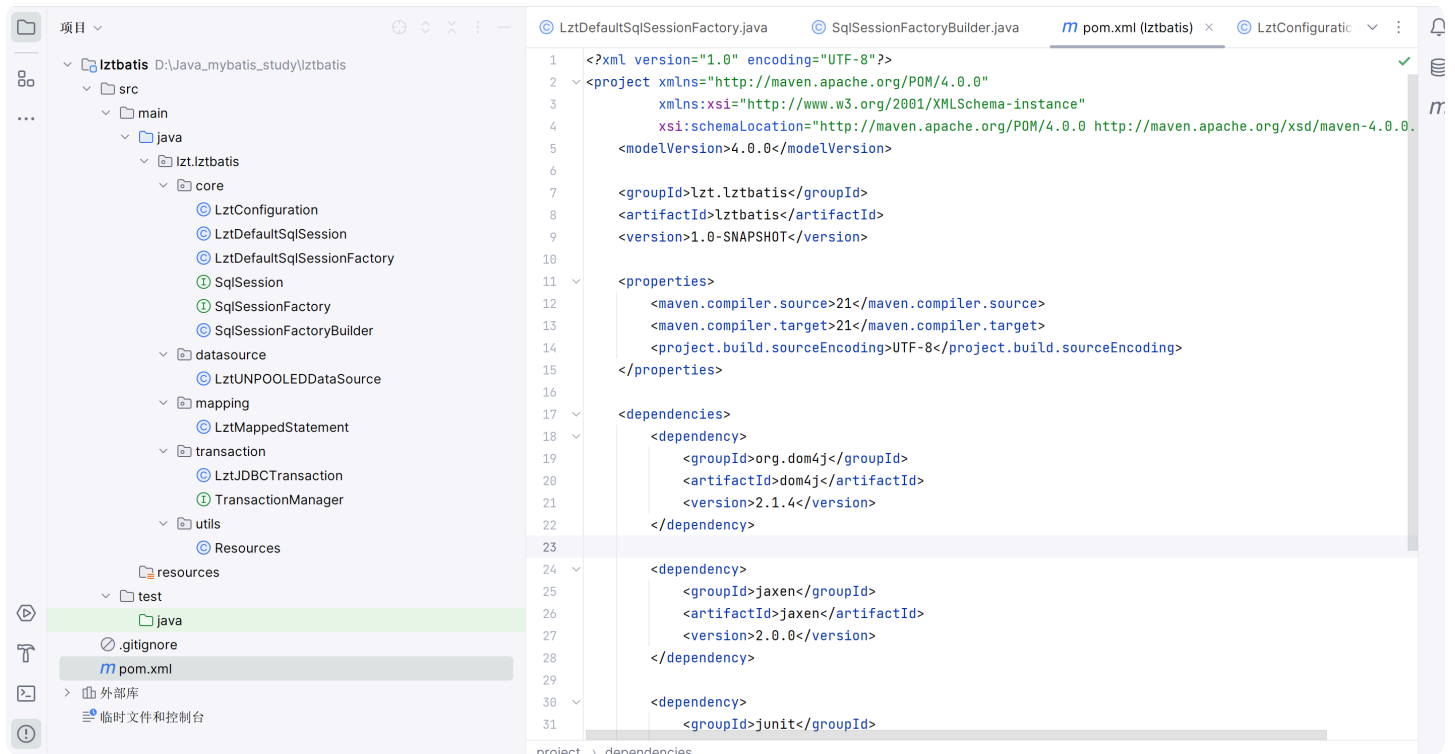
        e.printStackTrace();
    } finally {
        // 6. 关闭
        if (sqlSession != null) {
            sqlSession.close();
        }
    }
}

@Test
public void testSelectOne(){
    SqlSession sqlSession = null;
    try {
        // 1. 创建SqlSessionFactoryBuilder对象
        SqlSessionFactoryBuilder sqlSessionFactoryBuilder = new SqlSessionFactoryBuilder();
        // 2. 创建SqlSessionFactory对象
        SqlSessionFactory sqlSessionFactory = sqlSessionFactoryBuilder.build(Resources.getResourceAsStream("mybatis-
config.xml"));
        // 3. 创建SqlSession对象
        sqlSession = sqlSessionFactory.openSession();
        // 4. 执行SQL
        Car car = (Car)sqlSession.selectOne("selectCarByCarNum", "111");
        System.out.println(car);
        // 5. 提交
        sqlSession.commit();
    } catch (Exception e) {
        // 回滚
        if (sqlSession != null) {
            sqlSession.rollback();
        }
        e.printStackTrace();
    } finally {
        // 6. 关闭
        if (sqlSession != null) {
            sqlSession.close();
        }
    }
}

```

在 testInsert 和 testSelectOne 方法中，LztBatis 实现以下几个核心组件：

- Resources：源加载工具类
- SqlSessionFactoryBuilder：工厂构建器
- SqlSessionFactory：SqlSession 工厂
- SqlSession：核心会话对象
- Car：POJO 实体类



第一步：IDEA中创建模块

模块：lztbatis（创建普通的Java Maven模块，打包方式jar），引入相关依赖



- 配置 pom.xml
 - 核心的职责是解析 XML 配置和提供核心 API。

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>lzt.lztbatis</groupId>
  <artifactId>lztbatis</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
```

```

<dependencies>
  <dependency>
    <groupId>org.dom4j</groupId>
    <artifactId>dom4j</artifactId>
    <version>2.1.4</version>
  </dependency>

  <dependency>
    <groupId>jaxen</groupId>
    <artifactId>jaxen</artifactId>
    <version>2.0.0</version>
  </dependency>

  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>

</project>

```

第二步：编写 Resources 工具类

```

package lzt.lztbatis.utils;

import java.io.InputStream;

/**
 * LztBatis 框架提供的工具类
 * 专门用于从类路径中加载资源文件
 */
public class Resources {

    /**
     * 工具类的构造方法建议私有化，防止在外部被 new 出来
     */
    private Resources() {}

    /**
     * 从类路径中加载资源，返回一个输入流
     * @param resource 资源名称（例如："mybatis-config.xml"）
     * @return InputStream 输入流
     */
    public static InputStream getResourceAsStream(String resource) {
        // 获取当前线程的类加载器，去类路径下查找资源并返回输入流
        return Thread.currentThread().getContextClassLoader().getResourceAsStream(resource);

        // 另一种写法也可以：
        // return ClassLoader.getSystemClassLoader().getResourceAsStream(resource);
    }
}

```

第三步：定义SqlSessionFactoryBuilder类

提供一个无参数构造方法，再提供一个build方法，该build方法要返回SqlSessionFactory对象

```

package lzt.lztbatis.core;

```



```
import java.io.InputStream;

/**
 * LztBatis 框架的核心构建器类
 * 专门用于解析 XML 配置文件，并构建 SqlSessionFactory 对象
 */
public class SqlSessionFactoryBuilder {

    /**
     * 创建构建器对象
     */
    public SqlSessionFactoryBuilder() {

    }

    /**
     * 获取SqlSessionFactory对象
     * 该方法主要功能是：读取godbatis核心配置文件，并构建SqlSessionFactory对象
     * @param inputStream 指向核心配置文件的输入流
     * @return SqlSessionFactory对象
     */
    public SqlSessionFactory build(InputStream inputStream){
        // 解析配置文件，创建数据源对象
        // 解析配置文件，创建事务管理器对象
        // 解析配置文件，获取所有的SQL映射对象
        // 将以上信息封装到SqlSessionFactory对象中
        // 返回
        return null;
    }
}
```



SqlSessionFactoryBuilder.java D:\Java_mybatis_study\lztbatis\src\main\java\lzt\lztbatis\core 6 个问题
 无法解析符号 'SqlSessionFactory' :25

- 为什么目前代码会报错或返回 null？
 - 因为在 build 方法的返回值中，我们引用了 SqlSessionFactory，但这个类目前在你的项目中还不存在。手写框架就是这样一个从无到有、互相依赖被逐渐满足的过程。
 - 之后慢慢填充，解决报红

第四步：分析SqlSessionFactory类中有哪些属性

在真实的 MyBatis 源码和标准的手写教程中，SqlSessionFactory 通常被设计为一个接口，而真正包含属性的是它的实现类

为了搞清楚它需要哪些属性，我们要回想一下它在框架中的职责：它是用来生产 SqlSession 的工厂。

而 SqlSession 要负责去数据库里执行具体的 SQL 语句。这就意味着，生产出来的 SqlSession 必须知道连哪个数据库以及执行什么 SQL。

因此，在第三步中 **SqlSessionFactoryBuilder** 解析完 XML 文件后，必须要把解析出来的数据交给 **SqlSessionFactory** 存起来。

- SqlSessionFactory 有两种属性
 - 事务管理器：对应 MyBatis 原生的 Transaction 接口实现，负责管理数据库连接的事务生命周期：包括开启连接、提交事务、回滚事务、关闭连接等操作。
 - 编写 LztJDBCTransaction
 - SQL 映射对象集合：对应 MyBatis 原生的 MappedStatement 存储结构，用于缓存所有解析后的 SQL 映射元数据：
 - Map<String, LztMappedStatement>
 - key：通常是 namespace.id（如 com.example.CarMapper.selectById），用于快速定位 SQL；

- value (LztMappedStatement)：存储单条 SQL 的完整信息，包括 SQL 语句、参数类型、结果映射规则、执行类型（SELECT/INSERT/UPDATE/DELETE）等。

接下来要依次完成 SqlSessionFactory 接口和它的实现类，Configuration：核心全局配置对象，MappedStatement：SQL 映射对象集合。

第五步：定义LztJDBCTransaction

事务管理器最好是定义一个接口，然后每一个具体的事务管理器都实现这个接口。

在框架中，数据库的连接（Connection）、提交（Commit）、回滚（Rollback）和关闭（Close）是非常核心的动作。

- 创建 TransactionManager 接口

```
package lzt.lztbatis.transaction;

import java.sql.Connection;

/**
 * 事务管理器接口
 * 负责规范数据库事务的核心行为
 */
public interface TransactionManager {

    /**
     * 获取数据库连接
     */
    Connection getConnection();

    /**
     * 提交事务
     */
    void commit();

    /**
     * 回滚事务
     */
    void rollback();

    /**
     * 关闭连接，释放资源
     */
    void close();
}
```

- 创建 LztJDBCTransaction 实现类
 - 这是接口的具体实现，它底层依赖于 javax.sql.DataSource（数据源/连接池）和原生的 java.sql.Connection 来完成实际的数据库操作。

```
package lzt.lztbatis.transaction;

import javax.sql.DataSource;
import java.sql.Connection;
import java.sql.SQLException;

/**
 * JDBC 事务实现类
 * 依靠底层的数据源来获取连接，并使用原生的 Connection 方法管理事务
 */
public class LztJDBCTransaction implements TransactionManager {

    // 数据源（提供数据库连接的工厂，比如德鲁伊、HikariCP，或者自定义的数据源）
}
```

```

private DataSource dataSource;
// 是否自动提交事务
private boolean autoCommit;
// 内部维护的 JDBC 数据库连接对象
private Connection connection;

/**
 * 构造方法
 * @param dataSource 数据源
 * @param autoCommit 是否自动提交
 */
public LztJDBCTransaction(DataSource dataSource, boolean autoCommit) {
    this.dataSource = dataSource;
    this.autoCommit = autoCommit;
}

@Override
public Connection getConnection() {
    // 【核心细节：懒加载机制】
    // 只有当程序真正需要执行 SQL 语句，调用 getConnection() 时，才去数据源拿连接。
    // 如果只是创建了 SqlSession 但没执行 SQL，就不占用宝贵的数据库连接资源。
    if (this.connection == null) {
        try {
            this.connection = dataSource.getConnection();
            // 设置底层连接的自动提交属性
            this.connection.setAutoCommit(this.autoCommit);
        } catch (SQLException e) {
            // 实际框架中这里会抛出自定义的异常，这里为了简化暂时打印堆栈
            e.printStackTrace();
        }
    }
    return this.connection;
}

@Override
public void commit() {
    // 只有当连接存在，且不是自动提交模式时，才需要手动提交
    if (this.connection != null && !this.autoCommit) {
        try {
            this.connection.commit();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

@Override
public void rollback() {
    // 只有当连接存在，且不是自动提交模式时，才需要手动回滚
    if (this.connection != null && !this.autoCommit) {
        try {
            this.connection.rollback();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

@Override
public void close() {
    // 归还或关闭连接
    if (this.connection != null) {
        try {
            this.connection.close();
        } catch (SQLException e) {

```

```

        e.printStackTrace();
    }
}
}
}
}

```

第六步：事务管理器中需要数据源，定义LztUNPOOLEDDataSource

“UNPOOLED”：每次调用 getConnection() 时，都老老实实在地去数据库新建一个真实的物理连接；关闭时也直接销毁连接，不进行缓存和复用。



```

package lzt.lztbatis.datasource;

import javax.sql.DataSource;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.SQLFeatureNotSupportedException;
import java.util.logging.Logger;

/**
 * LztBatis 提供的不使用连接池的数据源实现
 * 每次获取连接都会新建一个真实的物理连接
 */
public class LztUNPOOLEDDataSource implements DataSource {

    // 数据库连接的四大核心参数
    private String driver;
    private String url;
    private String username;
    private String password;

    /**
     * 构造方法：在创建数据源对象时，传入这四个参数并注册驱动
     */
    public LztUNPOOLEDDataSource(String driver, String url, String username, String password) {
        this.driver = driver;
        this.url = url;
        this.username = username;
        this.password = password;

        // 注册数据库驱动
        try {
            Class.forName(driver);
        } catch (ClassNotFoundException e) {
            throw new RuntimeException("数据库驱动加载失败", e);
        }
    }

    /**
     * 核心方法：获取数据库连接
     */
    @Override
    public Connection getConnection() throws SQLException {
        // UNPOOLED 的精髓：每次都通过 DriverManager 建立全新的连接
        return DriverManager.getConnection(url, username, password);
    }

    @Override
    public Connection getConnection(String username, String password) throws SQLException {
        return DriverManager.getConnection(url, username, password);
    }
}

```

```
// =====
// 下面这些方法都是 DataSource 接口强制要求实现的，但对于我们基础版的手写框架来说
// 暂时用不到，所以可以直接返回 null 或者抛出不支持该操作的异常。
// =====

@Override
public <T> T unwrap(Class<T> iface) throws SQLException {
    throw new SQLException("不支持的操作");
}

@Override
public boolean isWrapperFor(Class<?> iface) throws SQLException {
    return false;
}

@Override
public PrintWriter getLogWriter() throws SQLException {
    return null;
}

@Override
public void setLogWriter(PrintWriter out) throws SQLException {
}

@Override
public void setLoginTimeout(int seconds) throws SQLException {
}

@Override
public int getLoginTimeout() throws SQLException {
    return 0;
}

@Override
public Logger getParentLogger() throws SQLFeatureNotSupportedException {
    throw new SQLFeatureNotSupportedException();
}
}
```

第七步：定义LztMappedStatement

```
package lzt.lztbatis.mapping;

/**
 * 核心映射语句类
 * 专门用于封装 Mapper XML 文件中每一个 SQL 标签的详细信息
 */
public class LztMappedStatement {

    /**
     * SQL 语句的唯一标识
     * 对应 XML 标签的 id 属性，或者是 namespace + id（例如："insertCar" 或 "car.insertCar"）
     */
    private String sqlId;

    /**
     * 返回值类型的全限定类名
     * 对应 XML 中的 resultType 属性（例如："com.Lzt.pojo.Car"）
     * 注意：insert/update/delete 通常不需要专门配置 resultType，但 select 查询通常必须有
     */
    private String resultType;
}
```

```

/**
 * 真正要交给 JDBC 执行的 SQL 语句文本
 * （在完善的框架中，这里会将 XML 里的 #{xxx} 预先转换成 ? 的预编译 SQL）
 */
private String sql;

// --- 以下为标准的无参/有参构造方法，以及 Getter/Setter 方法 ---

public LztMappedStatement() {
}

public LztMappedStatement(String sqlId, String resultType, String sql) {
    this.sqlId = sqlId;
    this.resultType = resultType;
    this.sql = sql;
}

public String getSqlId() {
    return sqlId;
}

public void setSqlId(String sqlId) {
    this.sqlId = sqlId;
}

public String getResultType() {
    return resultType;
}

public void setResultType(String resultType) {
    this.resultType = resultType;
}

public String getSql() {
    return sql;
}

public void setSql(String sql) {
    this.sql = sql;
}

@Override
public String toString() {
    return "LztMappedStatement{" +
        "sqlId='" + sqlId + '\'' +
        ", resultType='" + resultType + '\'' +
        ", sql='" + sql + '\'' +
        '}';
}
}

```

第八步：完善SqlSessionFactory类

在标准的设计中，为了遵循面向接口编程的原则，我们通常会定义一个 SqlSessionFactory 接口，并提供一个默认的实现类 LztDefaultSqlSessionFactory。

- 定义 SqlSessionFactory 接口

```

● ● ●
package lzt.lztbatis.core;

/**
 * SqlSession 工厂接口

```

```

* 核心职责：生产与数据库交互的 SqlSession 对象
*/
public interface SqlSessionFactory {

    /**
     * 开启并获取一个数据库会话
     * @return SqlSession 对象
     */
    SqlSession openSession();
}

```

- 创建 LztDefaultSqlSessionFactory 实现类

```

package lzt.lztbatis.core;

import lzt.lztbatis.mapping.LztMappedStatement;
import lzt.lztbatis.transaction.TransactionManager;
import java.util.Map;

/**
 * 默认的 SqlSessionFactory 实现类
 */
public class LztDefaultSqlSessionFactory implements SqlSessionFactory {

    /**
     * 事务管理器
     * 里面包含了数据源，负责获取数据库连接以及控制事务提交/回滚
     */
    private TransactionManager transactionManager;

    /**
     * 存放所有的 SQL 语句映射信息
     * Key: SQL 标签的唯一标识 (例如 "insertCar")
     * Value: 封装了具体 SQL 语句和返回类型的 LztMappedStatement 对象
     */
    private Map<String, LztMappedStatement> mappedStatements;

    /**
     * 构造方法：在工厂被实例化时，注入事务管理器和 SQL 映射集合
     */
    public LztDefaultSqlSessionFactory(TransactionManager transactionManager, Map<String, LztMappedStatement> mappedStatements) {
        this.transactionManager = transactionManager;
        this.mappedStatements = mappedStatements;
    }

    @Override
    public SqlSession openSession() {
        // 【核心逻辑解析】
        // 当用户调用 factory.openSession() 时，实际上是去 new 一个具体的 SqlSession 对象。
        // 为了让 SqlSession 能够去数据库执行具体的 SQL，必须把 factory 里的 "连接池/事务" 和 "SQL 字典" 交给它。

        // TODO: 返回一个真正的 SqlSession 实现类实例
        // 比如: return new LztDefaultSqlSession(this.transactionManager, this.mappedStatements);

        return null; // 因为我们还没写 SqlSession，所以这里暂时返回 null，下一步我们就会把它填上！
    }

    // --- Getter 和 Setter 方法 ---

    public TransactionManager getTransactionManager() {
        return transactionManager;
    }
}

```

```

    public void setTransactionManager(TransactionManager transactionManager) {
        this.transactionManager = transactionManager;
    }

    public Map<String, LztMappedStatement> getMappedStatements() {
        return mappedStatements;
    }

    public void setMappedStatements(Map<String, LztMappedStatement> mappedStatements) {
        this.mappedStatements = mappedStatements;
    }
}

```

第九步：完善 Configuration 类

```

package lzt.lztbatis.core;

import lzt.lztbatis.mapping.LztMappedStatement;
import lzt.lztbatis.transaction.TransactionManager;

import java.util.Map;

/**
 * LztBatis 框架的核心配置类（大管家）
 * 专门用于在内存中存放整个框架运行期间需要的所有配置信息
 */
public class LztConfiguration {

    /**
     * 事务管理器（内部已经封装了 DataSource 数据源）
     * 对应 mybatis-config.xml 中的 <environments> 标签解析结果
     */
    private TransactionManager transactionManager;

    /**
     * 所有的 SQL 映射语句集合
     * Key: SQL 语句的 id（例如 "insertCar"）
     * Value: 具体的映射语句对象（包含了 sql 和 resultType）
     * 对应所有的 Mapper XML 文件的解析结果
     */
    private Map<String, LztMappedStatement> mappedStatements;

    /**
     * 构造方法
     */
    public LztConfiguration() {
    }

    public LztConfiguration(TransactionManager transactionManager, Map<String, LztMappedStatement> mappedStatements) {
        this.transactionManager = transactionManager;
        this.mappedStatements = mappedStatements;
    }

    // --- Getter 和 Setter 方法 ---

    public TransactionManager getTransactionManager() {
        return transactionManager;
    }

    public void setTransactionManager(TransactionManager transactionManager) {
        this.transactionManager = transactionManager;
    }
}

```



```

    public Map<String, LztMappedStatement> getMappedStatements() {
        return mappedStatements;
    }

    public void setMappedStatements(Map<String, LztMappedStatement> mappedStatements) {
        this.mappedStatements = mappedStatements;
    }
}

```

第十步：完善SqlSessionFactoryBuilder中的build方法

```

package lzt.lztbatis.core;

import lzt.lztbatis.datasource.LztUNPOOLEDDataSource;
import lzt.lztbatis.mapping.LztMappedStatement;
import lzt.lztbatis.transaction.LztJDBCTransaction;
import lzt.lztbatis.transaction.TransactionManager;
import lzt.lztbatis.utils.Resources;
import org.dom4j.Document;
import org.dom4j.Element;
import org.dom4j.Node;
import org.dom4j.io.SAXReader;

import javax.sql.DataSource;
import java.io.InputStream;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * LztBatis 框架的核心构建器类
 * 专门用于解析 XML 配置文件，并构建 SqlSessionFactory 对象
 */
public class SqlSessionFactoryBuilder {

    public SqlSessionFactoryBuilder() {
    }

    /**
     * 核心 build 方法：通过读取输入流来构建会话工厂
     * @param inputStream 指向核心配置文件的输入流 (mybatis-config.xml)
     * @return SqlSessionFactory 工厂对象
     */
    public SqlSessionFactory build(InputStream inputStream) {
        try {
            // 1. 创建 dom4j 的解析器
            SAXReader reader = new SAXReader();
            Document document = reader.read(inputStream);
            Element root = document.getRootElement();

            // ===== 第一部分：解析数据库环境配置 =====

            // 获取默认的环境 id (对应 <environments default="development">)
            String defaultEnvId = root.element("environments").attributeValue("default");

            // 使用 XPath 语法定位到默认的 environment 标签
            String xpath = "/configuration/environments/environment[@id='" + defaultEnvId + "']";
            Element environment = (Element) root.selectSingleNode(xpath);

            // 1.1 解析 DataSource (数据源)
            Element dataSourceElt = environment.element("dataSource");
            String driver = "", url = "", username = "", password = "";

```

```

List<Element> properties = dataSourceElt.elements("property");
for (Element property : properties) {
    String name = property.attributeValue("name");
    String value = property.attributeValue("value");
    if ("driver".equals(name)) driver = value;
    if ("url".equals(name)) url = value;
    if ("username".equals(name)) username = value;
    if ("password".equals(name)) password = value;
}
// 实例化我们在第六步写好的 UNPOOLED 数据源
DataSource dataSource = new LztUNPOOLEDDataSource(driver, url, username, password);

// 1.2 解析 TransactionManager (事务管理器)
Element txManagerElt = environment.element("transactionManager");
String txType = txManagerElt.attributeValue("type");
TransactionManager transactionManager = null;
if ("JDBC".equals(txType.toUpperCase())) {
    // 实例化我们在第五步写好的 JDBC 事务管理器, 默认关闭自动提交
    transactionManager = new LztJDBCTransaction(dataSource, false);
}

// ===== 第二部分: 解析所有的 Mapper SQL 映射 =====

Map<String, LztMappedStatement> mappedStatements = new HashMap<>();

// 获取所有的 <mapper resource="..."> 标签
List<Node> mapperNodes = root.selectNodes("//mapper");
for (Node mapperNode : mapperNodes) {
    Element mapperElt = (Element) mapperNode;
    String resource = mapperElt.attributeValue("resource");

    // 读取具体的 Mapper XML 文件 (例如 CarMapper.xml)
    InputStream mapperInputStream = Resources.getResourceAsStream(resource);
    Document mapperDoc = reader.read(mapperInputStream);
    Element mapperRoot = mapperDoc.getRootElement();

    // 为了简化, 这里我们假设直接使用 namespace + id 作为 key, 或者像你的测试代码里那样直接用 id。
    // 真正的 MyBatis 是用 namespace.id。为了匹配你的测试类 sqlSession.insert("insertCar", car);
    // 我们这里以 SQL 标签的 id 作为 Map 的 key。

    // 遍历 <select>, <insert>, <update>, <delete> 标签
    List<Element> sqlElements = mapperRoot.elements();
    for (Element sqlElement : sqlElements) {
        String id = sqlElement.attributeValue("id");
        String resultType = sqlElement.attributeValue("resultType");
        // 获取标签内部的文本 (即具体的 SQL 语句)
        String sql = sqlElement.getTextTrim();

        // 实例化我们在第七步写好的 LztMappedStatement, 把 SQL 信息存进去
        LztMappedStatement mappedStatement = new LztMappedStatement(id, resultType, sql);

        // 放入 Map 集合中
        mappedStatements.put(id, mappedStatement);
    }
}

// ===== 第三部分: 组装并返回 SqlSessionFactory =====

// 实例化第八步写好的默认工厂, 并将解析好的“事务管理器”和“SQL字典”交由它保管
return new LztDefaultSqlSessionFactory(transactionManager, mappedStatements);
} catch (Exception e) {
    e.printStackTrace();
    throw new RuntimeException("解析 XML 配置文件失败, 无法构建 SqlSessionFactory!", e);
}

```

```
    }  
}  
}
```

第十一步：编写SqlSession类中commit rollback close方法

在 MyBatis 的架构中，SqlSession 是用户层面的核心接口，我们在客户端代码中直接打交道的也就是它。

但实际上，SqlSession 本身是个“甩手掌柜”，它并不直接去操作底层的 JDBC 事务，而是把这些任务委托给了我们在第五步写好的 TransactionManager。这种设计模式叫做委派模式。

- 定义 SqlSession 接口
 - 为了承上启下，我们把测试类中用到的 insert 和 selectOne 也先在接口里声明出来（具体实现我们放在后面的步骤）。

```
package lzt.lztbatis.core;  
  
/**  
 * 核心会话接口  
 * 提供了执行 SQL、提交、回滚和关闭会话的方法  
 */  
public interface SqlSession {  
  
    /**  
     * 执行插入操作  
     * @param sqlId SQL 语句的唯一标识  
     * @param pojo 要插入的实体类对象  
     * @return 影响的行数  
     */  
    int insert(String sqlId, Object pojo);  
  
    /**  
     * 执行查询单条记录的操作  
     * @param sqlId SQL 语句的唯一标识  
     * @param parameter 查询参数  
     * @return 查询到的结果对象  
     */  
    Object selectOne(String sqlId, Object parameter);  
  
    /**  
     * 提交事务  
     */  
    void commit();  
  
    /**  
     * 回滚事务  
     */  
    void rollback();  
  
    /**  
     * 关闭会话（释放连接）  
     */  
    void close();  
}
```

- 编写 LztDefaultSqlSession 实现类
 - 创建 LztDefaultSqlSession 类并实现刚才的接口。在这里，我们将利用上一步定义的 LztConfiguration 来获取事务管理器。

```
package lzt.lztbatis.core;  
  
import lzt.lztbatis.transaction.TransactionManager;
```

```

/**
 * 默认的 SqlSession 实现类
 */
public class LztDefaultSqlSession implements SqlSession {

    /**
     * 依赖核心配置大管家
     * 里面包含了事务管理器和 SQL 映射字典
     */
    private LztConfiguration configuration;

    /**
     * 构造方法: 当工厂 openSession() 时, 会把 configuration 传给它
     */
    public LztDefaultSqlSession(LztConfiguration configuration) {
        this.configuration = configuration;
    }

    @Override
    public void commit() {
        // 从大管家中获取事务管理器, 并委托它去提交事务
        TransactionManager transactionManager = this.configuration.getTransactionManager();
        if (transactionManager != null) {
            transactionManager.commit();
        }
    }

    @Override
    public void rollback() {
        // 从大管家中获取事务管理器, 并委托它去回滚事务
        TransactionManager transactionManager = this.configuration.getTransactionManager();
        if (transactionManager != null) {
            transactionManager.rollback();
        }
    }

    @Override
    public void close() {
        // 从大管家中获取事务管理器, 并委托它去关闭 (归还) 连接
        TransactionManager transactionManager = this.configuration.getTransactionManager();
        if (transactionManager != null) {
            transactionManager.close();
        }
    }

    // =====
    // 下面这两个是真正去数据库执行 SQL 的方法,
    // 我们先空着, 这通常是下一步的核心内容!
    // =====

    @Override
    public int insert(String sqlId, Object pojo) {
        // TODO: 执行具体的 INSERT 逻辑
        return 0;
    }

    @Override
    public Object selectOne(String sqlId, Object parameter) {
        // TODO: 执行具体的 SELECT 逻辑
        return null;
    }
}

```

第十二步：在SqlSessionFactory中添加openSession方法

- 关联回顾：工厂如何生产这个 Session？

回顾一下我们在第八步写的 LztDefaultSqlSessionFactory 的 openSession() 方法，现在有了 LztDefaultSqlSession 和 LztConfiguration，工厂类的完整形态就呼之欲出了：

- 请直接用这段代码替换你目前的 LztDefaultSqlSessionFactory.java：

```
package lzt.lztbatis.core;

/**
 * 默认的 SqlSessionFactory 实现类
 */
public class LztDefaultSqlSessionFactory implements SqlSessionFactory {

    /**
     * 核心配置大管家
     * 里面已经封装好了 TransactionManager 和 mappedStatements
     */
    private LztConfiguration configuration;

    /**
     * 构造方法：在工厂被实例化时，直接注入配置大管家
     */
    public LztDefaultSqlSessionFactory(LztConfiguration configuration) {
        this.configuration = configuration;
    }

    @Override
    public SqlSession openSession() {
        // 【核心逻辑】
        // 当用户调用 factory.openSession() 时，实际上是去 new 一个具体的 SqlSession 对象。
        // 我们把大管家交给新创建的 Session，让它具备执行 SQL 和控制事务的能力。
        return new LztDefaultSqlSession(this.configuration);
    }

    // --- Getter 和 Setter 方法 ---

    public LztConfiguration getConfiguration() {
        return configuration;
    }

    public void setConfiguration(LztConfiguration configuration) {
        this.configuration = configuration;
    }
}
```

- 完整的LztDefaultSqlSessionFactory代码

```
package lzt.lztbatis.core;

import lzt.lztbatis.mapping.LztMappedStatement;
import lzt.lztbatis.transaction.TransactionManager;
import java.util.Map;

/**
 * 默认的 SqlSessionFactory 实现类
 */
public class LztDefaultSqlSessionFactory implements SqlSessionFactory {

    /**
```

```

    * 事务管理器
    * 里面包含了数据源，负责获取数据库连接以及控制事务提交/回滚
    */
    private TransactionManager transactionManager;

    /**
     * 存放所有的 SQL 语句映射信息
     * Key: SQL 标签的唯一标识 (例如 "insertCar")
     * Value: 封装了具体 SQL 语句和返回类型的 LztMappedStatement 对象
     */
    private Map<String, LztMappedStatement> mappedStatements;

    private LztConfiguration configuration;

    /**
     * 构造方法: 在工厂被实例化时，注入事务管理器和 SQL 映射集合
     */
    public LztDefaultSqlSessionFactory(TransactionManager transactionManager, Map<String, LztMappedStatement>
mappedStatements) {
        this.transactionManager = transactionManager;
        this.mappedStatements = mappedStatements;
    }

    /**
     * 构造方法: 在工厂被实例化时，直接注入配置大管家
     */
    public LztDefaultSqlSessionFactory(LztConfiguration configuration) {
        this.configuration = configuration;
    }

    @Override
    public SqlSession openSession() {
        // 【核心逻辑】
        // 当用户调用 factory.openSession() 时，实际上是去 new 一个具体的 SqlSession 对象。
        // 我们把大管家交给新创建的 Session，让它具备执行 SQL 和控制事务的能力。
        return new LztDefaultSqlSession(this.configuration);
    }

    // --- Getter 和 Setter 方法 ---

    public LztConfiguration getConfiguration() {
        return configuration;
    }

    public void setConfiguration(LztConfiguration configuration) {
        this.configuration = configuration;
    }

    public TransactionManager getTransactionManager() {
        return transactionManager;
    }

    public void setTransactionManager(TransactionManager transactionManager) {
        this.transactionManager = transactionManager;
    }

    public Map<String, LztMappedStatement> getMappedStatements() {
        return mappedStatements;
    }

    public void setMappedStatements(Map<String, LztMappedStatement> mappedStatements) {
        this.mappedStatements = mappedStatements;
    }
}

```

- 同步修改：SqlSessionFactoryBuilder
 - 因为 LztDefaultSqlSessionFactory 的构造方法变了（现在只接收一个 configuration 参数），你需要回过头去检查一下第九步写的 SqlSessionFactoryBuilder 类的最后几行代码。
 - 请确保 build 方法的结尾是这样写的：

```

// ===== 第三部分：组装并返回 SqlSessionFactory =====

// 1. 先把解析出来的数据塞进大管家
LztConfiguration configuration = new LztConfiguration(transactionManager, mappedStatements);

// 2. 实例化工厂，并把大管家交给它
return new LztDefaultSqlSessionFactory(configuration);

} catch (Exception e) {
    e.printStackTrace();
    throw new RuntimeException("解析 XML 配置文件失败，无法构建 SqlSessionFactory！", e);
}
}

```

- 完整的 SqlSessionFactoryBuilder 代码

```

package lzt.lztbatis.core;

import lzt.lztbatis.datasource.LztUNPOOLEDDataSource;
import lzt.lztbatis.mapping.LztMappedStatement;
import lzt.lztbatis.transaction.LztJDBCTransaction;
import lzt.lztbatis.transaction.TransactionManager;
import lzt.lztbatis.utils.Resources;
import org.dom4j.Document;
import org.dom4j.Element;
import org.dom4j.Node;
import org.dom4j.io.SAXReader;

import javax.sql.DataSource;
import java.io.InputStream;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * LztBatis 框架的核心构建器类
 * 专门用于解析 XML 配置文件，并构建 SqlSessionFactory 对象
 */
public class SqlSessionFactoryBuilder {

    public SqlSessionFactoryBuilder() {
    }

    /**
     * 核心 build 方法：通过读取输入流来构建会话工厂
     * @param inputStream 指向核心配置文件的输入流 (mybatis-config.xml)
     * @return SqlSessionFactory 工厂对象
     */
    public SqlSessionFactory build(InputStream inputStream) {
        try {
            // 1. 创建 dom4j 的解析器
            SAXReader reader = new SAXReader();
            Document document = reader.read(inputStream);
            Element root = document.getRootElement();

```

```
// ===== 第一部分：解析数据库环境配置 =====

// 获取默认的环境 id (对应 <environments default="development">)
String defaultEnvId = root.element("environments").attributeValue("default");

// 使用 XPath 语法定位到默认的环境 environment 标签
String xpath = "/configuration/environments/environment[@id='" + defaultEnvId + "']";
Element environment = (Element) root.selectSingleNode(xpath);

// 1.1 解析 DataSource (数据源)
Element dataSourceElt = environment.element("dataSource");
String driver = "", url = "", username = "", password = "";
List<Element> properties = dataSourceElt.elements("property");
for (Element property : properties) {
    String name = property.attributeValue("name");
    String value = property.attributeValue("value");
    if ("driver".equals(name)) driver = value;
    if ("url".equals(name)) url = value;
    if ("username".equals(name)) username = value;
    if ("password".equals(name)) password = value;
}
// 实例化我们在第六步写好的 UNPOOLED 数据源
DataSource dataSource = new LztUNPOOLEDDataSource(driver, url, username, password);

// 1.2 解析 TransactionManager (事务管理器)
Element txManagerElt = environment.element("transactionManager");
String txType = txManagerElt.attributeValue("type");
TransactionManager transactionManager = null;
if ("JDBC".equals(txType.toUpperCase())) {
    // 实例化我们在第五步写好的 JDBC 事务管理器，默认关闭自动提交
    transactionManager = new LztJDBCTransaction(dataSource, false);
}

// ===== 第二部分：解析所有的 Mapper SQL 映射 =====

Map<String, LztMappedStatement> mappedStatements = new HashMap<>();

// 获取所有的 <mapper resource="..."> 标签
List<Node> mapperNodes = root.selectNodes("//mapper");
for (Node mapperNode : mapperNodes) {
    Element mapperElt = (Element) mapperNode;
    String resource = mapperElt.attributeValue("resource");

    // 读取具体的 Mapper XML 文件 (例如 CarMapper.xml)
    InputStream mapperInputStream = Resources.getResourceAsStream(resource);
    Document mapperDoc = reader.read(mapperInputStream);
    Element mapperRoot = mapperDoc.getRootElement();

    // 为了简化，这里我们假设直接使用 namespace + id 作为 key，或者像你的测试代码里那样直接用 id。
    // 真正的 MyBatis 是用 namespace.id。为了匹配你的测试类 sqlSession.insert("insertCar", car);
    // 我们这里以 SQL 标签的 id 作为 Map 的 key。

    // 遍历 <select>, <insert>, <update>, <delete> 标签
    List<Element> sqlElements = mapperRoot.elements();
    for (Element sqlElement : sqlElements) {
        String id = sqlElement.attributeValue("id");
        String resultType = sqlElement.attributeValue("resultType");
        // 获取标签内部的文本 (即具体的 SQL 语句)
        String sql = sqlElement.getTextTrim();

        // 实例化我们在第七步写好的 LztMappedStatement，把 SQL 信息存进去
        LztMappedStatement mappedStatement = new LztMappedStatement(id, resultType, sql);
    }
}

```



```

        // 放入 Map 集合中
        mappedStatements.put(id, mappedStatement);
    }
}

// ===== 第三部分: 组装并返回 SqlSessionFactory =====

// 实例化第八步写好的默认工厂, 并将解析好的“事务管理器”和“SQL 字典”交由它保管
//return new LztDefaultSqlSessionFactory(transactionManager, mappedStatements);

// 1. 先把解析出来的数据塞进大管家
LztConfiguration configuration = new LztConfiguration(transactionManager, mappedStatements);

// 2. 实例化工厂, 并把大管家交给它
return new LztDefaultSqlSessionFactory(configuration);

} catch (Exception e) {
    e.printStackTrace();
    throw new RuntimeException("解析 XML 配置文件失败, 无法构建 SqlSessionFactory!", e);
}
}
}
}

```

第十三步：编写SqlSession类中的insert方法

其核心本质就是对原生 JDBC 代码的封装。

导包：

```

package lzt.lztbatis.core;
import lzt.lztbatis.mapping.LztMappedStatement;
import lzt.lztbatis.transaction.TransactionManager;

import java.sql.ResultSet;
import java.sql.ResultSetMetaData;

import java.lang.reflect.Method;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.util.ArrayList;
import java.util.List;

```

```

public class LztDefaultSqlSession implements SqlSession {

    private LztConfiguration configuration;

    public LztDefaultSqlSession(LztConfiguration configuration) {
        this.configuration = configuration;
    }

    // ... 省略前面的 commit, rollback, close 方法 ...

    @Override
    public int insert(String sqlId, Object pojo) {
        try {
            // 1. 从“大管家”中获取对应的 SQL 映射信息
            LztMappedStatement mappedStatement = configuration.getMappedStatements().get(sqlId);
            if (mappedStatement == null) {
                throw new RuntimeException("未找到对应的 SQL 映射, sqlId: " + sqlId);
            }
        }
    }
}

```

```

// 这是从 XML 中读取出来的原始 SQL，包含 #{xxx}
String lztSql = mappedStatement.getSql();

// 2. 解析 SQL：将 #{xxx} 替换为 ?，并按顺序记录占位符对应的属性名
List<String> propertyNames = new ArrayList<>();
String jdbcSql = lztSql;
int startIndex;
// 循环查找并替换 #{...}
while ((startIndex = jdbcSql.indexOf("#{") >= 0) {
    int endIndex = jdbcSql.indexOf("}", startIndex);
    // 提取出属性名，例如 "carNum"
    String propertyName = jdbcSql.substring(startIndex + 2, endIndex).trim();
    propertyNames.add(propertyName);
    // 将 "#{xxx}" 替换为 JDBC 认识的 "?"
    jdbcSql = jdbcSql.substring(0, startIndex) + "?" + jdbcSql.substring(endIndex + 1);
}

// 3. 获取数据库连接并预编译 SQL
// jdbcSql 现在变成了类似: insert into t_car values(null, ?, ?)
Connection connection = configuration.getTransactionManager().getConnection();
PreparedStatement ps = connection.prepareStatement(jdbcSql);

// 4. 利用反射给 ? 赋值
for (int i = 0; i < propertyNames.size(); i++) {
    // 取出第 i 个位置需要的属性名
    String propertyName = propertyNames.get(i);

    // 拼接 Getter 方法名 (例如: carNum -> getCarNum)
    String getMethodName = "get" + propertyName.substring(0, 1).toUpperCase() + propertyName.substring(1);

    // 通过反射获取该方法
    Method getMethod = pojo.getClass().getDeclaredMethod(getMethodName);

    // 执行该方法，从 pojo 对象中拿到具体的属性值
    Object propertyValue = getMethod.invoke(pojo);

    // 给 PreparedStatement 中的 ? 赋值 (JDBC 的索引从 1 开始计算)
    ps.setObject(i + 1, propertyValue);
}

// 5. 执行更新并返回受影响行数
int count = ps.executeUpdate();

// 注意：这里只关闭 Statement，Connection 要留给 TransactionManager 统一管理和关闭
ps.close();
return count;

} catch (Exception e) {
    throw new RuntimeException("执行 insert 异常", e);
}

}

@Override
public Object selectOne(String sqlId, Object parameter) {
    // TODO
    return null;
}
}

```

第十四步：编写SqlSession类中的selectOne方法

```
@Override
public Object selectOne(String sqlId, Object parameter) {
    try {
        // 1. 从“大管家”中获取对应的 SQL 映射信息
        LztMappedStatement mappedStatement = configuration.getMappedStatements().get(sqlId);
        if (mappedStatement == null) {
            throw new RuntimeException("未找到对应的 SQL 映射, sqlId: " + sqlId);
        }

        // 2. 解析 SQL: 针对 selectOne 这种单参数查询, 我们可以用正则直接替换 #{xxx} 为 ?
        String lztSql = mappedStatement.getSql();
        // 将类似 "select * from t_car where car_num = #{carNum}" 替换为 "select * from t_car where car_num = ?"
        String jdbcSql = lztSql.replaceAll("#\\{[^}]+\\}", "?");

        // 3. 获取数据库连接并预编译 SQL
        Connection connection = configuration.getTransactionManager().getConnection();
        PreparedStatement ps = connection.prepareStatement(jdbcSql);

        // 4. 给 ? 赋值 (针对你的测试代码传参 "111", 这里只有一个占位符)
        if (parameter != null) {
            ps.setObject(1, parameter);
        }

        // 5. 执行查询, 获取结果集
        ResultSet rs = ps.executeQuery();

        // 6. 处理结果集并封装 POJO 对象 (ORM 的灵魂: 反射)
        Object resultObj = null;
        if (rs.next()) { // selectOne 默认只处理第一条记录
            // 6.1 获取 XML 中配置的返回值类型 (例如 "com.lzt.pojo.Car")
            String resultType = mappedStatement.getResultType();

            // 6.2 利用反射, 根据全限定类名实例化一个空对象
            Class<?> resultClass = Class.forName(resultType);
            resultObj = resultClass.getDeclaredConstructor().newInstance();

            // 6.3 获取 ResultSet 的元数据 (非常重要: 能知道查询结果有多少列、列名是什么)
            ResultSetMetaData metaData = rs.getMetaData();
            int columnCount = metaData.getColumnCount();

            // 6.4 遍历每一列, 把列的值塞进刚才实例化的空对象中
            for (int i = 1; i <= columnCount; i++) {
                // 获取列名 (例如 "car_num" 或 "brand")
                String columnName = metaData.getColumnName(i);
                // 实际的 MyBatis 这里会有一个复杂的转换逻辑 (下划线转驼峰: car_num -> carNum)
                // 为了框架跑通, 我们这里先假设数据库列名和实体类属性名一致, 或者忽略大小写匹配
                String propertyName = columnName;

                // 拼接 Setter 方法名 (例如 "brand" -> "setBrand")
                String setMethodName = "set" + propertyName.substring(0, 1).toUpperCase() +
                    propertyName.substring(1);

                // 获取数据库中该列的值
                Object columnValue = rs.getObject(i);

                // 遍历该类的所有方法, 找到名字匹配的 Setter 方法进行调用并赋值
                Method[] methods = resultClass.getDeclaredMethods();
                for (Method method : methods) {
                    // 忽略大小写匹配方法名, 增加容错率
                    if (method.getName().equalsIgnoreCase(setMethodName)) {
                        // 相当于调用 car.setBrand("宝马X7")
                    }
                }
            }
        }
    }
}
```

```

        method.invoke(resultObj, columnValue);
        break;
    }
}
}

// 7. 释放 JDBC 资源
rs.close();
ps.close();

// 8. 返回组装好的完整对象
return resultObj;

} catch (Exception e) {
    throw new RuntimeException("执行 selectOne 异常", e);
}
}

```

• 目录树

The screenshot displays an IDE interface with a project directory tree on the left and a Maven `pom.xml` file open in the center.

Project Directory Tree (Left):

- 项目
 - lztbatis D:\Java_mybatis_study\lztbatis
 - src
 - main
 - java
 - lzt.lztbatis
 - core
 - LztConfiguration
 - LztDefaultSqlSession
 - LztDefaultSqlSessionFactory
 - SqlSession
 - SqlSessionFactory
 - SqlSessionFactoryBuilder
 - datasource
 - LztUNPOOLEDDataSource
 - mapping
 - LztMappedStatement
 - transaction
 - LztJDBCTransaction
 - TransactionManager
 - utils
 - Resources
 - resources
 - test
 - java
 - .gitignore
 - pom.xml

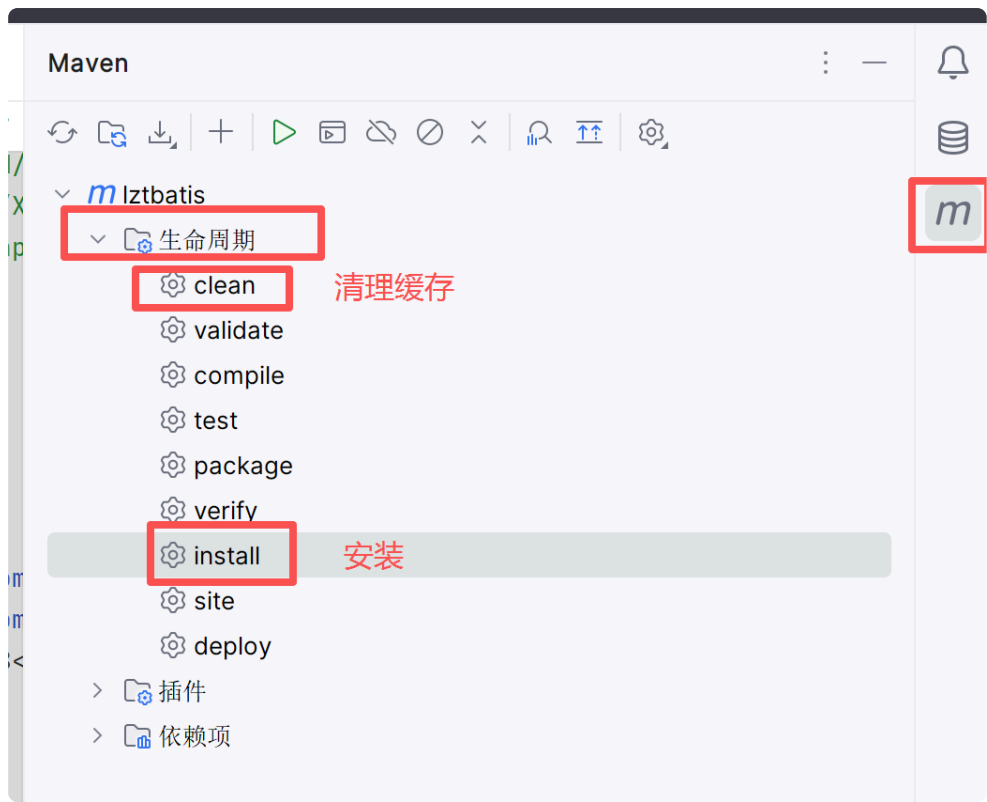
Maven pom.xml (Center):

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6
7      <groupId>lzt.lztbatis</groupId>
8      <artifactId>lztbatis</artifactId>
9      <version>1.0-SNAPSHOT</version>
10
11     <properties>
12         <maven.compiler.source>21</maven.compiler.source>
13         <maven.compiler.target>21</maven.compiler.target>
14         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15     </properties>
16
17     <dependencies>
18         <dependency>
19             <groupId>org.dom4j</groupId>
20             <artifactId>dom4j</artifactId>
21             <version>2.1.4</version>
22         </dependency>
23
24         <dependency>
25             <groupId>jaxen</groupId>
26             <artifactId>jaxen</artifactId>
27             <version>2.0.0</version>
28         </dependency>
29
30         <dependency>
31             <groupId>junit</groupId>

```

LztBatis使用Maven打包



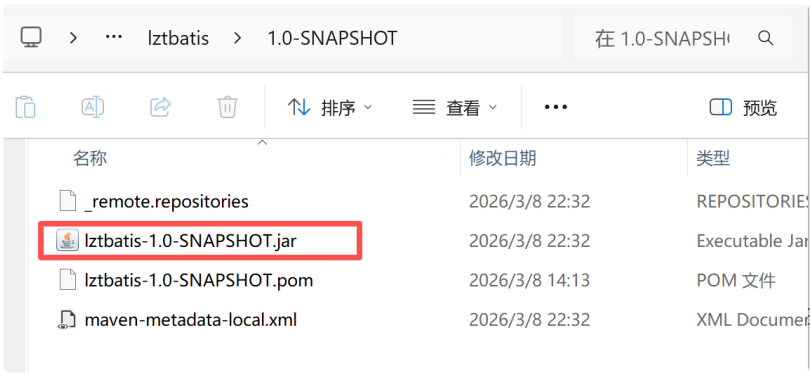
查看本地仓库中是否已经有jar包

- 核心查找路径解析
- Maven 本地仓库的目录结构是严格按照你在 pom.xml 里写的坐标来生成的：

```
<groupId>lzt.lztbatis</groupId>
<artifactId>lztbatis</artifactId>
<version>1.0-SNAPSHOT</version>
```

- GroupId (lzt.lztbatis) 会变成文件夹层级：lzt\lztbatis
- ArtifactId (lztbatis) 会变成下一级文件夹：lztbatis
- Version (1.0-SNAPSHOT) 会变成再下一级文件夹：1.0-SNAPSHOT

```
C:\Users\jack\.m2\repository\lzt\lztbatis\lztbatis\1.0-SNAPSHOT
```



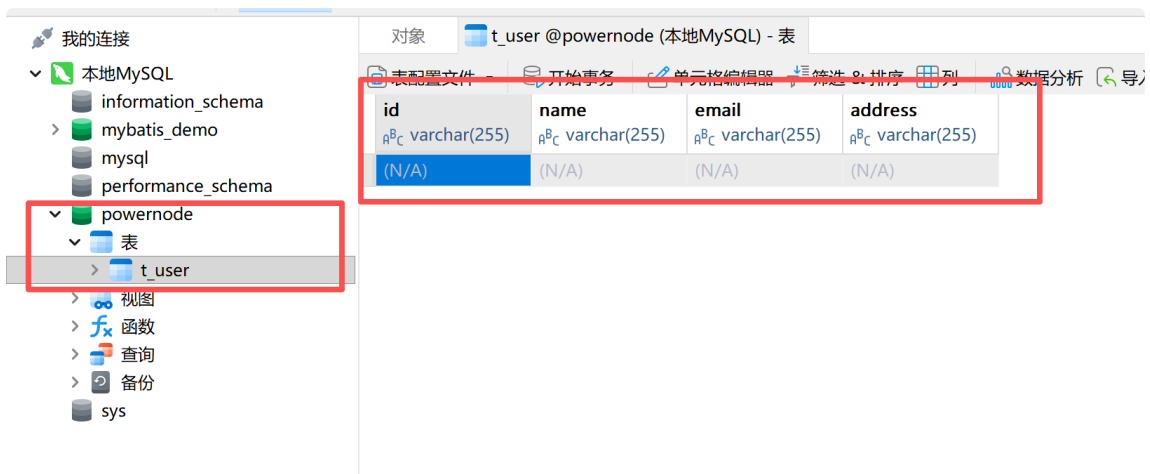
使用LztBatis

使用LztBatis就和使用MyBatis是一样的。

- 第一步：准备数据库表t_user

```
-- 如果没有 powernode 数据库，先创建它
CREATE DATABASE IF NOT EXISTS powernode DEFAULT CHARACTER SET utf8mb4;
USE powernode;

-- 创建 t_user 表
CREATE TABLE `t_user` (
  `id` varchar(255) PRIMARY KEY,
  `name` varchar(255),
  `email` varchar(255),
  `address` varchar(255)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```



字段	索引	外键	检查	触发器	选项	注释	SQL 预览				
名称	类型		长度	小数点	不是 null	虚拟	键	注释			
id	varchar	255			☒	☐	1				
name	varchar	255			☐	☐					
email	varchar	255			☐	☐					
address	varchar	255			☐	☐					

- 第二步：创建模块，普通的Java Maven模块：lztbatis-test



- 第三步：引入依赖
 - 编写：pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>lzt.lztbatis</groupId>
    <artifactId>lztbatis-test</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>lzt.lztbatis</groupId>
            <artifactId>lztbatis</artifactId>
            <version>1.0-SNAPSHOT</version>
        </dependency>

        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <version>8.0.33</version> </dependency>

        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

</project>
```

- 第四步：编写pojo类
 - lzt.lztbatis.pojo

```
package lzt.lztbatis.pojo;

/**
 * 对应数据库 t_user 表的实体类
 */
public class User {

    // 属性名与数据库表的字段名保持一致（或者符合下划线转驼峰的规则）
    private String id;
    private String name;
    private String email;
    private String address;

    /**
     * 无参构造方法
     * 框架在 selectOne 方法中，需要利用反射调用它来创建空对象：
     * resultClass.getDeclaredConstructor().newInstance();
     */
}
```

```

    */
    public User() {
    }

    /**
     * 全参构造方法，方便我们在测试 insert 的时候快速 new 一个对象
     */
    public User(String id, String name, String email, String address) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.address = address;
    }

    // --- 下面是 Getter 和 Setter 方法 ---
    // 你的框架在 insert 时会调用 Getter 取值，在 selectOne 时会调用 Setter 赋值

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    /**
     * 重写 toString 方法，方便我们在测试台直接打印出对象的内容，而不是内存地址
     */
    @Override
    public String toString() {
        return "User{" +
            "id='" + id + '\'' +
            ", name='" + name + '\'' +
            ", email='" + email + '\'' +
            ", address='" + address + '\'' +
            '}';
    }
}

```

- 第五步：编写核心配置文件：lztbatis-config.xml



```
<?xml version="1.0" encoding="UTF-8" ?>
<configuration>

    <environments default="development">

        <environment id="development">

            <transactionManager type="JDBC"/>

            <dataSource>
                <property name="driver" value="com.mysql.cj.jdbc.Driver"/>

                <property name="url" value="jdbc:mysql://localhost:3306/powernode?
useUnicode=true&characterEncoding=utf-8&useSSL=false&serverTimezone=UTC"/>

                <property name="username" value="root"/>

                <property name="password" value="20060329"/>
            </dataSource>
        </environment>
    </environments>

    <mapper resource="UserMapper.xml"/>

</configuration>
```

- 第六步：编写sql映射文件：UserMapper.xml



```
<?xml version="1.0" encoding="UTF-8" ?>
<mapper namespace="user">

    <insert id="insertUser">
        insert into t_user(id, name, email, address)
        values(#{id}, #{name}, #{email}, #{address})
    </insert>

    <select id="selectUserById" resultType="com.lzt.pojo.User">
        select * from t_user where id = #{id}
    </select>

</mapper>
```

- 第七步：编写测试类



```
package com.lzt.test;

import lzt.lztbatis.pojo.User;
import lzt.lztbatis.core.SqlSession;
import lzt.lztbatis.core.SqlSessionFactory;
import lzt.lztbatis.core.SqlSessionFactoryBuilder;
import lzt.lztbatis.utils.Resources;
import org.junit.Test;

import java.io.InputStream;

/**
 * LztBatis 框架的终极测试类
 */
public class UserTest {
```

```

/**
 * 测试向数据库插入一条 User 数据
 */
@Test
public void testInsertUser() {
    SqlSession sqlSession = null;
    try {
        // 1. 读取核心配置文件，获取输入流（调用你写的 Resources 工具类）
        InputStream inputStream = Resources.getResourceAsStream("lztbatis-config.xml");

        // 2. 创建 SqlSessionFactoryBuilder 对象
        SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();

        // 3. 解析 XML，构建出包含大管家 Configuration 的 SqlSessionFactory
        SqlSessionFactory sqlSessionFactory = builder.build(inputStream);

        // 4. 开启一个数据库会话（内部已经搞定了数据库连接和事务管理器）
        sqlSession = sqlSessionFactory.openSession();

        // 5. 准备要插入的数据对象（POJO）
        // 这里我们随机编造一个用户
        User user = new User("user-001", "张三", "zhangsan@powernode.com", "北京市海淀区");

        // 6. 执行 SQL！
        // 拿着 "insertUser" 这个 id 去 UserMapper.xml 里找 SQL，并利用反射把 user 里的属性塞进 SQL 里
        int count = sqlSession.insert("insertUser", user);
        System.out.println("成功插入了几条记录: " + count);

        // 7. 提交事务（极度重要！不提交的话数据进不去数据库）
        sqlSession.commit();

    } catch (Exception e) {
        // 发生异常时回滚事务
        if (sqlSession != null) {
            sqlSession.rollback();
        }
        e.printStackTrace();
    } finally {
        // 8. 释放资源，归还连接
        if (sqlSession != null) {
            sqlSession.close();
        }
    }
}

/**
 * 测试从数据库查询单条 User 数据并自动封装为对象
 */
@Test
public void testSelectUser() {
    SqlSession sqlSession = null;
    try {
        InputStream inputStream = Resources.getResourceAsStream("lztbatis-config.xml");
        SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();
        SqlSessionFactory sqlSessionFactory = builder.build(inputStream);
        sqlSession = sqlSessionFactory.openSession();

        // 执行查询 SQL！
        // 把刚才插入的 "user-001" 传过去。框架会执行 SQL，拿到 ResultSet，
        // 然后利用反射自动 new User()，并把查出来的列值通过 set 方法赋给它。
        Object result = sqlSession.selectOne("selectUserById", "user-001");

        System.out.println("从数据库查到的对象是: ");
        System.out.println(result);
    }
}

```

```

    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        if (sqlSession != null) {
            sqlSession.close();
        }
    }
}
}
}

```

第八步：运行结果

D:\Java21\bin\java.exe ...

成功插入了几条记录：1

对象 t_user @powernode (本地MySQL) - 表			
id	name	email	address
varchar(255)	varchar(255)	varchar(255)	varchar(255)
user-001	张三	zhangsan@powe	北京市海淀区

总结MyBatis框架的重要实现原理

- 四大核心运行机制：
 - 配置解析机制（XML + DOM4J）
 - 工厂与建造者模式（Factory & Builder）
 - JDBC 深度封装与事务控制
 - ORM 映射机制（反射机制的核心）

```
<?xml version="1.0" encoding="UTF-8" ?>
```

```

<mapper namespace="user">
  <insert id="insertUser">
    insert into t_user(id,name,email,address) values(#{id},#{name},#{email},#{address})
  </insert>
  <select id="selectUserById" resultType="com.powernode.godbatis.pojo.User">
    select id,name,email,address from t_user where id = #{id}
  </select>
</mapper>

```

思考两个问题：

- 为什么insert语句中 #{ } 里填写的必须是属性名？
 - 为了通过“字符串拼接”动态推断并调用对象的 Getter 方法。
- 为什么select语句查询结果列名要属性名一致？
 - 为了通过“数据库列名”动态推断并调用对象的 Setter 方法。